

LINUX ADMIN

JAI

DAY 01

- Introduction
- What you know?
- What we learn?
- Job importance
- Preparation
- Approach to success
- Conclusion

Introduction

- Let's take a moment for everyone to introduce themselves. Please share your name, current role, experience with Linux, and one fun fact about yourself.

What You Know?

- I'd like to get a sense of where we all stand. How many of you have knowledge with Linux before?
- Who has experience with distributions like Ubuntu, CentOS, or Fedora?
- How familiar are you all with basic Linux
- If you've faced any challenges or have specific questions about Linux, please make a note. We'll aim to address them during our sessions.

What We Learn?

- We'll dive deep into the Linux file system and its hierarchy.
- We'll discuss user and permissions management, ensuring you understand the ins and outs of access controls.
- We'll tackle system management, understanding the nuances between distributions.
- I'll guide you through system monitoring techniques and essential troubleshooting commands.
- Lastly, we'll explore setting up and securing vital services
- And many more (check contents)

Job Importance

- Linux is everywhere in today's IT infrastructure, from our smartphones to supercomputers.
- I'll share with you the pivotal role Linux plays in technologies like containerization and cloud platforms.
- We'll discuss why many enterprises opt for Linux, emphasizing its reliability and cost-effectiveness.
- Throughout our sessions, I'll pepper in real-world scenarios to highlight Linux's crucial role in various industries.

Preparation

- I suggest setting up a personal Linux lab environment. Virtualization tools like VirtualBox or VMware can be quite handy.
- There are several books, online courses, and websites I recommend.
- Practice makes perfect. I'll be providing hands-on tasks to help solidify your learning.

Approach to Success

- In my experience, the "learn by doing" approach is invaluable. While theory is essential, there's no substitute for hands-on practice.
- I always advocate for joining open-source projects and communities. They can be excellent learning platforms.
- Networking has been crucial in my journey. Connect with other Linux professionals, and consider joining Linux user groups in your area.
- The lessons I've learned from mistakes.

Conclusion

- We're at the brink of an exciting journey into the world of Linux administration. With the rapid evolution of IT, Linux remains a foundational skill.
- I encourage you to stay curious, keep learning, and practice regularly.
- Thank you for joining me today, and I'm looking forward to our sessions together!

Day 02

- Discuss what a Linux admin do
- Incident Management
- Change Management
- Daily Tasks

Daily Job

- **Morning Checkup:** I start my day by checking our **monitoring tools** for any alerts that might've come overnight. Tools like Nagios, Zabbix, or Prometheus give a quick overview of server health and performance.
- **Tickets:** Using our ITIL-based **Ticketing tool**, I review, prioritize, and address tickets raised by users or automated systems. This can range from access requests to service disruptions.
- **Updates and Patch Management:** I set specific times in a week to check for and apply system updates. This ensures our systems are protected against known vulnerabilities.
- **Collaboration:** I regularly collaborate with the application teams, database admins, and network teams to ensure seamless service delivery. We often have joint sessions to plan out system upgrades or troubleshoot complex issues.

Work Culture

- **Continuous Learning:** Given the dynamic nature of the tech world, I always allocate time to catch up on the latest trends, updates, and best practices in Linux administration.
- **Automation:** Wherever repetitive tasks are identified, I use tools like Ansible or Puppet to automate them, saving time and reducing human error.
- **Documentation:** Everything we do, from minor changes to major migrations, gets documented. This helps in training new team members and serves as a reference for future troubleshooting.

On-call Support

- While I have specific work hours, as a critical member of the infrastructure team, I'm also on a rotational on-call duty. This means if there's a significant incident outside of business hours, I'm alerted and expected to address it promptly.
- We use tools to manage on-call rotations and ensure that incidents are promptly addressed.

Weekly Tasks

- **Mondays:** Team meetings to discuss the week's agenda, any planned migrations, or system upgrades.
- **Wednesdays:** I allocate time for server backups and check our DR (Disaster Recovery) strategy's effectiveness. It's also when I usually schedule non-urgent patches and updates since it's mid-week.
- **Fridays:** Review performance metrics, check server logs, and ensure everything is ready for the weekend. It's also when we have a knowledge-sharing session, where team members share something new they've learned or a unique challenge they've encountered and solved.

Incident Management

- Definition and importance of incident management.
- Differentiating between an incident and a problem.
- Steps in the incident management process:
 - Identification.
 - Logging.
 - Categorization.
 - Prioritization.
 - Initial diagnosis.
 - Escalation (if required).
 - Resolution and recovery.
 - Closure.
 - Review.
- Importance of effective communication during incidents.
- Tools commonly used for incident tracking and management in Linux.

Incident Vs Problem

- **Incident:**

1. **Definition:** An incident refers to any unplanned interruption or reduction in quality of an IT service. It's an event that disrupts normal service operation and impacts the business or its user's productivity.
2. **Duration:** Typically short-term in nature. It exists until the service is restored back to its normal state.
3. **Goal:** The primary goal of incident management is to restore the normal service operation as quickly as possible and minimize the adverse impact on business operations.
4. **Examples:**
 1. A user being unable to access their email.
 2. A website going down.
 3. A specific application failing to launch.
5. **Management:** Incidents are typically managed through an incident management process, where they are logged, classified, prioritized, and then resolved.

Incident Vs Problem

- **Problem:**

1. **Definition:** A problem is the underlying cause of one or more incidents. It's a condition often identified as a result of multiple incidents that exhibit common symptoms.
2. **Duration:** Typically long-term in nature. Problems exist until the root cause is identified and a permanent solution is implemented.
3. **Goal:** The primary goal of problem management is to prevent incidents from happening and to minimize the impact of incidents that cannot be prevented.
4. **Examples:**
 1. A recurring memory leak in an application that causes it to crash periodically.
 2. A faulty server component that leads to intermittent disruptions.
 3. A software bug causing multiple users to experience the same issue.
5. **Management:** Problems are managed through a problem management process, which focuses on root cause analysis and finding permanent solutions. This might involve temporary workarounds until a full resolution can be implemented.

Incident Vs Problem

- While both incidents and problems deal with disruptions in IT service, they approach them from different angles. **Incidents** focus on the immediate relief and restoration of service, whereas **problems** dive deeper to identify and rectify the underlying causes to prevent future incidents.

Change Process

- Introduction to change management and its importance in ensuring stability.
- The lifecycle of change management:
 - Request for Change (RFC).
 - Assessment and evaluation.
 - Approval/rejection.
 - Planning and coordination.
 - Implementation.
 - Review and close.
- Rollback strategies in case of unsuccessful changes.
- Documentation and its importance in change management.
- Tools and platforms commonly used for change management in Linux environments.

Patch Management

- What is patch management and why is it crucial for system security and stability?
- The patch management process:
 - Vulnerability identification.
 - Patch testing in a controlled environment.
 - Scheduling and notifying users about planned maintenance.
 - Deployment of patches.
 - Verification and monitoring.
 - Documentation.
- Automating patch management using tools and scripts.
- Handling issues that might arise post-patch deployment and rollback strategies.
- Staying updated with patch releases and security advisories.

Patch Management

- **1. What is patch management and why is it crucial for system security and stability?**
 - **Definition:** Patch management is the process of acquiring, testing, and installing multiple code changes (patches) to an administered computer system, ensuring that systems are up-to-date and protected against known vulnerabilities.
 - **Importance:** Keeping systems patched ensures they are protected against known security vulnerabilities, which can be exploited by attackers to gain unauthorized access, launch denial of service attacks, or otherwise compromise the integrity or availability of services. Furthermore, patches often include bug fixes and performance improvements, enhancing system stability and performance.

Patch Management

- **a. Vulnerability identification.**

- Continuously monitor sources like the National Vulnerability Database, vendor advisories, and security news outlets.
- Use automated vulnerability scanning tools (e.g., Nessus, OpenVAS) to identify unpatched vulnerabilities in your environment.

- **b. Patch testing in a controlled environment.**

- Before deploying a patch system-wide, it's tested in a controlled, isolated environment that mirrors the production system.
- This ensures that the patch doesn't introduce new issues or conflicts with existing configurations.

- **c. Scheduling and notifying users about planned maintenance.**

- Coordinate with stakeholders to determine the best time for patch deployment, causing minimal disruption.
- Notify users in advance about potential downtime or limited functionality.

- **d. Deployment of patches.**

- Use automated deployment tools (e.g., Ansible, Puppet, WSUS for Windows) to roll out patches across the infrastructure.
- Always ensure you have backups before the deployment in case a rollback is necessary.

Patch Management

- **e. Verification and monitoring.**
 - After patch deployment, monitor systems for any irregularities or issues.
 - Use monitoring tools (e.g., Nagios, Zabbix) to ensure services are running optimally.
- **f. Documentation.**
 - Document details about the patch, including its source, reason for deployment, affected systems, and any issues encountered.
- **g. Automating patch management using tools and scripts.**
 - Tools like Red Hat Satellite (for RHEL systems) or Landscape (for Ubuntu) help in automating patch management.
 - Write custom scripts to automate routine tasks related to patching.
- **h. Handling issues that might arise post-patch deployment and rollback strategies.**
 - Always have a rollback plan in case a patch introduces critical issues.
 - Tools like Git or configuration management systems can help roll systems back to their pre-patched state.

Patch Management

- **i. Staying updated with patch releases and security advisories.**

- Subscribe to vendor mailing lists or security advisory services to stay informed about the latest patches.
- Regularly check vendor websites for updates.

Patch management is a critical aspect of IT administration, ensuring systems remain secure, stable, and efficient. With the constant emergence of new vulnerabilities and the rapid evolution of software, it's imperative to have a robust patch management process in place. Automation and documentation are key, ensuring both consistency and traceability in the patching process.

Day 03

Understanding Linux Concepts

- Where we Use Linux
- What is Linux?
- Unix Versus Linux
- Download, Install, and Configure
- How virtualization works
- Creating the First Virtual Machine

Where we use Linux?

1. Linux : Linux OS is ubiquitous and can be found in numerous applications.

2. Internet Access

1. WiFi routers typically have an embedded Linux copy for web management.

3. Entertainment Devices

1. TVs, smartphones, tablets, and e-books are powered by Linux OS.

4. Web and Search

1. Websites and platforms like Google Search use Linux.

5. Windows Recovery

1. Windows data recovery disks use Linux for the recovery process.

6. Home Appliances

1. Modern appliances with touchscreens are operating on Linux.

7. Self-Driving Cars

1. The testing and operating systems in self-driving cars primarily utilize Linux.

What is Linux?

1. Operating System:

1. Think of Linux as the brain of your computer. Just as our brain controls our body, Linux controls computer tasks.

2. Free & Open Source:

1. It's like a community-driven recipe. Anyone can use it, modify it, and share their own version.

3. Not Just One Version:

1. Imagine Linux as ice cream. There are many flavors (called "distributions") like Ubuntu, Fedora, and Debian. You pick what you like best!

4. Safe & Secure:

1. Linux is like a fortress. It's known for strong security.

5. Everywhere and Invisible:

1. You probably use Linux without knowing it! It's in smartphones (Android), cars, refrigerators, and even TVs.

6. Bridge Between You and the Computer:

1. If you're speaking English and your friend speaks French, you'd need a translator. Linux is like that translator between you (using apps) and the computer (doing the task).

Unix vs. Linux

1. Origin:

1. **Unix:** Originated in the late 1960s at AT&T Bell Labs.
2. **Linux:** Inspired by Unix, created by Linus Torvalds in 1991.

2. Price:

1. **Unix:** Most Unix variants are not free (e.g., AIX, HP-UX, Solaris).
2. **Linux:** Generally free. There are many distributions available for no cost (e.g., Ubuntu, Fedora).

3. Code Base:

1. **Unix:** Proprietary (owned by respective companies).
2. **Linux:** Open source. Anyone can view, modify, and distribute the code.

4. Hardware Platforms:

1. **Unix:** Often tailored for specific hardware platforms.
2. **Linux:** Versatile. Can run on a wide variety of hardware, from personal computers to servers to mobile devices.

5. Usage:

1. **Unix:** Often found in enterprise environments with specific hardware.
2. **Linux:** Used everywhere! From servers to smartphones (Android is based on Linux) to personal computers.

6. Development & Distribution:

1. **Unix:** Developed and maintained by respective companies.
2. **Linux:** Developed collaboratively by a global community. Many distributions are available (e.g., Debian, CentOS, Arch).

7. User Interface:

1. **Unix:** Originally command-line based, though graphical interfaces exist.
2. **Linux:** Both command-line and graphical interfaces are available (e.g., GNOME, KDE).

8. Software & Package Management:

1. **Unix:** Varies by vendor. Some use proprietary package management.
2. **Linux:** Many options, such as apt for Debian-based systems or yum for Red Hat-based systems.

Download, Install, and Configure

Distributions

1. Introduction

1. Discussion on various Linux operating systems and flavors.
2. Highlighting the main and popular distributions.

2. Red Hat

1. Enterprise-level operating system.
2. Used by major companies.
3. Charges for support.
4. Offers customized, pre-configured builds.
5. Not open-source.
6. Provides support for any OS issues.

3. CentOS (Cent Operating System)

1. Chosen OS for the course.
2. Similarity to Red Hat makes it a great learning platform.
3. Managed by Red Hat.
4. Completely free.
5. No official support.

4. Fedora

1. Popular and free.
2. Used by many companies.

5. SUSE

1. Highly customizable.
2. Companies, like VMware, run SUSE appliances.

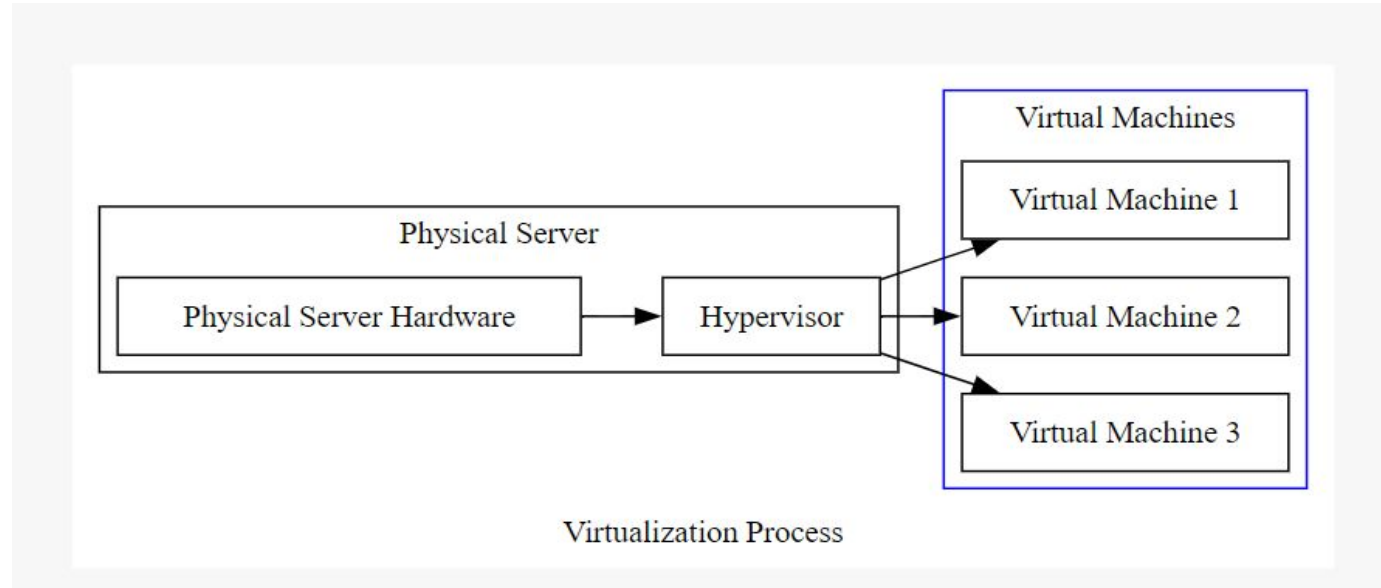
6. Debian

1. Popular for personal computers.

7. Ubuntu

1. Another major Linux distribution.

How virtualization works



- The "Physical Server" contains the "Physical Server Hardware" and the "Hypervisor".
 - The "Hypervisor" is responsible for managing multiple "Virtual Machines" (VM1, VM2, VM3) on the same physical hardware.
 - Each "Virtual Machine" operates as if it were a separate physical device, capable of running its own operating system and applications.
- This setup allows for efficient utilization of physical resources, isolation between different virtual machines, and flexibility in managing computing resources.

Hypervisors, "Type 1" and "Type 2"

- **Type 1 Hypervisors (Bare Metal)**
- **Direct Interaction with Hardware:** Type 1 hypervisors run directly on the physical hardware of the host machine, without an underlying operating system. This direct interaction allows for better performance and efficiency.
- **Examples:** VMware ESXi, Microsoft Hyper-V (when installed as standalone), Citrix Hypervisor, Red Hat Virtualization (KVM).
- **Use Cases:** Often used in enterprise environments for server virtualization, where performance, scalability, and resource isolation are critical.
- **Characteristics:**
 - Higher performance and efficiency.
 - Better security due to a smaller attack surface (no underlying OS).
 - Typically managed remotely.

- **Type 2 Hypervisors (Hosted)**

- **Runs on Top of an OS:** Type 2 hypervisors are installed on top of a host operating system, like Windows, Linux, or macOS. They are essentially software applications that allow virtualization.
- **Examples:** VMware Workstation, Oracle VM VirtualBox, Parallels Desktop.
- **Use Cases:** Commonly used for desktop virtualization, testing, development, and educational purposes. They are popular among developers and testers who need to run multiple operating systems or environments on a single physical machine.
- **Characteristics:**
 - Easier to set up and use, especially for individual users.
 - Performance is generally lower compared to Type 1, as it has to go through the host OS.
 - Offers more flexibility for desktop or personal use.

Hypervisor Providers

- Several companies and organizations provide hypervisor technology, each offering their own unique features and capabilities. Here are some of the notable hypervisor providers:

1. VMware

1. **Product:** VMware vSphere/ESXi
2. **Type:** Type 1 (Bare Metal)
3. **Description:** One of the most widely used and feature-rich hypervisors, known for its robustness and extensive support for various operating systems and hardware.

2. Microsoft

1. **Product:** Hyper-V
2. **Type:** Type 1 (Bare Metal)
3. **Description:** Integrated with Windows Server, Hyper-V is a popular choice for Windows-centric environments, offering good integration with other Microsoft products.

3. Citrix

1. **Product:** Citrix Hypervisor (formerly XenServer)
2. **Type:** Type 1 (Bare Metal)
3. **Description:** Known for its strong virtual desktop infrastructure (VDI) capabilities, Citrix Hypervisor is a commercial version of the open-source Xen.

4. Red Hat

1. **Product:** Red Hat Virtualization (based on KVM)
2. **Type:** Type 1 (Bare Metal)
3. **Description:** Built on the Kernel-based Virtual Machine (KVM), it's a robust solution especially in Linux environments and is known for its performance and scalability.

5. Oracle

1. **Product:** Oracle VM
2. **Type:** Type 1 (Bare Metal)
3. **Description:** Oracle VM is optimized for Oracle's own software products but also supports other applications, offering a good solution for enterprises using Oracle databases and applications.

1. IBM

1. **Product:** IBM PowerVM
2. **Type:** Type 1 (Bare Metal)
3. **Description:** Designed for IBM's Power Systems, PowerVM is known for its performance and reliability in enterprise environments.

2. Proxmox

1. **Product:** Proxmox VE
2. **Type:** Type 1 (Bare Metal)
3. **Description:** An open-source virtualization platform that combines KVM and containers, offering a flexible and scalable solution for virtual environments.

3. Nutanix

1. **Product:** Nutanix AHV
2. **Type:** Type 1 (Bare Metal)
3. **Description:** Part of the Nutanix hyper-converged infrastructure, AHV is designed for simplicity and integration with Nutanix's own ecosystem.

4. VMware (again)

1. **Product:** VMware Workstation/Fusion
2. **Type:** Type 2 (Hosted)
3. **Description:** Popular for desktop virtualization, allowing users to run multiple operating systems on a single physical machine, mainly used for testing and development purposes.

5. Oracle (again)

1. **Product:** Oracle VM VirtualBox
2. **Type:** Type 2 (Hosted)
3. **Description:** A free and open-source hypervisor for x86 hardware, suitable for desktop virtualization and widely used for development and testing environments.

Downloading and Installing Oracle VirtualBox

Creating the First Virtual Machine

Installing Linux can be done in several ways

- **1. Standard Installation (Using a Live CD/DVD or USB Drive)**
- **Download Linux Distribution:** Choose a Linux distribution (distro) like Ubuntu, Fedora, or Debian, and download the ISO file from its official website.
- **Create Bootable Media:** Use a tool like Rufus or Etcher to create a bootable USB drive or burn the ISO to a DVD.
- **Boot from Media:** Insert the USB drive or DVD into the computer and reboot. You may need to change the boot order in the BIOS/UEFI settings to boot from the USB/DVD.
- **Follow Installation Wizard:** Most distros provide an easy-to-follow installation wizard. You'll need to select language, time zone, keyboard layout, and partitioning options.
- **Complete Installation:** Follow the prompts to complete the installation. This usually involves setting up a user account and password.

Installing Inside Windows (Using a Virtual Machine)

- **Download a Virtualization Software:** Download and install virtualization software like VMware Workstation or Oracle VM VirtualBox.
- **Create a New Virtual Machine:** Open the virtualization software and create a new VM, allocating resources like disk space and memory.
- **Install Linux on the VM:** Use the Linux ISO file as the installation media in the VM and follow the installation wizard.

Installing on a Cloud Platform

- **Choose a Cloud Provider:** Providers like AWS, Google Cloud, or Azure offer Linux installations.
- **Create a Virtual Machine:** Use the cloud provider's interface to create a new VM instance and select a Linux distribution as the OS.
- **Access via SSH or Cloud Console:** Once the VM is running, you can access it via SSH or through the cloud provider's console.

Using a Network Installation (PXE Boot)

- **Setup a PXE Server:** This requires a server set up with network booting capabilities (PXE).
- **Boot from Network:** Computers on the same network can boot from this server and install Linux over the network.

Installing on a Raspberry Pi or Other Single-Board Computers

- **Download the Distro for ARM Devices:** Get a Linux version compatible with ARM devices (like Raspberry Pi OS).
- **Write to SD Card:** Use a tool to write the image to an SD card.
- **Boot the Raspberry Pi from SD Card:** Insert the SD card into the Raspberry Pi and power it up.

Linux Desktop

- Linux desktop environments are designed for end-users for daily computing tasks, similar to Windows or macOS. They come with a graphical user interface (GUI) and are equipped with software for everyday use.
- **Popular Linux Desktop Providers:**
 1. **Ubuntu:** Known for its ease of use and regular updates. Ubuntu is popular among new Linux users.
 2. **Fedora:** Offers the latest Linux features. It's known for its innovation and close relationship with the open-source community.
 3. **Linux Mint:** Based on Ubuntu, it's known for its simplicity and full multimedia support out-of-the-box.
 4. **Debian:** Known for its stability and vast software repositories.
 5. **Manjaro:** Based on Arch Linux, it offers cutting-edge software and a user-friendly experience.

Linux Server

- Linux server distributions are optimized for running server applications like web servers, database servers, and network management tools. They are designed for stability, security, and scalability.
- **Popular Linux Server Providers:**
 1. **Red Hat Enterprise Linux (RHEL):** A widely-used enterprise server platform known for its stability and support.
 2. **CentOS:** Community-driven free software based on Red Hat Enterprise Linux, known for its RHEL compatibility.
 3. **Ubuntu Server:** Offers the same ease of use as the desktop version but optimized for server use.
 4. **Debian Server:** Known for its robustness and a large number of packages.
 5. **SUSE Linux Enterprise Server:** Focused on enterprise environments, offering strong support and management features.

Day - 04

- **Using Essential Tools**
- **Logging in to Linux**
- **Linux prompts**
- **How to access linux Servers**
- **Using su and sudo**
- **Starting on the Command Line**
- **Using the Essential Linux Command Line Tools**
- **Getting Help with man**
- **Using Other Systems for Getting Help**

Logging in to Linux

1. To operate Linux, a user login is required.
2. Every Linux distribution mandates the creation of a user account.
3. During installation, current Linux distributions create an administrative user.
4. Ubuntu sets an administrative user as default.
5. Red Hat and its derivatives offer a choice during user creation.
6. If a root user is created on Red Hat, an additional user account with administrative privileges can also be made.
7. Without a root user, Red Hat will create an administrative user with sudo privileges.
8. The concept of "sudo" will be discussed in a subsequent classes.
9. While the root user is an option, it's not recommended due to its unlimited privileges.
10. Using the root user can be risky due to the potential for mistakes.
11. It's advisable to use administrative privileges only when necessary.
12. Users can log in via the console, a graphical user interface (GUI), or SSH for remote connections.
13. Using only the Linux GUI limits functionality.
14. Many servers don't run a GUI.
15. Mastery of the Linux shell is essential for effective Linux operation.
16. Logging into the system remains consistent regardless of the interface.
17. Installing a GUI and accessing the terminal from it provides full Linux shell access.
18. Relying solely on graphical programs is discouraged as they might be limited or unavailable on certain Linux machines.

Linux Prompts

- **Definition of Command Prompt:** A command prompt, also known as a prompt, is a short text at the start of the command line in a command line interface.
- **Appearance in Linux:** In a Linux terminal, the command prompt typically displays the username, hostname, and a prompt symbol. For example, `root@MyFirstLinuxVM#`.

Prompt Symbol Significance:

- `#` (hash sign) indicates the user is logged in as root.
- `$` (dollar sign) is shown when logged in as any user other than root.
- These symbols can be reconfigured.

Regaining the Prompt: If a command gets stuck and the prompt doesn't return, pressing `Ctrl + C` will interrupt the current command and return to the prompt.

Handling Stuck Commands:

- Examples of commands that can get stuck, like `cat` without arguments or incomplete commands.
- Using `Ctrl + C` to interrupt commands like `top` or when facing a redirection input (`>`).

How to access linux Servers

1. Accessing Linux and Unix Systems:

1. Linux and Unix systems can be accessed through a terminal, such as TeraTerm.

2. Operating Systems and Their Protocols:

1. Different operating systems use specific protocols or clients for remote access.

3. Windows Remote Access:

1. To connect to a Windows machine remotely, use a Remote Desktop Client (RDP client).
2. Requires the IP address of the target machine, followed by username and password.

4. Connecting to Linux Machines:

1. Linux machines can be accessed using clients like Putty, SecureCRT, or other terminal applications.
2. Requires the IP address, username, and password.
3. Supports SSH for Linux-to-Linux connections.

5. SSH Protocol for Linux:

1. From a Linux or Unix machine, SSH protocol can be used for remote connections.
2. Use the SSH command followed by the IP address of the remote Linux machine.

6. Demonstration of Clients:

1. Remote Desktop Client:

1. Found in Windows machines, accessible through Start and Search.
2. Enter the computer IP, username, and password.

2. PuTTY Client for Linux:

1. Enter the IP address in the client.
2. Default port is 22 for SSH.
3. Prompts for username and password upon connection.

Download and Install Putty

Network Commands

Changes in CentOS/RHEL 7:

- In CentOS/RHEL versions 5 or 6, the ifconfig command was used to check interface IP addresses.
- CentOS/RHEL 7 replaced the ifconfig command with the ip command.
- From CentOS/RHEL 7.5 onwards, the ifconfig command has been deprecated.

Using the IP Command:

- In CentOS/RHEL 7.5 and later, users should use the ip command.
- To list all available IPs or interfaces, use the command: ip addr.

Installing ifconfig in Newer Versions:

- To use ifconfig in newer versions of CentOS/RHEL, install the net-tools package.
- The command to install net-tools is: yum install net-tools.
- Ensure the machine has internet access before attempting the installation.

Demonstration of Commands:

- The ifconfig command lists all interfaces on the Linux machine.
- This IP is typically assigned in Oracle VirtualBox with Host-only Adapter settings.
- Changing to Bridged Adapter in VirtualBox settings alters the IP address.

Using the IP Command for Similar Output:

- The ip -a or ip addr command provides similar information to ifconfig.
- Focus on interface enp0s3 and its IP address 192.168.56.101.
- man ip can be used to explore different options available with the ip command.

• Future Usage in Lessons:

- We use both ifconfig and ip commands in future lessons.
- If ifconfig is unavailable, it's recommended to use ip -a or ip addr for similar information.

Day 07

Using su and sudo

1. Linux Privileges

1. Linux has two main types of user accounts: ordinary user and root (administrator level).
2. Root users have unrestricted access, while ordinary users have limited privileges.

2. Root User

1. Historically, Linux had the root user and the ordinary user.
2. Directly working as a root user is not recommended due to its unrestricted nature.
3. Modern Linux distributions may disable the root account or not set a password for it.
4. Ubuntu doesn't set a password for the root user during installation.
5. Red Hat Linux allows users to choose if they want to work as root during installation.

3. Using Sudo

1. It's safer to use "sudo" instead of directly working as root.
2. To use sudo, a user must be a member of the "wheel" group on Red Hat or "sudo" group on Ubuntu.
3. Sudo allows authorized users to run commands with enhanced privileges.
4. Example command to test sudo: `sudo ls /root`.

4. Using su -

1. "su -" is a command to open a shell as another user.
2. Using "su -" with the minus ensures full access to the target user environment.
3. Without any arguments, "su -" tries to open a root shell.
4. If the root user is disabled or doesn't have a password, "su -" won't work.
5. Using "su -" with a username opens a shell for that specific user.

5. Checking Sudo Access

1. The command `sudo ls /root` is useful to check if sudo is working for a user.
2. It lists files in the root user's home directory.
3. If the command prompts for a password and displays the contents, sudo is working for that user.
4. Without sudo, the command will result in a "permission denied" message.

Starting on the Command Line

1. Linux primarily operates using commands.
2. Linux is case sensitive.
 1. Commands in Linux are typically in lowercase.
 2. Using the wrong case can result in error messages.
3. Commands in Linux can be accompanied by options.
 1. Options modify the behavior of the command.
 2. There are long options and short options.
 1. Long options start with "--".
 2. Short options start with a single "-".
4. Examples:
 1. "uname--help" uses the long option "--help".
 2. "ls -lrt" uses three short options: L, R, and T.
5. To discover available options for a command, use "--help".
6. Demonstrative example:
 1. "ls --help" displays options for the "ls" command.
 2. "mkdir --help" displays options for the "mkdir" command.
 3. Using a single "-" instead of "--" for "help" results in an error.
 4. Typing commands in uppercase, like "LS", can result in a "command not found" error.
7. Important takeaway: In Linux, it's crucial to use the correct case (uppercase or lowercase) for commands

Using the Essential Linux Command Line Tools

1. Introduction to working with Linux, logging in, and user accounts.
2. Introduction to some of the most useful Linux commands for beginners.
3. "whoami" command: Displays the login name.
4. "ls" command: Lists files in the current directory.
 1. "ls -a": Shows hidden files.
 2. "ls -l": Displays a long listing of files.
 3. "ls -la": Combines both to show a long listing of hidden files.
5. "ip a" command: Displays the IP address.
6. "cat" command: Used to read the contents of a text file.
7. "passwd" command: Allows changing the current password.
8. "touch" command: Creates an empty file.
 1. Used to verify writability in specific locations.
 2. Can also be used to update the timestamp of a file.
9. "pwd" command: Prints the current working directory.
10. "cd" command: Changes the directory.
 - "cd /var/log": Navigates to the log directory.
 - "cd": Returns to the home directory.
- 11. "history" command: Displays the command history.

Getting Help with man

1. Introduction to "man" - the System Programmers Manual.
2. Historical context: Originated in the early 1970s with Unix systems.
3. Man pages were initially created on typewriters.
4. The Museum of Computer Science in Mountain View, California, has the original man pages.
5. Transition from printed pages to an online system.
6. Importance of the "man" command for Linux users.
7. Linux's open-source nature leads to multiple solutions for problems.
8. Dominance of "man" as the primary source for Linux command line information.
9. Usage of "man" to understand commands, configuration files, and more.
 - Command "man who am I"
10. Structure of a man page: Header line, command in uppercase, man sections.
11. Explanation of optional items in man pages using square brackets.
12. Usage of "dot dot dot" to indicate multiple options.
13. Different sections in man: Section 1 (end user commands), Section 8 (administrator commands), Section 5 (configuration files).
14. Search functionality in man pages using the "/" symbol.
15. Exiting man pages using the "Q" command.
16. Examples provided in complex man pages.
17. "See also" section for related commands.

Using Other Systems for Getting Help

1. Overview of help systems in Linux.
2. "Man" is the primary source for obtaining command information.
3. Advise to use "man" over Google due to accuracy and distribution-specific information.
4. "Command --help" provides short usage information.
5. Use "command --help" if you know the command but need the right option.
6. Use "man" if you're unfamiliar with the command and want to understand its function.
7. "/usr/share/doc" contains additional help for some commands.
8. "tldr" is a newer system that shows command usage examples.
9. "/usr/share/doc" has subdirectories for additional packages and programs.
10. Some directories in "/usr/share/doc" may contain useful additional documentation.
11. "tldr" is not always available in all distributions and may need to be installed.
12. "tldr" provides additional documentation if it has been written for the command.
13. "Man" should be the primary source of information.

- **Essential File Management Tools**

- Understanding the Linux File System Hierarchy
- Listing Files with ls
- Using Wildcards
- Copying Files with cp
- Working with Directories
- Using Absolute and Relative Paths
- Moving Files with mv
- Removing Files with rm

Understanding the Linux File System Hierarchy

1. Introduction to Linux Filesystem Hierarchy

1. Importance of files in Linux.
2. Everything in Linux is treated as a file.
3. Configuration is stored in ASCII text configuration files.
4. Devices are addressed using device files.
5. Linux filesystem is standardized as per the Linux Filesystem Hierarchy Standard.

2. Standardized Directories in Linux

1. `/usr` directory: Contains program files.
2. `/var` directory: Used by services to dynamically create files.
 1. `/var/log`: Contains log files.
 2. `/var/cache`: Used for caching.
3. `/etc` directory: Contains configuration files.
4. Filesystem Hierarchy Standard (FHS) is defined by the Linux Foundation.
5. Regular users have write access to `/home` and `/tmp` directories.

1. Exploring the Linux Filesystem

1. "man hier": Provides information about the filesystem hierarchy.
2. "/bin" and "/sbin": Contains binary files (commands) for users and system respectively.
3. "/lib" and "/lib64": Contains libraries related to binaries.
4. "/boot": Contains files required to boot the system.
5. "/dev": Contains device files.
6. "/etc": Contains text-based configuration files.
7. "/home": Contains user home directories.
8. "/media" and "/mnt": Used for mounting devices.
9. "/opt": Optional directory.
10. "/proc": Provides an interface to the kernel.
11. "/root": Home directory for the root user.
12. "/run": Contains temporary files.
13. "/srv": For services.
14. "/sys": For managing hardware.
15. "/tmp": For temporary files.
16. "/usr": Contains a hierarchy for binaries and libraries.
 1. "/usr/local": For site-specific programs.
17. "/var": For dynamically created content.

2. Linux Foundation and FHS

1. Linux Foundation is the organization behind Linux.
2. All Linux distributions are members of the Linux Foundation.
3. FHS defines the structure of the filesystem.

Listing Files with ls

1. Introduction to the ls utility.
2. Use of pwd to identify the current directory.
3. Purpose of ls: listing files and directories.
4. Options of ls:
 1. "ls -a": Reveals hidden files.
 2. "ls -l": Offers a detailed listing of files.
 3. "ls -ld": Displays directory properties without its contents.
 4. "ls -lrt": Presents files in a time-sorted manner, with the most recent at the end.
5. Demonstration of "ls -l" to display directory properties.
6. "ls -a" reveals hidden configuration files.
7. "ls -lrt" in the var log directory displays logs from various processes.
8. Log naming differences between CentOS and Ubuntu.
9. "ls -l /etc" displays contents of the etc directory.
10. "ls -ld /etc" showcases properties of the etc directory, crucial for permission management.

Introduction to Wildcards (Globbing)

- Wildcards help in pattern matching for files.
- Commands like 'ls' support wildcards.
 1. Example: "ls a*" shows files starting with an 'a'.

1. Different Wildcard Patterns

2. "ls a?*": Matches files with at least two characters.
3. "ls a[nm]": Matches files with either 'n' or 'm' after 'a'.
4. "ls a[a-e]": Matches files with characters between 'a' and 'e' after 'a'.

2. Working with Groups and Ranges

2. Commands can create multiple items using groups and ranges.
 3. Example: "touch file{1..100}" creates 100 files.
 4. "mkdir /data/{sales,account}" creates two directories.
- "mkdir /data/{sales,account}" tries to create two directories.
 - Errors encountered:
 - Cannot create a directory within a non-existent directory.
 - Permission denied when trying to write in the root of the file system.
 - Solution: Use "-p" to create parent directories and "sudo" for permissions.

Copying Files with cp

- **Introduction to the CP Command**
- **"CP" stands for copy.**
- **Basic Usage:**
 - Command: "cp somefile.txt somedir"
 - Description: This command copies the file somefile.txt to the directory somedir.
- **Understanding the Basic Structure of CP**
- **Copying with New Name if Directory Doesn't Exist:**
 - Command: "cp somefile.txt newname"
 - Description: If newname doesn't exist as a directory, somefile.txt will be copied and renamed to newname.
- **Ensuring Destination is a Directory:**
 - Command: "cp somefile.txt somedir/"
 - Description: Adding a slash at the end of somedir/ ensures that the destination is a directory.
- **Advanced CP Options**
- **Copying All Files Including Hidden Ones:**
 - Command: "cp -a sourcedir/ destdir/"
 - Description: The -a option is used to copy all files from sourcedir to destdir, including hidden ones.
- **Using Wildcard for Hidden Files:**
 - Command: "cp sourcedir/*. * destdir/"
 - Description: The .* wildcard includes hidden files in the copy process.
- **Recursive Copy (Including Subdirectories):**
 - Command: "cp -R sourcedir/ destdir/"
 - Description: The -R option is used to copy files recursively, including all subdirectories and files within sourcedir to destdir.

Overview of directories and their manipulation

1. Changing Directories:

1. Use "cd" to change directories.
2. Example: "cd tmp" changes the current directory to tmp.
3. "pwd" command is used to display the current directory.

2. Creating Directories:

1. "mkdir" is used to create a new directory.
2. Example: "mkdir data" creates a directory named data.
3. To view properties of a directory, use "ls -ld".
4. "mkdir /files" results in a permission error because it tries to create a directory in the root, which requires special permissions.
5. Avoid creating too many files in the root directory as it's a bad practice.
6. "mkdir files/data" without the -p option fails because the parent directory "files" doesn't exist.
7. Using "mkdir" with the "-p" option creates the entire path.

3. Removing Directories:

1. "rmdir" is used to remove an empty directory.
2. Example: "rmdir data" removes the data directory.
3. "rmdir files/data" removes the data directory inside the files directory.
4. "rmdir" with the "-p" option removes the entire path.

Using Absolute and Relative Paths

- **Types of Paths:**

- **Absolute Path:**

- Contains the full name from the root directory to a file.
- Example: `"/var/log/messages"`.
- Advantage: Clear and specific.

- **Relative Path:**

- Related to the current directory.
- Example: If current directory is set to `/var`, then `"log/messages"` points to `"/var/log/messages"`.
- Use `".."` to move one directory up. For instance, if current directory is `"/var/cache"`, then `"../log/messages"` points to `"/var/log/messages"`.
- Tip: Prefer using absolute paths to avoid confusion.

Moving Files with mv

- **Introduction to the MV Command**
- **"MV" stands for move.**
- **Basic Usage:**
 - Command: `mv sourcefile.txt destinationdir`
 - Description: Moves the file "sourcefile.txt" to the directory "destinationdir".
- **Understanding the Basic Structure of MV**
- **Moving and Renaming a File:**
 - Command: `mv oldfilename.txt newfilename.txt`
 - Description: Renames "oldfilename.txt" to "newfilename.txt".
- **Moving Multiple Files to a Directory:**
 - Command: `mv file1.txt file2.txt destinationdir`
 - Description: Moves "file1.txt" and "file2.txt" to "destinationdir".
- **Advanced MV Options**
- **Interactive Move:**
 - Command: `mv -i sourcefile.txt destinationdir`
 - Description: Prompts before overwriting any existing file in "destinationdir".
- **Verbose Mode:**
 - Command: `mv -v sourcefile.txt destinationdir`
 - Description: Provides detailed output of the move operation.
- **No Overwrite:**
 - Command: `mv -n sourcefile.txt destinationdir`
 - Description: Prevents overwriting an existing file in "destinationdir".
- **Moving Directories:**
 - Command: `mv sourcedir destinationdir`
 - Description: Moves the entire "sourcedir" directory to "destinationdir".

Removing Files with rm

1. Removing Files and Directories:

1. "rmdir" is used to remove empty directories.
2. If the directory is not empty, "rm" command is used.
3. "rm" can remove both files and directories with contents.

2. Options with "rm" Command:

1. "rm -i" is used to increase the level of protection by confirming every single file that you want to remove.
2. "rm -f" is used to forcefully remove files or directories without asking for confirmation.
3. "rm -rf" is a strong command used to remove an entire directory tree without asking for confirmation.
4. "rm -rf / --no-preserve-root" is the strongest command that will remove everything and should never be used.

Quiz

- 1. What command is used to list files and directories in Linux?
 - **a) ls**
 - b) dir
 - c) list
 - d) show
- 2. In the Linux file system hierarchy, what does the /etc directory contain?
 - a) User personal files
 - **b) System configuration files**
 - c) Temporary files
 - d) Installed software
- 3. How do you use wildcards to list all .txt files in a directory?
 - a) ls *.txt
 - b) ls .txt*
 - c) ls txt.
 - d) ls *txt

- 4. Which command is used to copy files?
 - a) mv
 - b) cp
 - c) rm
 - d) dir
- 5. How do you create a new directory in Linux?
 - a) mkdir newdir
 - b) mkfile newdir
 - c) newdir create
 - d) dircreate newdir
- 6. What is the difference between absolute and relative paths?
 - a) Absolute paths start from the root directory, while relative paths start from the current directory.
 - b) Absolute paths are longer, while relative paths are shorter.
 - c) There is no difference.
 - d) Absolute paths are for files, while relative paths are for directories.

- 7. What does the command `mv file1.txt /home/user/Documents` do?
 - a) Copies file1.txt to /home/user/Documents
 - b) Moves file1.txt to /home/user/Documents
 - c) Deletes file1.txt from /home/user/Documents
 - d) Renames file1.txt to /home/user/Documents
- 8. Which command is used to remove files in Linux?
 - a) del
 - b) erase
 - c) rm
 - d) remove
- 9. What happens if you use the `rm -r` command on a directory?
 - a) It lists all files in the directory.
 - b) It copies the directory.
 - c) It moves the directory to another location.
 - d) It removes the directory and its contents.
- 10. How can you copy all contents of a directory, including hidden files, to another directory?
 - a) `cp -a sourcedir/ destdir/`
 - b) `cp sourcedir/* destdir/`
 - c) `mv sourcedir/ destdir/`
 - d) `ls -a sourcedir/ > destdir/`

- Answers:

- a) ls

- b) System configuration files

- a) ls *.txt

- b) cp

- a) mkdir newdir

- a) Absolute paths start from the root directory, while relative paths start from the current directory.

- b) Moves file1.txt to /home/user/Documents

- c) rm

- d) It removes the directory and its contents.

- a) cp -a sourcedir/ destdir/

Do it Yourself

- **Lab Tasks**

1. Essential File Management Tools

- Task: Identify and list at least three file management tools available in Linux.

2. Understanding the Linux File System Hierarchy

- Task: Explore the root directory (/) and list the purposes of at least five subdirectories.

3. Listing Files with ls

- Task: Use the ls command to list all files, including hidden ones, in your home directory.

4. Using Wildcards

- Task: Find all .log files in the /var/log directory using wildcards.

5. Copying Files with cp

- Task: Copy the /etc/passwd file to a backup directory /backup.

6. Working with Directories

- Task: Create a new directory named Projects in /home/username and within it, create two subdirectories: Active and Archived.

7. Using Absolute and Relative Paths

- Task: Move a file report.txt from the current directory to /home/username/Documents using both relative and absolute paths.

8. Moving Files with mv

- Task: Rename a file old_config.conf to new_config.conf in the /etc/nginx directory.

9. Removing Files with rm

- Task: Remove all .bak files in the /home/username/Backup directory.

Solutions

1. Essential File Management Tools

- Solution: Common file management tools include ls, cp, mv, rm, mkdir, touch, find, and grep.

2. Understanding the Linux File System Hierarchy

- Solution:
- /bin: Essential user command binaries.
- /etc: System configuration files.
- /home: User home directories.
- /var: Variable data like logs.
- /usr: Secondary hierarchy for user data.

3. Listing Files with ls

- Solution: "ls -a ~"

4. Using Wildcards

- Solution: "ls /var/log/*.log"

- 5. Copying Files with cp

- Solution: "cp /etc/passwd /backup/"

6. Working with Directories

- Solution:
- "mkdir /home/username/Projects"
- "mkdir /home/username/Projects/Active"
- "mkdir /home/username/Projects/Archived"

7. Using Absolute and Relative Paths

- Solution:
- Absolute Path: "mv report.txt /home/username/Documents/"
- Relative Path (assuming /home/username is the current directory): "mv report.txt Documents/"

8. Moving Files with mv

- Solution: "mv /etc/nginx/old_config.conf /etc/nginx/new_config.conf"

9. Removing Files with rm

- Solution: "rm /home/username/Backup/*.bak"
- These tasks and solutions are designed to simulate real-world scenarios that a Linux administrator might encounter, providing practical experience in managing files and directories on a Linux system.

Advanced File Management Tools

- Understanding Hard and Symbolic Links
- Managing Hard and Symbolic Links
- Finding Files with find
- Using which and locate
- Archiving Files with tar
- Managing File Compression
- Mounting File Systems

Understanding Hard and Symbolic Links

- Soft links and hard links are essential concepts in Linux, particularly for system administration. Understanding them is crucial for managing files and directories effectively.

Understanding Inodes:

- An inode is a unique identifier for a file or directory in a filesystem. It's like a reference number that the computer uses to identify and access files, instead of using human-readable names.
- Example: If you create a file named example.txt, the system uses an inode to reference it internally.

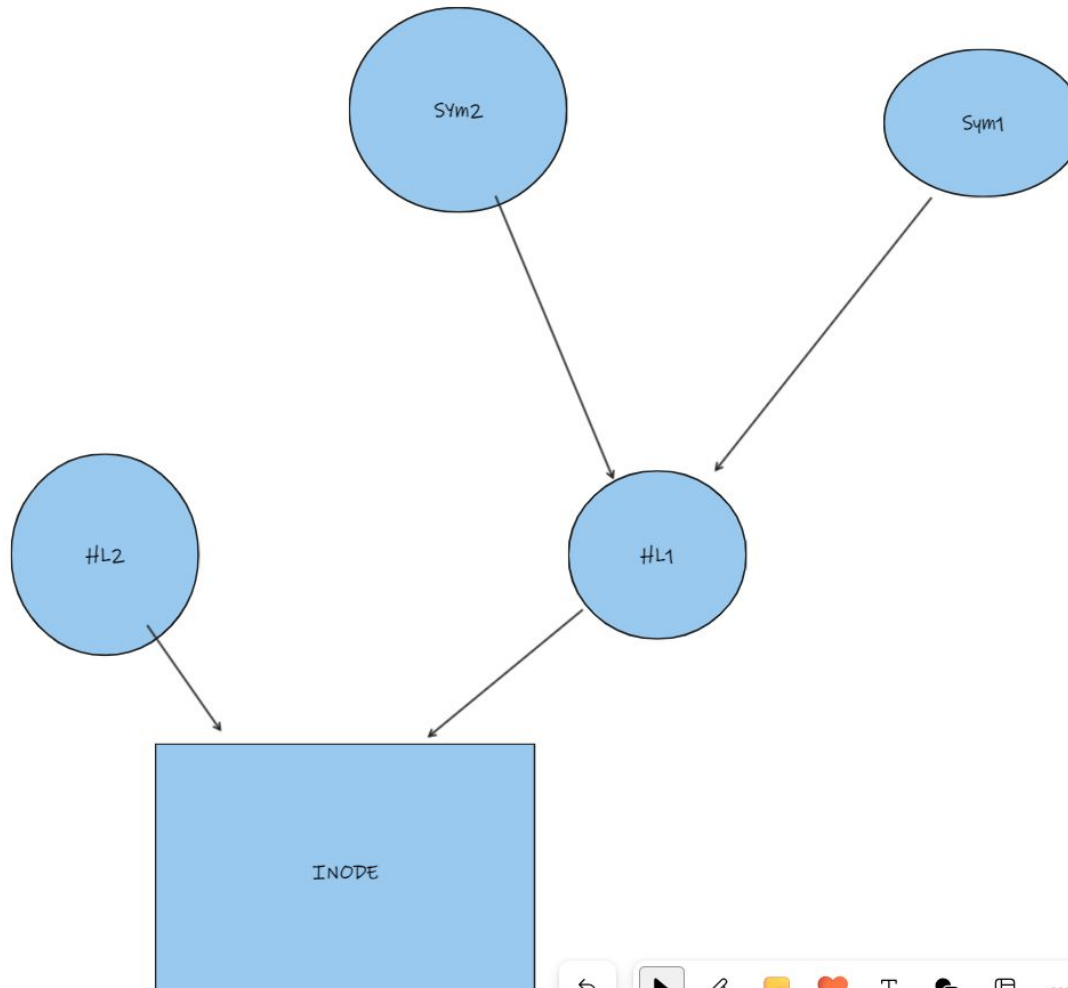
Symbolic Links (Soft Links):

- A soft link is like a shortcut to another file. It points to the file's name, not directly to its inode.
- Creation: Use “ln -s” source_file link_name to create a soft link.
- Behavior: If the original file is deleted or renamed, the soft link becomes invalid or "broken."
- Example: Creating a soft link link_to_example to example.txt. If example.txt is removed, link_to_example won't work anymore.

Hard Links:

- A hard link directly points to the inode of a file. It acts almost like a duplicate of the original file, but doesn't take up additional space.
- Creation: Use “ln” source_file link_name to create a hard link.
- Behavior: If the original file is deleted, the hard link remains functional since it points to the inode.
- Example: Creating a hard link hard_link_example to example.txt. If example.txt is deleted, hard_link_example still retains the content.

Hard Links and Symbolic Links



1. Understanding Inodes:

1. In Linux file systems, each file has an inode. An inode contains all the metadata about a file, except for its name.
2. An inode is linked to data blocks, which are the physical units of storage on a disk where the file's data is stored.

2. Concept of Links in Linux:

1. Links in Linux allow you to assign multiple names to a file.

3. Hard Links:

1. A hard link is a direct reference to a file's inode.
2. It can be created in a directory and allows you to access the inode and, through it, the file's data blocks.
3. Linux allows multiple names (hard links) for the same file, with no hierarchical relationship between these names.
4. Removing one hard link does not affect the existence of the other hard links.
5. Restrictions:
 1. Hard links must be on the same filesystem; cross-device hard links are not allowed.
 2. Hard links cannot be created for directories.

4. Symbolic Links (Symlinks):

1. Symbolic links, or symlinks, refer to a file by its name, not directly to the inode.
2. They are often referred to as soft links, but the correct term is symbolic links.
3. You can have multiple symbolic links referring to the same file name.
4. Dependency:
 1. If the original file (the name to which the symlink points) is removed, the symbolic link becomes invalid or "broken."
 2. Broken symlinks can only be fixed by restoring the original file.

5. Key Differences:

1. Hard links link directly to the inode of a file, while symbolic links refer to a file by its name.
2. Hard links are independent of each other; deleting one doesn't affect others. Symbolic links depend on the existence of the original file name they point to.

Demonstration with Commands:

Symbolic Links:

- `pwd --` shows `[/home/jai]`
- `touch India`
- `cd /tmp`
- `pwd`
- `ln -s /home/jai/India`
- `ls -lrt --` creates a link with India -> `/home/jai/India -->` File permission shows as `"l"[link]`
- `cd`
- `pwd`
- `echo "India won 2023 Cricket world cup" > India`
- `cat India`
- `cd /tmp`
- `cat India --` It will have same content
- `cd`
- `pwd`
- `ls -lrti --` Inode numbers will be different for sym links
- `ls -lrti /tmp/India`
- `rm India --` From home
- `cd /tmp`
- `cat hulk --` No such file or Directory -- as source is removed

Hard Links:

- cd
- touch Kohli
- echo "Kohli is the Top scorer in 2023 cricket world cup" > Kohli
- cat Kohli
- cd /tmp
- ln /home/jai/Kohli
- ls -ltr -- It shows link is created with name Kohli (There wont be no pointing to another file)
- cat Kohli
- cd
- echo "Highest Centuries in cricket world" >> Kohli
- cat Kohli
- cd /tmp
- cat Kohli
- cd
- rm Kohli
- cd /tmp
- ls -lrt -- Still File exists and also content

Different Names and Inode checking

- root@Jai:~# touch kohli
- root@Jai:~# echo "Highest centuries in oneDays" > kohli
- root@Jai:~# cd /tmp
- root@Jai:/tmp# ln /root/kohli
- root@Jai:/tmp# ls
- root@Jai:/tmp# ls -li kohli

```
54772 -rw-r--r-- 2 root root 29 Nov  6 21:44 kohli
```

- root@Jai:/tmp# ls -li /root/kohli

```
54772 -rw-r--r-- 2 root root 29 Nov  6 21:44 /root/kohli
```

- root@Jai:/tmp# ln /root/kohli kohli-hl; root@Jai:/tmp# ls
- root@Jai:/tmp# echo "writing in diff file name for kohli" >> kohli-hl
- root@Jai:/tmp# cd
- root@Jai:~# cat kohli

Finding Files with find

- Introduction to the find Command

Complexity & Utility: The find command is a complex utility in Unix-like systems, and I'll be covering its various aspects. It's powerful for searching files in a directory hierarchy.

- Basic Usage of find

Using the find Command:

- Imagine you're in your home directory, say /home/user, and you're looking for a file named example.txt which you thought was in this directory.
- The syntax to use find is: find [path] -name [filename].
- For example, to search for example.txt from the current directory, you'd use:

```
find . -name "example.txt"
```

Demonstration: I'll start with a basic command: find / -name "SampleFile". This command searches for files named "SampleFile" in the root directory.

Handling Errors: As a non-root user, you might encounter many error messages. To suppress these, add 2> /dev/null to the command, making it find / -name "SampleFile" 2> /dev/null.

- Will Discuss /dev/null in coming classes

Using Wildcards

- Extended Search: To broaden the search, use wildcards. For instance, `find / -name "*SampleFile*"`. This finds files containing "SampleFile" in their names, like "MySampleFile.txt".

Searching by User

- Find Files by Owner: Use `find / -user [username]` to locate files owned by a specific user. For example, "`find / -user hanu`" finds files owned by Hanu.
- Using sudo: Running this command with `sudo` is often necessary to avoid permission errors, as in `sudo find / -user hanu`.
- Finding Large Files
- Size-based Search: To find files larger than a certain size, use "`find / -size +2G`" for files over 2 gigabytes.

Using which and locate

Using find Command:

- **Powerful and Thorough:** The find command is potent, capable of searching for any file regardless of its location or properties.
- **Drawback:** It can be slow as it thoroughly explores the file system.
- **Example:** Imagine you need to find a file with specific properties deep within your file system - find would be the tool to use, albeit with some patience for its thorough search.

Using locate Command:

- **Speed Advantage:** locate is much faster compared to find.
- **Database Dependency:** It relies on a database created by updatedb, which needs regular updates to stay current.
- **Limitation:** If the database isn't updated, locate won't find newly created files.
- **Example:** After creating a file named "example.txt" using touch example.txt, running locate example.txt might not find it immediately if updatedb hasn't been run since the file was created.

Using which Command:

- **Specific Use Case:** which is ideal for finding the location of executables in the directories listed in the \$PATH variable.
- **Limitation:** It only searches in \$PATH directories, so it won't find files outside those paths.
- **Example:** To find the location of the passwd binary, you would use which passwd. However, for a file like "script.sh" not in \$PATH, which script.sh would not yield a result.

Demonstration:

locate Command Demonstration:

- Created a file named "jai" using touch jai.
- Attempted to find it using locate jai but failed initially due to an outdated database.
- After updating the database with sudo updatedb, locate jai successfully found the file.

which Command Demonstration:

- Showed how which passwd successfully locates the passwd binary.
- Highlighted the limitation of which by showing it couldn't find a file named "jai" as it's not in the \$PATH directories.

Which demonstration

- root@Jai:~# echo \$PATH
- /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
- root@Jai:~# touch /usr/bin/naperujai
- root@Jai:~# which naperujai
- root@Jai:~#
- root@Jai:~# cd /usr/bin/
- root@Jai:/usr/bin# chmod +x naperujai
- root@Jai:/usr/bin# cd
- root@Jai:~# which naperujai
/usr/bin/naperujai

Understanding the tar Command in Linux

- A staple in the Linux world. Initially designed for tape archiving, tar stands for Tape Archiver. Though we're not using tapes for backups anymore, tar remains crucial for creating archive files.

Basic Uses of tar:

- Compression, Extraction, and Listing: The primary functions of tar are to compress files into an archive, extract them, or list their contents.

Creating an Archive (tar -cvf):

- Example: `tar -cvf archive.tar /home/user/documents`
- This command creates (c) an archive with verbose (v) output, specifying the file name (f). The .tar extension is recommended for clarity, though optional in Linux.

Extracting from an Archive (tar -xvf):

- Example: `tar -xvf archive.tar`
- This extracts (x) files from an archive. By default, extraction happens in the current directory. Use the -C option to specify a different directory.

Viewing Archive Contents (tar -tvf):

- Example: `tar -tvf archive.tar`
- This lists (t) the contents of the archive, showing relative paths stored by default.

Identifying File Types with file:

- If an archive lacks an extension, use file to identify its type.
- Example: `file unknownfile`

Compression in tar:

- By default, tar doesn't compress files. In a subsequent discussion, we'll explore how to add compression to tar archives.

UNIX Styles in tar:

- BSD Style (without a minus): `tar cvf`
- System-V Style (with a minus): `tar -cvf`
- Linux, mainly based on System-V, typically uses the minus, but tar, being a BSD-origin command, allows both styles.

Practical Examples:

Creating an Archive:

- `sudo tar cvf /tmp/files.tar /etc /home`
- Notice the use of a temporary directory (`/tmp`) to avoid recursive archiving.

Inspecting an Archive:

- `ls -l /tmp/files.tar`
- `file /tmp/files.tar`: Confirms it's a POSIX tar archive.

Extracting :

- `sudo tar xvf /tmp/files.tar -C /root`
- This extracts the archive to the `/root` directory, showcasing the utility of relative paths.

By understanding these aspects of tar, you can efficiently manage archives in Linux, whether you're backing up files, transferring data, or simply organizing your file system.

Managing File Compression

Introduction to Common Compression Utilities:

- gzip: The most commonly used compression utility in Linux. It strikes a balance between compression ratio and time to create the archive.
- Example: Use gzip with the tar command by adding the -z option.

Alternative Utilities:

- bzip2 (-j): Offers better compression than gzip but takes longer to compress. Used with tar by adding the -j option.
- xz (-J): A relatively new utility providing the best compression but is significantly slower. In tar, it's used with the uppercase -J option.
- zip: A legacy utility with Windows-compatible syntax, less relevant now as Windows can read Linux native formats like gzip.

Compression Ratio and Time Considerations:

- The primary factors to consider are the compression ratio and the time it takes to compress.
- Example: Use the time command to measure how long compression takes.

Demonstration with tar and time:

- gzip Compression:
- File Extension: .tar.gz or .tgz.
- Command: `time tar -czf /tmp/files.tar.gz /path/to/directory.`
- Result: Compression took 8 seconds in the demonstration.

bzip2 Compression:

- File Extension: Can use any, but .tar.bz2 is standard.
- Command: `time tar -cjf /tmp/files.tar.bz2 /path/to/directory.`
- Result: Took 22 seconds, more than double the time of gzip.

xz Compression:

- File Extension: .tar.xz.
- Command: `time tar -cJf /tmp/files.tar.xz /path/to/directory.`
- Result: Took 1 minute and 10 seconds, significantly longer.

Compression Ratios:

- Comparative Sizes: `ls -lrt`
- Original Archive: 339 MB.
- gzip: ~100 MB.
- bzip2: ~90 MB.
- xz: 78 MB.
- The more effective the compression (like xz), the longer it takes.

Usage in Command Line:

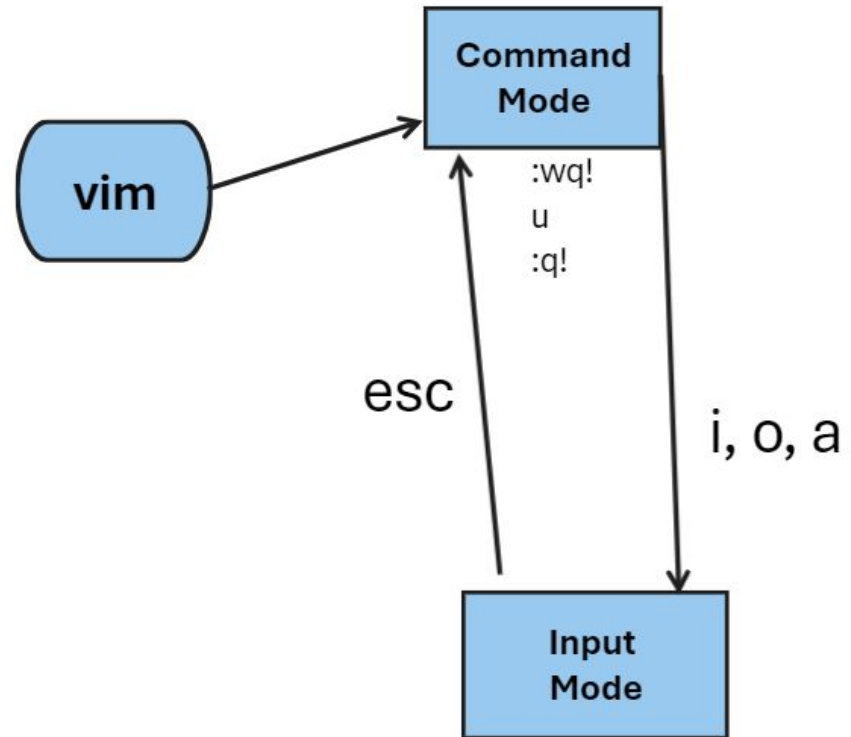
- Compression:
- `gzip /tmp/files.tar` will compress the file, prompting to overwrite if the compressed file already exists.
- bzip2 and xz can be used similarly.
- Decompression:
- `gunzip` for decompressing gzip files.

Working with text file

- Linux Command Line Text Editors: Linux operations are mostly done via the command line, necessitating knowledge of command line text editors. For example, when working in a non-GUI (Graphical User Interface) Linux environment, you will need to rely on these editors.
- Nano Editor: Nano is user-friendly and typically comes pre-installed on many Linux systems. It's suitable for those who do not wish to delve deeply into Linux. For instance, opening a file in Nano can be as simple as typing `nano filename`, and the interface provides intuitive command instructions at the bottom of the screen.
- Vim Editor: Vim, which stands for "Vi Improved," is a more powerful editor with advanced features for programming. Vim is essential for those looking to master Linux, as it is ubiquitous even on older UNIX systems. Vim was created in 1984, illustrating its long-standing presence in UNIX and Linux systems.
- Learning Vim: It is advised to learn Vim due to its availability on almost all systems, including the oldest UNIX systems. For instance, typing `vi` on any system is likely to open Vim or a version of it.
- Vi vs. Vim: The course does not focus on the minor differences between Vi and Vim, emphasizing instead the importance of learning to use them effectively because of their powerful and programmable nature.

- Using Nano: Nano's interface is designed to be intuitive, with shortcut keys displayed at the bottom. For example, to exit Nano, one would use the Ctrl-X command, following the instructions for saving or discarding changes.
- Permission Issues: If you encounter permission issues, such as a "permission denied" error when saving a file in Nano, it indicates a need to understand file permissions.
- Saving Files in Nano: When exiting Nano after editing, it prompts for saving the buffer and allows renaming the file and choosing the file format. For example, after editing, pressing Ctrl-X will prompt to save changes, and you can choose to save in the current or a different name, and even change the format if needed.
- Creating Files in Nano: The process of creating and saving a new file in Nano was demonstrated, showing the ease of the editor. For instance, by entering nano newfile, typing some text, and then using Ctrl-X, followed by confirming save options, a new file is created.

vim



1. **Starting Vim:** Vim opens in command mode by default, which only accepts commands.
 1. *Example:* Upon opening Vim, typing letters won't input text until you switch modes.
2. **Switching Modes:** Press 'i' to enter insert mode to type text, or 'o' to open a new line and enter insert mode.
 1. *Example:* To start editing a file right where the cursor is, press 'i'.
3. **Saving and Exiting:** Use ':wq!' to save changes and exit Vim without complaints.
 1. *Example:* After making changes to a file, type ':wq!' to save and close Vim.
4. **Undoing Changes:** The 'u' command undoes the last change, and can be used multiple times to undo more changes.
 1. *Example:* If you accidentally delete text, press 'u' to undo the action.
5. **Redoing Changes:** The 'Ctrl-r' command redoes an undone change.
 1. *Example:* If you undo too many changes, use 'Ctrl-r' to redo the last undo.
6. **Deleting Lines:** The 'dd' command deletes the current line, which is handy since many operations in Linux are line-oriented.
 1. *Example:* To remove an entire line where the cursor is positioned, use 'dd'.
7. **Substitution:** The ':%s/old/new/g' command finds all occurrences of 'old' and replaces them with 'new'.
 1. *Example:* To replace every instance of 'foo' with 'bar' in a document, type ':%s/foo/bar/g'.
8. **Learning with vimtutor:** Vimtutor is a built-in tutorial command that guides users through Vim's basics.
 1. *Example:* Run 'vimtutor' in the terminal for a hands-on Vim tutorial.
9. **Navigating Text:** Use 'gg' to move the cursor to the top of the document and 'G' to move to the end.
 1. *Example:* To quickly jump to the beginning of your file, press 'gg'.

1. **Visual Mode:** Press 'v' to enter visual mode, which allows text selection using arrow keys for operations like delete ('d') or copy ('y').
 1. *Example:* To select a block of text, enter visual mode with 'v' and use arrow keys to highlight the text.
2. **Deleting Multiple Lines:** '3dd' would delete three lines starting from the cursor's current position.
 1. *Example:* To delete the next three lines, position the cursor on the first line and type '3dd'.
3. **Searching Text:** Use '/text' to search for the occurrence of 'text' within the document.
 1. *Example:* To find the word 'error' in the document, type '/error'.
4. **Word Navigation:** 'w' moves the cursor to the next word, and 'b' moves it back to the beginning of the previous word.
 1. *Example:* To move forward to the start of the next word, use 'w'.
5. **Line Start and End:** The caret (^) moves to the start of the current line, and the dollar sign (\$) moves to the end.
 1. *Example:* To jump to the end of a line, press '\$'.
6. **Deleting Words:** 'dw' deletes the word under the cursor.
 1. *Example:* To delete the word that the cursor is on, use 'dw'.
7. **Opening Lines:** 'O' opens a new line above the cursor, and 'o' opens a new line below it.
 1. *Example:* To insert a new line above the current one, press 'O'.
8. **Displaying Line Numbers:** ':se number' turns on line numbering, which is useful for reference.
 1. *Example:* To view line numbers in the file, type ':se number'.
9. **Replacing Characters:** 'r' replaces the character under the cursor without entering insert mode.
 1. *Example:* To change a specific character, move the cursor over it and press 'r', then the new character.

Creating Text Tiles with vim

Ubuntu and VI Editor:

- Ubuntu doesn't come with VIM pre-installed because it's considered not very user-friendly by Ubuntu developers.
- Example: When you type `vim my_file` in Ubuntu, you might see a "command not found" error because VIM needs to be installed first.

Installing VIM:

- You can install VIM on Ubuntu using the command `sudo apt install vim`.
- Example: After installing VIM, when you run `vim my_file`, it should open the file in VIM editor without any error

Using VIM - Basic Commands:

- Insert mode in VI can be accidentally entered by typing commands such as 'o'.
- Example: If you're in normal mode and press 'o', you'll open a new line below the current one and switch to insert mode.

Switching Modes:

- Escape key is used to go back to command mode from insert mode.
- Example: After typing your text, pressing the Escape key will stop you from inserting text and allow you to enter commands.

Text Manipulation:

- VIM has multiple keys to enter insert mode and other modes like visual mode (v), which allows you to select text.
- Example: In command mode, pressing 'v' enters visual mode, letting you select text for operations like copy (y) and paste (p).

Navigating and Editing:

- Use 'w' to move the cursor to the next word and 'b' to move back to the beginning of the word.
- Example: Pressing 'dw' will delete the word your cursor is on.

Search and Replace:

- The substitute command in VIM (:%s) is used to replace words throughout the text.
- Example: To change every occurrence of "world" to "moon" in the text, you would use :%s/world/moon/g.

Undoing Changes

- The 'u' command is used to undo changes.
- Example: If you accidentally delete a line with 'dd', pressing 'u' will restore it.

Saving and Exiting:

- shift + ZZ is a shortcut to save and exit the file you are working on in VIM.
- Example: Instead of typing :wq!, you can just press 'ZZ' to save your changes and exit.

VIM Tutor:

- The vimtutor is recommended for learning VIM because it provides interactive lessons.
- Example: When you start vimtutor, one of the first things you'll learn is cursor movement with the keys 'h', 'j', 'k', 'l', which are alternatives to arrow keys on the keyboard.

Browsing Text Files with more and less

- More: This is the original file pager which allows you to read text files screen by screen. When using more, after displaying one screen of the text, it prompts 'more' at the bottom line. You need to hit the space bar to see the next screen.
- Less: This tool does everything more can do and more. It was developed to offer advanced features, such as the ability to navigate backward. With less, you can use the arrow keys to move up and see the previous screens. Example: Using less filename allows you to scroll forward and backward within the file.

Using head and tail to See File Start and End

Utility Introduction: The video introduces two utilities, tail and head, which display the beginning and end of text files, respectively.

- Example: `tail /etc/passwd` displays the last ten lines of the file `/etc/passwd`.

Basic Commands:

- tail shows the last part of files by default.
- head displays the first part of files by default.
- Example: `head /etc/passwd` will show the initial lines of `/etc/passwd`.

Modifying Output:

- You can modify the number of lines displayed by these commands using options like `-n`.
- Example: `head -3 /etc/passwd` will display the first three lines of `/etc/passwd`.

Combining Commands:

- Combining head and tail with a pipe (`|`) can filter specific lines from a file.
- Example: `head -3 /etc/passwd | tail -1` displays the third line of `/etc/passwd`.

Real-time Monitoring:

- Using `tail -f` allows monitoring of files in real time, such as log files.
- Example: `sudo tail -f /var/log/messages` opens the log file and displays new entries as they are logged.

Observing Logged Events:

- Real-time monitoring can help observe system events as they are being logged.
- Example: After a failed `su` command, the message "FAILED SU to root student on pts/1" is displayed in the log file in real time.

Exiting Commands:

- The generic method to stop a command that is running in the terminal is using `Control + C`.
- Example: Pressing `Control + C` will exit the `tail -f` command.

Displaying File Contents with cat and tac

cat Command Overview: cat is a command in Linux that allows you to display the contents of a text file directly to the terminal without pagination.

- Example: If you have a file named example.txt, you can view its contents by typing `cat example.txt` in the terminal.
- Options for cat:
- `-A` option shows all non-printable characters.
- `-b` numbers all lines.
- `-N` numbers lines but skips empty lines.
- `-S` suppresses repeated empty lines.
- Example: To display non-printable characters in a file named example.txt, you would use `cat -A example.txt`. This will show characters like tabs as `^I`.

Usefulness of cat Options: These options are particularly helpful when troubleshooting issues in configuration files where non-printable characters may cause problems.

- Example: If a configuration file isn't working, you can use `cat -A` to reveal hidden characters that may be causing the issue.

tac Command: tac is similar to cat but it displays the content in reverse order. It's considered less useful and is not commonly used.

- Example: To display the contents of example.txt in reverse order, you would type `tac example.txt` in the terminal.
- Demonstration of Commands: use of tac on the etc/hosts file to display its content in reverse order, and cat to show it in the correct order.
- Example: To compare the regular and reversed content of etc/hosts, you could use `cat etc/hosts` and `tac etc/hosts` to see the difference.
- Recommendation: It's recommended to remember the `-A` option with cat for troubleshooting non-printable characters.
- Example: Always keep in mind the `cat -A` option as a go-to troubleshooting tool when dealing with configuration issues.

Working with grep

Introduction to grep:

- What: grep is a powerful text search utility in Linux.
- Why: It is capable of finding text through patterns and regular expressions.
- Where: Can search within files or filter command output using a pipe.
- Example: "In this tutorial, we're exploring the find utility, a counterpart to grep that locates files within a directory hierarchy."

Basic Usage of grep:

- Simple Search: Using grep followed by a string and asterisk (e.g., `grep alice *`) searches for the term "alice" in all files in the current directory.
- Piped Search: Combining grep with a pipe (e.g., `ps aux | grep server`) filters the output of a command to only show lines containing the term "server".
- Example: "For a basic search, typing `find samuel *` would list all occurrences of 'samuel' in file names within the current directory."

grep Options for Enhanced Search:

- -i: Ignores case sensitivity during searches.
- -v: Excludes patterns from the search.
- -l: Lists files containing the pattern without showing the matching lines.
- Context Lines: -A, -B, and -C options show lines after, before, or around the match, respectively.
- -R: Recursively searches directories for the pattern.
- Example: "Using `find -name 'AMY' -type f` would ignore the case and find all files named 'amy' regardless of how the letters are capitalized."

Understanding grep Command

Command Line Examples

Introduction to grep

The grep command is used to search text in Unix and Unix-like systems.

It searches files for specified patterns and, by default, outputs the matching lines.

It's a powerful tool for text processing and data analysis.

Basic grep Search

Search for a pattern within a file.

Command: `grep "pattern" file.txt`

This command will output lines containing the pattern.

Case-Insensitive Search

Ignore case distinctions in patterns and data.

Command: `grep -i "pattern" file.txt`

Useful when the case of the text is unknown or varied.

Counting Matches

Count the number of matching lines.

Command: `grep -c "pattern" file.txt`

Can be used to quantify the presence of a pattern.

Displaying Line Numbers

Show line numbers along with the matching lines.

Command: `grep -n "pattern" file.txt`

Helps in locating the match within the file.

Matching Whole Words

Match patterns as whole words.

Command: `grep -w "pattern" file.txt`

Prevents partial matches within larger words.

Inverting the Match

Select lines that do not match the pattern.

Command: `grep -v "pattern" file.txt`

Useful for excluding specific patterns.

Recursive Search

Search recursively through directories.

Command: `grep -r "pattern" /path/to/directory`

Allows searching through complex directory trees.

Conclusion

The grep command is versatile and can be customized with various options.

It's an essential tool for anyone working with text files on Unix-like systems.

**Using cut and
sort**

CUT and SORT

Text manipulation utilities, focusing on 'cut' and 'sort'.

Using 'cut':

- 'cut' is a utility for filtering columns from text files.
- To use it effectively, you must specify the field delimiter with -d.
- Example 1: `cut -f 3 /etc/passwd` results in printing the entire content, as it lacks a field delimiter.
- Example 2: `cut -d : -f 3 /etc/passwd` correctly prints the third field by specifying the colon as the delimiter.

Using 'sort':

- 'sort' is often used in conjunction with 'cut' to organize the output.
- By default, 'sort' orders alphabetically.
- To sort numerically, use the -n option.
- Comparing Sort Commands:
- Alphabetical Sort: `sort` alone might not give desired results as it sorts starting with the first character.
- Numeric Sort: `sort -n` organizes the output in a numeric order, which is often more useful.

Using tr

- The tr command in Linux is a versatile utility used for translating, deleting, or squeezing repeating characters from the standard input and writing the result to standard output. It stands for "translate" and is particularly useful for modifying text.

Basic Syntax:

- Command: `tr [OPTIONS] SET1 [SET2]`
- Functionality: Translates or deletes characters from SET1 optionally into SET2.

Examples:

- Replacing Characters:
 - Command: `echo "hello world" | tr 'a-z' 'A-Z'`
 - Description: Converts lowercase letters to uppercase. The output will be HELLO WORLD.
- Deleting Characters:
 - Command: `echo "hello 123 world 456" | tr -d '0-9'`
 - Description: Deletes all numbers from the input string. The output will be hello world .
- Complement Set Deletion:
 - Command: `echo "hello 123 world 456" | tr -cd '0-9'`
 - Description: Deletes all characters that are not numbers (-c is used for the complement set). The output will be 123456.
- Character Class Usage:
 - Command: `echo "hello world" | tr '[:lower:]' '[:upper:]'`
 - Description: Similar to the first example, but uses character classes for lowercase ([:lower:]) and uppercase ([:upper:]) conversion.

TAB Completion and Up Arrow Keys

1. Tab Completion:

1. **Function:** When you press the tab key, it auto-completes commands, file names, or directory names based on the initial characters typed.
2. **Examples:**
 1. Typing "chm" and pressing tab auto-completes to "chmod".
 2. Typing "ls j" and pressing tab lists all files starting with 'J'.
 3. Typing "cd Des" and pressing tab shows directories starting with 'Des'.
3. **Benefit:** Simplifies command entry, reducing the need to type entire commands.
4. **Advice:** Be cautious with commands you're unfamiliar with; use the "man" command for guidance.

2. Using Up Arrow Key:

1. **Function:** Pressing the up arrow key cycles through previously executed commands.
2. **Behavior:** Sequentially brings up commands in the order they were run.
3. **Down Arrow Key:** If you press it after cycling through commands with the up arrow key, it returns to more recent commands.
4. **Benefit:** Saves time by avoiding retyping command

Re-Directs

1. **Introduction to Adding Text to Files:** The transcript begins with an introduction to adding text, digits, or letters to empty files in Linux. It mentions the use of the 'touch' command for creating empty files.
2. **Methods to Add Text to Files:**
 1. **vi Editor:** The first method discussed for adding text to files is using the 'vi' editor, a command-line text editor in Linux.
 2. **Redirect Command Output:** The second method involves using the redirect command to output text into a file.
 3. **Echo Command:** The third method is using the 'echo' command to output text into a file.
3. **Appending vs. Overwriting with Redirects:** The difference between single ('>') and double ('>>') redirect signs is explained. Single redirects overwrite existing content, while double redirects append new content.

Stdout to a file using Tee command

- Using the tee Command: The tee command in Linux is used to both display the output of a command on the screen and save it into a file simultaneously.
- Example: `echo "Hello World" | tee output.txt`, the phrase "Hello World" will be displayed on the screen and also saved in output.txt.
- Difference from Redirect Command: Unlike the redirect command (using `>`), tee allows you to see the output on the screen as well as save it. The redirect command only saves the output to a file without displaying it.
- Example: `echo "Sample Text" > file.txt`, the text is saved to file.txt but not displayed on the screen.
- Appending Data with tee: To append data to a file without overwriting, use tee with the `-a` option.
- Example: Append text to a file by using `echo "New Line" | tee -a existingfile.txt`.

- Using tee with Multiple Files: tee can be used to output to multiple files simultaneously.
- Example: can list directory contents and save them in multiple files using `ls -l | tee file1.txt file2.txt file3.txt`.
- Viewing tee Command Version and Help: To see the version and options of the tee command, use `tee --version` and `tee --help`.
- Example: can explore more about the tee command using `tee --help` to understand its various options.

Pipes

The pipe (|) is a powerful utility in Linux and Unix-like operating systems, used to pass the output of one command as input to another. It's a form of redirection that allows the output of one process to be directly used as input for another process, enabling a flow of data between programs.

Basic Concept:

Symbol: |

- **Functionality:** Connects the output of one command (left side of the pipe) to the input of another command (right side of the pipe).

Examples:

- **Combining ls and grep:**
- **Command:** `ls | grep 'pattern'`
- **Description:** Lists all files in the current directory (`ls`), then filters the list to show only files that match a specific pattern (`grep 'pattern'`).

Linking cat and wc:

- **Command:** `cat filename.txt | wc -l`
- **Description:** Displays the contents of `filename.txt` (`cat filename.txt`), then counts the number of lines in this content (`wc -l`).

Chaining ps and sort:

- **Command:** `ps -ef | sort`
- **Description:** Displays a list of current processes (`ps -ef`), then sorts this list alphabetically (`sort`).

Sequential grep commands:

- **Command:** `cat filename.txt | grep 'pattern1' | grep 'pattern2'`
- **Description:** Displays the contents of `filename.txt`, filters lines containing `'pattern1'`, then further filters the remaining lines for a second pattern, `'pattern2'`.

awk

- The awk command in Linux is a powerful programming language and utility used for pattern scanning and processing. It's particularly effective for processing and analyzing text files and strings.

Basic Syntax:

- Command: `awk [OPTIONS] 'pattern {action}' file`
- Functionality: Executes the action on lines matching the pattern from the file or standard input.

Key Features:

- Pattern Matching: Executes actions on lines or records that match a specific pattern.
- Built-in Variables: Such as NR (record number), NF (number of fields in a record), FS (field separator), etc.
- Field-wise Processing: Automatically splits a line into fields, making it easy to manipulate individual columns.
- Arithmetic and String Operations: Supports various operations for processing and transforming data.
- Conditional Statements and Loops: Allows complex programming constructs.

Practical Examples of Awk Usage:

Example 1: Basic Printing

- Command: `awk '{print $0}' /etc/passwd`
- Description: Prints everything in the file.

Example 2: Printing Specific Fields

- Command: `awk -F: '{print $1}' /etc/passwd`
- Description: Prints the first field of each line, using colon as a field separator.

Example 3: Filtering Based on Line Length

- Command: `awk 'length($0) > 40' /etc/passwd`
- Description: Displays lines longer than 40 characters.

Example 4: Searching for a Specific User ID

- Command: `awk -F: '/Alex/{print $3}' /etc/passwd`
- Description: Finds the user ID for user 'Alex'.

Same output if u use below:

- `grep jai /etc/passwd`
- `grep jai /etc/passwd | cut -d : -f 3`

sed

Introduction to sed:

- Topic: Sed, a stream editor for non-visual file editing, is introduced following a discussion on awk.
- Purpose: Sed is presented as a programmable editor for file manipulation.
- Basic sed Command Example:
 - Command: `sed -n 5p /etc/passwd`
 - Function: Prints the 5th line of the file.
 - Validation: This is cross-checked using `head -5 /etc/passwd | tail -1`.
- Substitution with sed:
 - Command: `sed -i 's/Anna/Bob/g' users`
 - Function: Globally substitutes 'Anna' with 'Bob' in the 'users' file.
 - Caution: The change is irreversible, emphasizing the need for careful operation.
- Deleting Entries Using sed:
 - Command: `sed -i -e '2d' users`
 - Function: Deletes the 2nd line from the 'users' file.
 - Context: Useful for removing erroneous lines identified by line number.

Connecting to a Server

Understanding Root User

Definition and Power of Root User in Linux:

- The root user is the ultimate administrator in Linux, with unrestricted access and capabilities.
- Unlike Windows administrators who have extensive privileges, the root user in Linux operates beyond the realm of standard privileges, directly in the Linux kernel space.

Linux Kernel and Root User:

- The Linux kernel, being the core of the operating system, has no limitations in its operations.
- The root user is essential in this environment as it possesses kernel-level access, making it an unlimited user account.

Caution and Alternatives to Using Root User:

- Due to its unlimited power, using the root account is generally discouraged.
- Modern Linux distributions often do not set a default password for the root user or make its activation optional, reducing the risk of its misuse.
- Historically, setting a root password was common during Linux installation, but this practice has changed.

Using sudo and su as Alternatives:

- The preferred method for executing commands with superuser privileges is using sudo, which requires authorization and is limited to authorized users.
- sudo is used by prefixing it to a command to grant it superuser powers temporarily.
- The su command can also be used to switch to a root shell, but it requires the root password, which may not be set in many modern distributions, limiting its usefulness.

Examples and More Context:

- Example of sudo: To install a software package, instead of logging in as root, a user can use `sudo apt-get install [package_name]`.
- Example of su: To switch to a root shell for a series of administrative tasks, a user might use `su` - but would need the root password.
- Additional Example of Root User Limitations: In some secure environments, even the root user might be restricted from accessing certain encrypted user data, illustrating the evolving security practices in Linux distributions.

SU

Understanding su Command:

- The su command stands for "switch user." It is used to open a shell as a specific user.
- Example: Typing su Lisa in the command line will prompt for Lisa's password and then open a shell logged in as Lisa.

Root User Privileges:

- The root user can use su to switch to any other user without needing to enter a password.
- Example: As root, typing su John will directly open a shell as John without a password prompt.

Opening a Root Shell:

- Typing su with no arguments prompts for the root password and opens a root shell.
- Example: An ordinary user can type su, enter the root password, and gain access to the root shell.

Root Shell Identification:

- In a root shell, the prompt typically changes to include root@hostname with a hash symbol.
- Example: If the prompt changes to root@centos #, it indicates you are in a root shell.

Exiting the Shell:

- Typing `exit` will exit the current shell and return to the previous user's shell.
- Example: If you are in a shell as Linda and type `exit`, you will return to the shell from which you switched to Linda.

Using `su` on Different Systems:

- On Ubuntu, using `su` or `su -` is not typically effective because the root user does not have a password by default.
- Example: On Ubuntu, typing `su -` will prompt for a root password, but since there is none set, it won't work.

Alternative in Ubuntu: `sudo`:

- Instead of `su`, Ubuntu users often use `sudo` for administrative tasks, which will be discussed in the next video.
- Example: Typing `sudo apt-get update` on Ubuntu will execute the update command as a superuser.

Additional Example - Switching Between Users:

- As a regular user, you can switch to another user (e.g., `su David`), enter David's password, and work with David's permissions.
- Example: If you're logged in as Alice and need to perform tasks as David, use `su David`, enter his password, and you'll be in David's shell.

Security Consideration:

- Knowing the root password or having `sudo` privileges gives significant control over the system, hence should be managed securely.
- Example: Only trusted users should be given the root password or `sudo` access to prevent unauthorized system changes.

sudo

1. Use of Sudo for Administrative Tasks:

1. Sudo is preferred for running administrative tasks due to its sophistication and flexibility.
2. It allows specific users to perform certain tasks with elevated permissions, unlike the binary root/non-root system in traditional admin setups.

2. Configuration and User Privileges:

1. Users can be configured to have limited administrative tasks.
2. The default "admin" user, typically created during the Linux distribution installation, usually has sudo privileges.

3. Distribution-Specific Details:

1. On Red Hat systems, users in the 'wheel' group are considered admin users.
2. In contrast, there is no default "admin" user if a root user is set up with a password during installation. Admin privileges must be explicitly granted.
3. On Ubuntu, the default "admin" user is part of the 'sudo' group.

Using Sudo Commands:

- To execute a command with elevated privileges, use `sudo` followed by the command.
- Example: To list the contents of the root directory, use `sudo ls -l /root`.
- `sudo -i` opens a root shell, providing broader access but is generally not recommended.

Understanding Sudo in Different Contexts:

- The `id` command shows the groups a user belongs to, indicating if they have sudo privileges.
- Using `sudo` in front of a command grants temporary elevated privileges for that command only.
- `sudo -i` and `sudo su -` both open a root shell, but the latter uses the `su` mechanism.

Additional Examples and Considerations:

- In a scenario where continuous administrative work is needed, using `sudo -i` might be more convenient.
- Always exit a sudo shell with the `exit` command to return to the normal user environment.
- The process of granting sudo privileges varies slightly between distributions but generally revolves around user group memberships.

Creating a Simple sudo Configuration

Introduction to Sudo Configuration:

- visudo is used to edit the /etc/sudoers file, which is the main configuration file for sudo.
- Drop-in files can be added to /etc/sudoers.d as an alternative to editing the main configuration file directly.

Understanding the Sudoers File:

- The %wheel ALL=(ALL) ALL entry in the sudoers file allows all members of the 'wheel' group to execute any command.
- A risky configuration is %wheel ALL=(ALL) NOPASSWD: ALL, which allows password-less sudo access for the 'wheel' group. This is discouraged due to security concerns.

Managing Sudo Credentials Cache:

- By default, sudo requires re-authentication after a short period. This can be changed.
- Example: Defaults timestamp_type=global,timestamp_timeout=240; increases the token validity to 4 hours.

Editing the Sudoers File with visudo:

- Always use sudo visudo to edit the sudoers file to avoid permission issues.
- visudo checks for syntax errors and prevents saving incorrect configurations.

visudo

- # Defaults env_keep += "HOME"
- Defaults secure_path = /sbin:/bin:/usr/sbin:/usr/bin
- Defaults timestamp_type=global,timestamp_timeout=240
- ## Allows members of the users group to mount and unmount the
- ## cdrom as root
- # %users ALL=/sbin/mount /mnt/cdrom, /sbin/umount /mnt/cdrom
- Rakesh ALL=/usr/bin/passwd, ! /usr/bin/passwd root, /usr/sbin/useradd, /usr/sbin/usermod

- The line `%wheel ALL=(ALL) ALL` is a configuration entry in the `sudoers` file, which is used in Unix-like operating systems to control the `sudo` command's behavior. This particular line is a rule that specifies permissions for users who are members of the wheel group. Let's break down what each part means:
- `%wheel`: This specifies the group name prefixed with a `%` sign. In this case, it's the wheel group. The `%` indicates that this rule applies to a group, not an individual user.
- `ALL` (first occurrence): This is the host alias. `ALL` in this position means that the rule applies to all hosts. In a more complex setup, you could have different rules for different hosts or host groups.
- `ALL` (second occurrence, within parentheses): This specifies which users the members of the wheel group can act as when they use `sudo`. The `ALL` here means that users in the wheel group can execute commands as any user (including the root user). If this were a specific username instead of `ALL`, then members of the wheel group could only execute commands as that particular user.
- `ALL` (third occurrence): This is the command alias, which specifies which commands this rule applies to. `ALL` in this position means that the members of the wheel group are allowed to execute all commands.
- Putting it all together, `%wheel ALL=(ALL) ALL` means that any user who is a member of the wheel group can use `sudo` to execute any command as any user on any host. This is a very powerful and unrestricted permission, typically granted to system administrators or trusted users, as it essentially grants full control over the system.

- john ALL=/usr/bin/apt-get, /usr/bin/systemctl
 - Explanation: This rule allows the user john to execute two specific commands: /usr/bin/apt-get and /usr/bin/systemctl.
 - Details:
 - john: The username. This rule applies only to the user named John.
 - ALL: John can use these commands on all hosts.
 - /usr/bin/apt-get, /usr/bin/systemctl: The only commands John is allowed to execute with sudo.
-
- %developers ALL=(ALL) /usr/bin/git, /usr/bin/npm
 - Explanation: Members of the developers group can execute git and npm commands as any user.
 - Details:
 - %developers: The group name (developers), prefixed with %.
 - ALL: Applies to all hosts.
 - (ALL): Group members can execute the commands as any user.
 - /usr/bin/git, /usr/bin/npm: Only these commands are allowed.

- `lisa ALL=(ALL) NOPASSWD: /usr/bin/rsync`
 - Explanation: The user `lisa` can execute the `rsync` command without entering a password.
 - Details:
 - `lisa`: Specific user.
 - `ALL`: On all hosts.
 - `(ALL)`: As any user.
 - `NOPASSWD: /usr/bin/rsync`: The `NOPASSWD` tag allows running `rsync` without a password prompt.
-
- `%junior_admins ALL=(ALL) ALL, !/usr/bin/passwd`
 - Explanation: Members of the `junior_admins` group can execute all commands except changing passwords.
 - Details:
 - `%junior_admins`: Group name.
 - `ALL`: On all hosts.
 - `(ALL) ALL`: Execute any command as any user.
 - `!/usr/bin/passwd`: The `!` negates the command, preventing this group from changing passwords.

Introduction to SSH (Secure Shell):

- SSH provides secure remote shell access.
- It's essential for accessing Linux systems in the cloud or virtual machines.
- Example: Accessing a Linux server hosted on AWS or Azure.

Installing and Enabling SSH:

- On Ubuntu: Use `sudo apt install openssh-server` to install.
- On Red Hat: SSH is usually pre-installed. If not, use `sudo dnf install openssh-server` followed by `sudo systemctl enable sshd`.
- Example: Setting up SSH on a local Ubuntu machine for remote access.

SSH Clients for Different Operating Systems:

- Linux and MacOS: Native command line SSH clients are available.
- Windows: MobaXterm can be used for easy SSH access.
- Example: Using MobaXterm on Windows to connect to a Linux server.

Using SCP for File Transfers:

- SCP (Secure Copy Protocol) is included in the SSH package.
- It allows file transfers between local and remote computers.
- Command example: `scp /etc/hosts 192.168.29.111:/tmp/`.
- Example: Transferring configuration files from a local machine to a remote server.

Demonstration of Starting with SSH:

- On Ubuntu, installing SSH and finding the machine's IP address using `ip a`.
- Connecting from a CentOS machine to Ubuntu using `ssh jai@192.168.29.141`.
- Example: Accessing a remote Ubuntu server from a CentOS local machine.

Handling SSH Key Fingerprints:

- SSH clients store the identity of previously connected hosts for security.
- Initial connections prompt for key fingerprint verification.
- Example: Connecting to a new server and accepting its SSH key fingerprint.

Creating and Transferring Files Using SSH and SCP:

- Creating a file on Ubuntu and transferring it to CentOS using SCP.
- Command example: `scp ubunut.txt 192.168.29.141:/tmp/`.
- Example: Transferring a log file from an Ubuntu server to a CentOS server.

Using Different User Accounts in SSH:

- SSH allows connection as different users using `username@server`.
- Example: Copying a file to the root user's home directory on a CentOS machine.
- Example: Accessing a server using a non-root user for added security.
- Sudo Operations Without Password Prompt:

Working with the Bash Shell

- Understanding the Shell and other Core Linux Components
- Using I/O Redirection and Piping
- Working with history
- Using Command Line Completion – Covered in Tab Completion
- Using Variables
- Using Other Bash Features
- Working with Bash Startup Files

Understanding the Shell and other Core Linux Components

1. Introduction to Linux Architecture

1. The core of Linux is the kernel, responsible for interacting with the system's hardware.
2. Example: The kernel is like the engine of a car, handling the core operations and ensuring everything runs smoothly.

2. Role of glibc

1. glibc (GNU C Library) is a fundamental component of Linux, providing essential libraries.
2. These libraries contain specific functions crucial for the operating system.
3. Example: Think of glibc as a toolkit that provides the necessary tools (functions) for building and running applications on Linux.

3. Understanding the Shell

1. The shell, particularly bash (Bourne Again SHell), is the default user interface on Linux.
2. It's where users type commands to interact with the system.
3. Example: The shell can be likened to a command center, where you input commands (like typing a destination into a GPS) to navigate and control the Linux system.

4. User Commands and the Shell

1. Every command a user types is processed through the shell.

5. Services and System D

1. Services in Linux don't necessarily go through the shell.
2. They use a different component called System D, which manages system processes.
3. System D connects directly to glibc and the kernel, offering an alternative route for services.
4. Example: If the shell is like manual driving, then services using System D are like an autonomous driving system, operating independently but still connected to the main engine (kernel).

6. The Shell's Role in Command Line Operations

1. The shell is essential for making command line operations easier and more efficient.

Using I/O Redirection

Redirection manipulates the input and output of commands.

- Three types: standard input (stdin), standard output (stdout), and standard error (stderr).

Standard Input (stdin)

- Represented by file descriptor 0.
- Normally, stdin is your keyboard.
- Use the < (less than) symbol for redirection from a file (e.g., `sort < /etc/services`).

Standard Output (stdout)

- Represented by file descriptor 1.
- Normally, stdout is your screen or terminal.
- Use > for redirecting output to a file (e.g., `ls > myfile`).
- > overwrites the file, >> appends to the file.

Standard Error (stderr)

- Represented by file descriptor 2.
- Use 2> for redirecting error messages (e.g., `command 2> errorfile`).
- 2> /dev/null discards error messages.

Combining stdout and stderr

- Use &> to redirect both stdout and stderr (e.g., `command &> outfile`).

- Additional Examples for I/O Redirection
- Redirecting Input: `grep 'searchText' < inputFile.txt`
- Appending Output: `echo 'new line' >> existingFile.txt`
- Discarding Errors: `find / -name 'myfile' 2> /dev/null`

- **What are File Descriptors?**

- **Basic Concept:** Think of file descriptors as "labels" or "references" to open files or data streams. When a program opens a file or starts reading/writing data, the operating system assigns a unique number, called a file descriptor, to that operation.
- **Why Use Them?:** File descriptors allow the system and programs to keep track of where to read data from or write data to. They are used for files, devices, pip

- Standard File Descriptors
- There are three standard file descriptors in Unix/Linux systems:

Standard Input (stdin) - File Descriptor 0

- Purpose: Used for reading input.
- Default: Keyboard input.
- Example: When you run `cat` without specifying a file, it reads from stdin (your keyboard).
- `cat` (then type something and press Enter)

Standard Output (stdout) - File Descriptor 1

- Purpose: Used for writing output.
- Default: Displaying output on the screen (terminal).
- Example: When you run `ls`, the output goes to stdout, which is by default your screen.
- `ls > output.txt` (Redirects the output of `ls` to a file named `output.txt`)

Standard Error (stderr) - File Descriptor 2

- Purpose: Used for error messages.
- Default: Displaying errors on the screen (terminal).
- Example: If you try to `cat` a non-existent file, the error message is sent to stderr.
- `cat non_existent_file 2> error.txt` (Redirects the error message to a file named `error.txt`)

- Using File Descriptors in Redirection

Redirecting stdin: <

- `grep 'text' < file.txt` (Searches for 'text' in file.txt instead of stdin)

Redirecting stdout: >

- `echo 'hello' > file.txt` (Writes 'hello' to file.txt instead of stdout)

Redirecting stderr: 2>

- `ls invalid_dir 2> error.txt` (Redirects error message to error.txt)

Combining File Descriptors

- Redirecting both stdout and stderr: &>
- `command &> output.txt` (Redirects both output and error messages to output.txt)

- `echo "Hello, world!" > example.txt` # Create a file
- `ls -i example.txt` # View the inode number
- `tail -f example.txt`
- `lsof | grep example.txt` -- In another terminal, find the file descriptor
- `tail 12345 user 3r REG 252,1 13 12345678 /path/to/example.txt`
- In this line:
 - `tail` is the command.
 - `12345` is the process ID of `tail`.
 - `3r` under FD is the file descriptor number (3) and the mode (r for read).
 - `12345678` at the end is the inode number, matching the one from `ls -i`

- Conclusion
- Inodes provide a unique identifier for the file within the filesystem, storing metadata about the file.
- File Descriptors are used by processes (like tail in this example) to reference open files. They are specific to each process and are assigned when the file is opened.

Definition of History in Bash:

History is a Bash feature that records all commands typed by the user.

- These commands are stored in a file named `.bash_history` in the user's home directory.
- The history file is persistent, retaining commands even after a system reboot.

Basic History Commands:

- `history`: Displays the command history.
- `history -c`: Clears the current history.
- `history -w`: Writes the current history to the history file.
- `history -d [line_number]`: Deletes a specific line from the history, useful for removing commands with sensitive information.

Using Ctrl+R for Reverse-I-Search:

- Ctrl+R allows backward searching through the command history.
- Typing a command partially will display matches from the history.
- Pressing Ctrl+R again cycles through previous matches.

Using Exclamation Mark for Command Repetition:

- `![number]`: Repeats a specific command from history based on its line number.
- `!H`: Repeats the last command starting with 'H'.
- `!G`: Repeats the last command starting with 'G'.

Viewing and Managing History File:

- `ls -l .bash_history`: Displays details of the `.bash_history` file.
- The default history size is set to 1000 lines in Linux.
- `cat .bash_history`: Views the contents of the history file.

Synchronizing History:

- Normally, history is updated upon exiting the shell.
- `history -w` can be used to manually update the history file.

Deleting Specific Commands from History:

- Example: `history -d 205` deletes command number 205 from history.
- This is useful for removing incorrect or sensitive commands.

Additional Examples of History Usage:

- `!204`: Repeats command number 204 from history.
- `!!s`: Repeats the last command that started with 's'.
- `history | grep 'ssh'`: Searches for all instances of 'ssh' commands in the history.

Best Practices:

- Be cautious when using history modification commands, especially in a shared environment.
- Regularly review and clean up your history file to maintain efficiency and security.

Using Variables

- Definition of Variables: Variables are labels assigned dynamic values, useful in scripting for storing site-specific information like usernames.
- Use in Scripting: Variables allow for flexibility and reusability in scripts, enabling the separation of site-specific information from static content (decoupling).
- System Variables in Linux: Linux uses system variables for default settings. Example: PATH variable, which lists directories searched for binaries.
- Setting and Using Variables: Variables are set with `varname=value` and referenced with `$varname`. Use `echo $varname` to check its value. They are case-sensitive and by default, known only to the current shell.
- Exporting Variables: To make a variable available in sub-shells, use `export`. Example: `export COLOR=green` makes COLOR accessible in sub-shells.
- Environment Variables: These are set by the operating system, startup shells, or commands. Use `env` command to view them.
- Practical Example with Shell Script: Demonstrated creating a shell script (`hello.sh`), making it executable, and addressing issues with the PATH variable for script execution.

Example

- `vim hello.sh`
 `echo Hello World`
- `chmod +x hello.sh`
- `hello.sh --` throws error

- `sudo mv bin/hello.sh /usr/local/bin`
`hello.sh`

Env

- Viewing and Setting Environment Variables:
- To view all environment variables, use the **printenv** or **env** command.
- To view a specific variable, use `echo $VARIABLE_NAME` (e.g., `echo $SHELL` or `echo $PATH`).
- To set a temporary environment variable, use `export VARIABLE_NAME=value`. For example, `export TEST=1`.
- To set a permanent environment variable, modify the `.bashrc` file with the desired variable and value, then export it (e.g., `export TEST='123'`).

Examples of Common Environment Variables:

- SHELL: Indicates the default shell path.
- PATH: Lists directories the shell searches for executable files.
- HOME: Specifies the user's home directory.
- LANG: Defines the system language and locale.
- HISTCONTROL: Controls how commands are saved in the history.

Setting Environment Variables Permanently

- For permanent changes, edit `.bashrc` in the user's home directory.
 - For system-wide changes, modify `/etc/profile` or `/etc/bashrc`.
 - Caution is advised when making global changes, especially in a production environment.
-
- `export TEST=one`
 - `echo $TEST`
-
- `cp .bashrc bashrc-backup-original`
 - `vi .bashrc`
 - `TEST='TWO THREE'`
 - `export TEST`
-
- `echo $TEST` gives ONE -- You need to logout from current session
 - `echo $TEST` gives TWO THREE

Using Other Bash Features

Alias allows defining custom commands in Bash.

- Typically, aliases are made permanent through Bash startup files like `/etc/profile`.
- Example: The command `alias jai='ls -l'` creates an alias named 'jai' for the command `ls -l`.
- Aliases are fast as they become part of the Bash shell.
- Making Aliases Permanent

To ensure aliases persist across sessions, they should be added to Bash startup files.

- Modified Example: Adding `alias ll='ls -la'` to `.bashrc` makes the 'll' alias permanent.

Removing Aliases with `Unalias`

- The `unalias` command is used to remove an alias.
- Example: `unalias jai` removes the previously created 'jai' alias.

Useful Keyboard Shortcuts in Bash

- Control-l: Clears the screen (alternative to typing `clear`).
- Control-u: Clears the current command line.
- Control-a: Moves the cursor to the beginning of the line.
- Control-e: Moves the cursor to the end of the line.
- Control-c: Interrupts the current process.
- Control-d: Exits the current process or terminal session.
- Using Control-r to search through command history.
- Control-k to delete text from the cursor to the end of the line.

Working with Bash Startup Files

Introduction to Bash Startup Files

- Bash startup files are essential for setting up the environment and variables when a Bash shell starts.
- They provide standard solutions for initializing Bash settings during boot-up.

Key Startup Files in Bash

- `/etc/environment`:
 - Contains a list of environment variables, It's the first file processed when starting a Bash shell.
 - Not mandatory and may be absent in some distributions like Red Hat.

`/etc/profile`:

- A crucial startup script executed during user login.
- Works in conjunction with `/etc/profile.d` directory.
- Scripts in `/etc/profile.d` are included in `/etc/profile`.

User-specific profiles:

- Users can have their own `.bash_profile` for settings specific to them.
- `.bash_logout` can be used for cleanup tasks when a user logs out.

Difference Between Login and Subshells

- `/etc/bashrc` (or `.bashrc` in user home):
 - Processed every time a subshell is started.
 - Differentiates settings for login shells and subshells.
 - In many distributions, `/etc/bashrc` and `.bashrc` might have similar configurations.

Modifying Bash Startup Files

- Caution advised when modifying `/etc/profile` as it's a managed file.
- User-specific changes can be made in `/etc/profile.d` or in user's `.bashrc`.
- Example: Modifying history file size for all users can be done in `/etc/profile.d/history.sh`.
- Changes in `/etc/profile.d` require re-login or sourcing the file to take effect.

```
vim /etc/profile.d/history.sh
```

```
    export HISTSIZE=2500
```

```
    export HISTFILESIZE=5000
```

```
echo $HIST
```

```
echo $HISTSIZE
```

```
source /etc/profile.d/history -- no need to relogin if we use source
```

```
vim .bash_profile □ reads.bashrc
```

```
vim .bashrc
```

```
    export COLOR=blue -- it will work for subshells
```

```
bash
```

```
echo $COLOR
```

User and Group Management and Permissions

User and Group Management

- Understanding Users
- Understanding the Role of Ownership
- Creating Users with `useradd`
- Creating Groups with `groupadd`
- Managing User and Group Properties
- Configuring Defaults for New Users
- Managing Password Properties
- Managing Current Sessions

Understanding Users

Definition of a User in Linux:

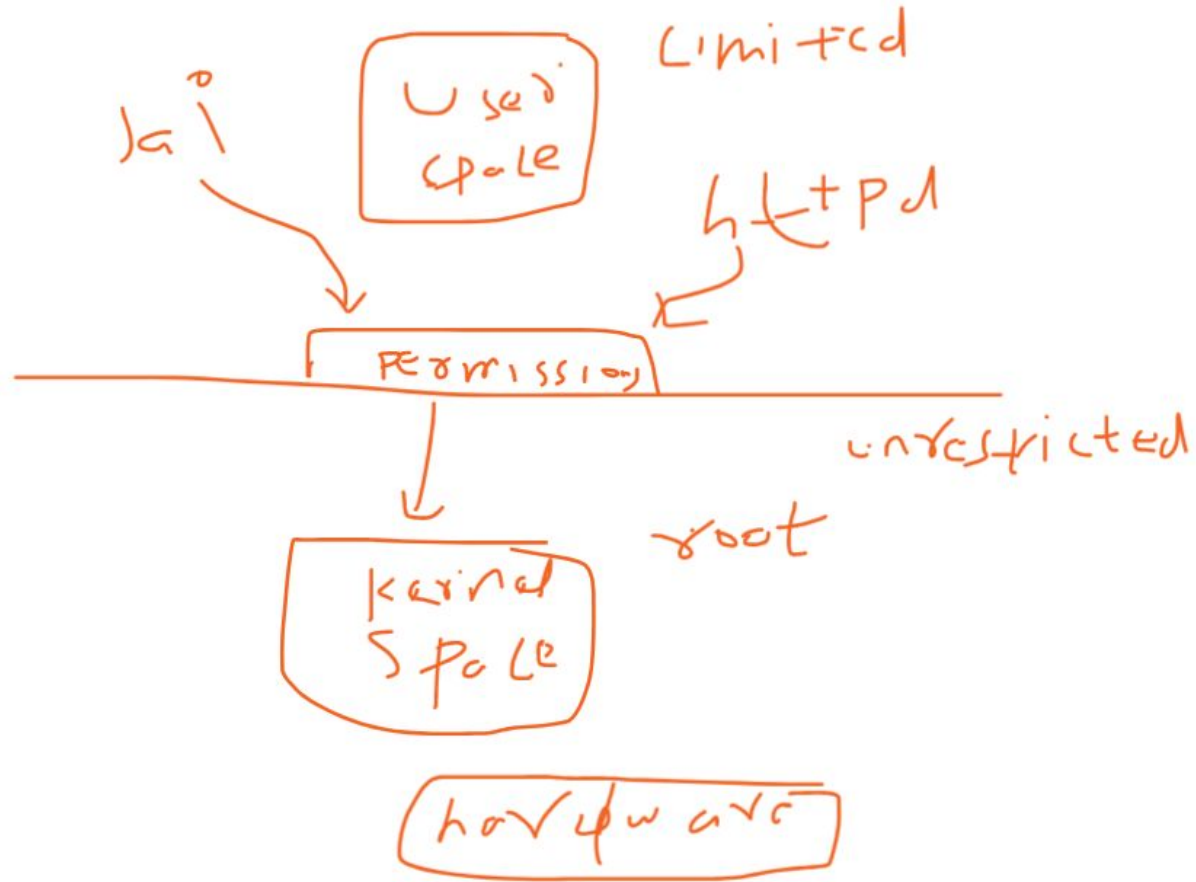
- A user in Linux is an entity that grants permissions to specific parts of the Linux operating system.
- There are two main types of users: system users and people users.

System Users:

- These are user accounts used by services within the Linux operating system.
- They are created to run services with restricted permissions, avoiding the excessive permissions of a root user.
- Example: A web server running under a specific user account, like httpd, to limit its access and permissions.

People Users:

- These users are primarily IT staff who manage and support the Linux system.
- In a typical organization, end-users do not directly log into Linux systems but interact with applications running on Linux.
- Example: IT support staff having user accounts for system maintenance, while end-users access services via applications.



Linux Operating System Architecture:

- The architecture is divided into kernel space and user space.
- Kernel space is where the Linux kernel runs with unrestricted access to hardware.
- User space is about restriction and limited access.
- The root user exists in kernel space with unrestricted access.

Role of Permissions:

- Permissions are used to manage the interaction between user space and kernel space.
- They ensure that users (both people and services) have filtered access to system resources.
- Example: A user like Jai might have specific permissions to perform certain tasks on the hardware, controlled through the Linux permission system.

Practical Example of User Processes:

- The command **ps aux** can be used to view running processes and their associated user accounts.
- This demonstrates how different processes run under different user accounts, each with specific permissions.
- Example: Processes running as 'root' have more access compared to those running under a service-specific user like 'polkitd'.

Understanding the Role of Ownership

Concept of Ownership in Linux:

- Ownership in Linux refers to the association of files with users.
- When a user creates a file, they become its owner.
- File ownership is crucial for managing Linux permissions.

User and Group Ownership:

- Every Linux user belongs to at least one group, known as the primary group.
- When a file is created, the user's primary group becomes the group owner of the file.
- The primary group is a default group for a user and plays a key role in file ownership.

Private Groups in Linux:

- Many Linux distributions use a concept called private groups.
- A private group is created with the same name as the user and the user is its only member.
- This approach enhances security by isolating user and group permissions.

Understanding Group ID (GID):

- GID is an identifier for a user's primary group.
- It is crucial in determining group ownership of newly created files.
- For example, if a user with GID 1000 creates a file, the file's group owner will be the user's primary group associated with GID 1000.

Practical Demonstration of Ownership:

- The `id` command can be used to display a user's ID and GID.
- Creating a file (e.g., using `touch`) shows the user and group ownership.
- For instance, if user 'student' creates a file, the file ownership will be displayed as 'student student', indicating the user and group ownership respectively.

File Permissions and Ownership:

- Permissions are assigned to User (U), Group (G), and Others (O).
- The `ls -l` command displays these permissions.
- For example, permissions like `rw- rw- r--` indicate read and write permissions for the user and group, and only read permission for others.

Creating Users with useradd

Creating User Accounts Manually:

- In some cases, you may need to manually create user accounts on a Linux system.
- The primary command for this is useradd.

Difference Between useradd and adduser:

- On Ubuntu, there's an alternative command adduser, which essentially performs the same function as useradd but in a slightly different manner.
- It's not necessary to remember multiple commands for the same function.

Importance of Home Directory:

- Users typically require a home directory.
- On Ubuntu and some other Linux distributions, useradd does not create a home directory by default.
- Use useradd -m on Ubuntu to ensure the creation of a home directory.

Default Settings from /etc/login.defs:

- When users are created, default settings from /etc/login.defs are applied.
- To view these settings, use useradd -D.

Creating a User Example:

- Example: **sudo useradd bill** creates a user named Bill.
- To check if the user is created, use **grep bill /etc/passwd**.
- This command shows the user's details including home directory, default shell, user ID, and group ID.

Verifying Home Directory Creation:

- After creating a user, verify the creation of the home directory using **ls /home**.
- On Ubuntu, if the home directory is not created, it indicates the need to use **useradd -m**.

Deleting and Recreating a User with Home Directory:

- If needed, a user can be deleted using **userdel -rf [username]**.
- To recreate the user with a home directory, use **useradd -m [username]**.

Additional Examples:

- Creating a user with a specific shell: **useradd -s /bin/bash [username]**.
- Assigning a user to a specific group: **useradd -G [groupname] [username]**.
- Setting an expiry date for the user account: **useradd -e [YYYY-MM-DD] [username]**.

Creating groups with groupadd

Creating Groups in Linux:

- To create a group, use the command **groupadd** followed by the group name.
- Example: **sudo groupadd marketing** to create a group named 'marketing'.

Adding Users to Groups:

- Users can be added to a group as a secondary group using `usermod -aG groupname username`.
- Example: `sudo usermod -aG marketing alice` adds user 'alice' to the 'marketing' group.

Understanding Primary and Secondary Groups:

- When a user is created (e.g., 'bill'), a primary group with the same name is automatically created.
- Use `id username` to view user properties, including group memberships.
- Secondary groups are used for granting additional permissions, not for default ownership.

Verifying Group Creation:

- Groups are listed in `/etc/group`.
- Use `grep groupname /etc/group` to verify if a group exists.
- Example: `grep marketing /etc/group` checks if the 'marketing' group exists.

Assigning Users to Secondary Groups:

- The command **sudo usermod -aG groupname username** adds a user to a secondary group.
- Example: **sudo usermod -aG developers bob** adds 'bob' to the 'developers' group.

Ownership and Secondary Groups:

- Files created by a user belong to the user's primary group by default.
- Secondary groups do not change file ownership.
- Example: If 'alice' creates a file, it is owned by 'alice' and her primary group, not the 'marketing' group.

Practical Use of Secondary Groups:

- Secondary groups are mainly for granting access to shared resources.
- They allow users from different primary groups to collaborate.
- Example: Users from 'sales' and 'tech' groups can both be added to a 'projectX' secondary group for joint access to project resources.

Additional Example - Managing Group Membership:

- To remove a user from a secondary group, use **gpasswd -d username groupname**.
- Example: **sudo gpasswd -d alice marketing** removes 'alice' from the 'marketing' group.

Listing Group Memberships:

- Use **groups username** to list all groups a user belongs to.
- Example: **groups bob** shows all groups 'bob' is a member of, including primary and secondary groups.

Managing User and Group Properties

User and Group Creation in Linux:

- Users and groups are fundamental entities in Linux, created for managing access and permissions.

User Properties in Linux:

- Stored in `/etc/passwd`.

Properties include:

- Username: Identifies the user.
- UID (User ID): A unique numerical identifier. Linux primarily uses UID instead of the username.
- GID (Group ID): Represents the primary group ID of the user.
- GECOS Field: Originally for General Electric Common Object Subscription, now used for additional user descriptions.
- Home Directory: The default directory for a user after login.
- Shell: The program started after login, often `/bin/bash`.
- Example: For a user 'alice', the entry might look like `alice:x:1001:1001:Alice Johnson:/home/alice:/bin/bash`.

Viewing and Editing User Properties:

- Use **`getent passwd [username]`** to list properties of a user.
- Passwords are stored separately in `/etc/shadow` for security.
- **`vipw`** can edit these files with added safety checks to prevent configuration errors.

System User IDs:

- System users typically have a UID lower than 1000.
- Example: A system user 'daemon' might have a UID of 2.

Special Shell Settings for System Users:

- System users often have /sbin/nologin as their shell to prevent login.
- Some system users might have a specific command as their shell, like /bin/sync for the sync user.

Group Properties in Linux:

- Stored in /etc/group.

Properties include:

- Group Name: Identifies the group.
- Group ID (GID): A unique numerical identifier for the group.
- Group Members: Lists users who are secondary members of the group.
- Primary group membership is managed in /etc/passwd.
- Use **vg** to view or edit group properties.

Configuring Defaults for New Users

Configuring Defaults for New Users in Linux:

- Use **useradd -D** on the command line to view current default settings for new users.
- The primary configuration file is `/etc/login.defs`. This file allows setting defaults that affect only new users, not existing ones.

Settings in `/etc/login.defs`:

- Password Expiration: By default, passwords are set to expire every 99,999 days (about 272 years). It's advisable to reduce this for better security. For example, changing `PASS MAX DAYS` to 90 days is more reasonable.
- Minimum Password Age: Set `PASS MIN DAYS` to define the minimum number of days a password must be used before it can be changed. Changing this to 5 days can enhance security.
- Password Expiry Warning: Adjust the `PASS WARN AGE` to control how many days in advance users are warned about password expiration. Setting this to 7 days is a common practice.

Other Significant Parameters:

- Encryption Method: The default method is SHA512. If security policies require a different standard, it should be set here.
- User Group Settings: `USERGROUPS ENAB` enables private user groups.
- Automatic Home Directory Creation: `CREATE HOME` ensures a home directory is created for new users.

Using /etc/skel for User Home Directories:

- The contents of /etc/skel are copied to a user's home directory upon account creation.
- Example: Adding a file named example.txt to /etc/skel ensures it appears in every new user's home directory.

Applying Settings to New Users:

- When creating a new user (e.g., `sudo useradd peter`), these settings will automatically apply.
- Example: If `PASS MAX DAYS` is set to 90, this will apply to the password of the new user, peter.

Limitations:

- These configurations do not retroactively apply to existing users.
- Linux lacks an easy solution to apply new defaults to previously created users

Practical Demonstration:

- Creating a new user (e.g., `sudo useradd jai`) and checking their home directory (using `ls -l`) will show the effects of these settings, including files copied from /etc/skel

Managing Password Properties

User Password Management in Linux:

- Users can change their password using the **passwd** command.
- Administrators can manage user passwords using **passwd** and **chage** (change age) commands.

Using chage for Password Settings:

- `chage -l` lists current password settings for a user.
- `chage [username]` prompts for changes in password settings, making it user-friendly.
- Example: `chage * Linda` will prompt for changes in Linda's password settings.

Checking Password Status with passwd:

- `passwd -S` shows current password options for a user.
- Example: `sudo passwd -S [username]` displays password status for a specified user

Changing Passwords Non-Interactively:

- On Red Hat: `echo [password] | sudo passwd --stdin [username]`.
- On Ubuntu: `echo [username]:[password] | sudo chpasswd`.
- These methods allow batch updating of passwords, useful in scripts.

Password Storage and Encryption:

- Passwords are stored in `/etc/shadow`.
- This file contains the username, encrypted password hash, and password settings.
- Password encryption uses an algorithm, creating a secure one-way crypto string.

Changing Passwords Interactively:

- Example: `sudo passwd [username]` prompts for a new password twice.
- This method is less efficient for updating multiple users.

Password Complexity and Security:

- Ubuntu employs stricter password policies, often rejecting simple passwords.
- This is due to the Pluggable Authentication Modules (PAM) system, which enforces password complexity.

Additional Examples for Password Management:

- Setting a complex password: `echo [username]:'P@ssw0rd123!' | sudo chpasswd.`
- Checking password expiry: `sudo chage -l [username]`.
- Setting password expiry: `sudo chage -E 2023-01-01 [username]`.

Managing Current Sessions

Introduction to Session Management

- Session management involves managing and monitoring activities on a system.
- It's crucial for understanding who is doing what on your system.

Basic Commands: who and w

- who: Shows users currently logged in.
- w: Similar to who, but provides more details like login time, idle time, CPU usage, and current user activities.
- Example: Running who might show users 'student' and 'linda', while w will show additional details like 'linda' being idle for 17 seconds.

Using loginctl for Advanced Session Management

- Part of the systemd suite, loginctl offers enhanced session management capabilities.
- Commands include:
- loginctl list-sessions: Lists all current sessions with session IDs.
- loginctl show-session [session ID]: Provides detailed information about a specific session.
- loginctl show-user [user]: Displays properties for a specific user.
- loginctl terminate-session [session ID]: Terminates a specific session.
- Additional Example: To terminate a suspicious session, use loginctl terminate-session 5 after identifying the session ID with loginctl list-sessions.

Permissions Management

- Understanding Basic Linux Permissions
- Managing Basic Linux Permissions
- Understanding Advanced Linux Permissions
- Managing Advanced Linux Permissions
- Managing umask

Understanding Basic Linux Permissions

- 1. Introduction to Basic Linux Permissions:** The three basic permissions in Linux: read, write, and execute.
- 2. Using chmod and Octal Mode:** It explains assigning permissions using the 'chmod' command in octal mode, where read equals 4, write equals 2, and execute equals 1. This is applied to 'ugo', which stands for user, group, and others.
- 3. Permissions on Files and Directories:**
 1. Read: On a file, it allows opening and reading the file. On a directory, it enables listing the files in that directory.
 2. Write: On a file, it allows modifying the file. On a directory, it enables adding or removing files. Deleting files is highlighted as a directory operation, not requiring file permissions.
 3. Execute: Essential for running commands and ensuring Linux's security. On directories, it allows changing into the directory.
- 4. Assigning Permissions to ugo:** The importance of assigning permissions to the owner (user), group owner (group), and everyone else (others).
- 5. Summarization of Basic Permissions:**
 1. Read: Allows reading files and listing directory contents.
 2. Write: Enables modifying files and adding/removing files in directories.
 3. Execute: Allows running executable files and changing into directories.

	File	Dir
Read (4)	read	list
Write (2)	modify	add/delete
Execute (1)	run	cd

chmod ugo

Managing Basic Linux Permissions

Managing Permissions in Linux:

- To manage permissions, focus first on ownership, then on setting permissions.
- Standard Linux permissions include one user-owner and one group-owner per file.

Understanding Ownership:

- The creator of a file becomes the user-owner.
- The primary group of the creating user becomes the group-owner.
- All other users are categorized as 'others'.
- This ownership model is abbreviated as UGO (User, Group, Others).

Changing Ownership:

- Use the **chown** command to change user ownership, e.g., **chown alice myfile** or **chown alice:marketing /data/marketing**.
- Use the **chgrp** command to change group ownership independently.

Setting File Permissions with chmod:

- **chmod** can be used in absolute mode (e.g., **chmod 750**), setting permissions for user, group, and others.
- The numbers in **chmod** represent permissions: 7 (read, write, execute), 5 (read, execute), 0 (none).
- Relative mode, like **chmod +x myscript**, is used for quick permission changes.
- Absolute mode is recommended for full control; relative mode for quick changes.

- Practical Demonstration:
- Example of creating a directory /data/sales and checking permissions with `sudo ls -l /data`.
 - `mkdir -p /data/sales`
 - `sudo ls -l /data -- root root`
- Changing group ownership to marketing using
 - `sudo chgrp sales /data/sales` – shows root and sales as group owner
- Setting permission mode to 770 with
 - `sudo chmod 770 /data/sales`
- Testing directory access and file creation as a group member.
 - `grep sales /etc/group -- add rakesh in this group`
 - `sudo su – rakesh`
 - `touch rakeshfile`
 - Now rakesh can create file in this dir

- `sudo touch /home/rakesh/rootfile`
- `sudo su – rakesh`
- `ls -l`
- `rm -rf rootfile` ☐ will this work??
- `ls -ld .` ☐ rakesh is owner of his home dir

examples

Read, Write, and Execute for Owner Only (rwx----- or 700):

- Symbolic: `chmod u=rwx,go= filename`
- Explanation: This command sets the permissions so that only the file owner has read, write, and execute permissions. Group and others have no permissions.

Read and Write for Owner, Read for Group and Others (rw-r--r-- or 644):

- Symbolic: `chmod u=rw,go=r filename`
- Explanation: The owner can read and write the file, but cannot execute it. The group members and others can only read the file.

Read and Execute for Everyone (r-xr-xr-x or 555):

- Symbolic: `chmod a=rx filename`
- Explanation: Everyone can read and execute the file, but no one can write to it.

examples

Write and Execute for Group (----wx--- or 030):

- Symbolic: `chmod g=wx,o= filename`
- Explanation: Only the group members can write and execute the file. The owner and others have no permissions.

Read for Owner and Group, No Permissions for Others (rw-r----- or 640):

- Symbolic: `chmod u=rw,g=r,o= filename`
- Explanation: The owner can read and write, group members can only read, and others have no permissions.

Execute Only for Owner and Group (--x--x--- or 110):

- Symbolic: `chmod u=x,g=x,o= filename`
- Explanation: Both the owner and group members can execute the file, but they can't read or write it. Others have no permissions.

examples

Read, Write, and Execute for Owner Only (rwx----- or 700):

- Numeric: `chmod 700 filename`
- Explanation: Sets permissions so that the owner can read, write, and execute the file, but the group and others have no permissions.

Read and Write for Owner, Read for Group and Others (rw-r--r-- or 644):

- Numeric: `chmod 644 filename`
- Explanation: The owner can read and write, while the group and others can only read the file.

Read and Execute for Everyone (r-xr-xr-x or 555):

- Numeric: `chmod 555 filename`
- Explanation: Everyone can read and execute the file, but no one has write permissions.

examples

Write and Execute for Group (----wx--- or 030):

- Numeric: `chmod 030 filename`
- Explanation: Grants write and execute permissions to the group, while the owner and others have no permissions.

Read for Owner and Group, No Permissions for Others (rw-r----- or 640):

- Numeric: `chmod 640 filename`
- Explanation: The owner has read and write permissions, the group has read permission, and others have no permissions.

Execute Only for Owner and Group (--x--x--- or 110):

- Numeric: `chmod 110 filename`
- Explanation: Both the owner and group members can execute the file, but they can't read or write it. Others have no permissions.

Understanding Advanced Linux Permissions

→ (Permissions) =

↳ User ownership: U

470

-[rwx][rwx][rwx]

4 7 0

4 4 4

SUID File (Run as owner) ✓

SGID (Group owner) ✓

SB

(Share dir) ✓

(Delete file) ✓

chmod 4770 u+s file

1. Introduction to Linux Basic Permissions: Linux basic permissions were introduced long ago, originating from the Unix operating system which dates back to the early 1970s. This initial permissions system had limitations due to its age.
2. Advancements in Permissions System: To address these limitations, Linux has incorporated improvements, including Advanced Permissions.
3. Concept of Set User ID (SUID): SUID, with an octal value of four, is a type of Advanced Permission. When applied to a file, it allows the file to be run as the user who owns that file.
4. Understanding Set Group ID (SGID): SGID, which has an octal value of two, works differently for files and directories. For files, it enables running the file as the group owner. For directories, it sets the group owner for newly created items within that directory.
5. Distinction Between SUID and SGID: SUID is relevant only for files, while SGID applies to both files and directories.
6. Explaining Sticky Bit: Sticky Bit is an Advanced Permission applicable only to directories. It allows a user to delete files within the directory if the user owns the file or the directory.

suid

```
[root@server ~]# su - triel
Last login: Tue Nov 10 11:18:30 EST 2020 on pts/2
[triel@server ~]$
[triel@server ~]$
[triel@server ~]$ passwd^C
[triel@server ~]$
[triel@server ~]$
[triel@server ~]$ which passwd
/bin/passwd
[triel@server ~]$ ls -lrt /bin/passwd
-rwsr-xr-x. 1 root root 27856 Mar 31 2020 /bin/passwd
[triel@server ~]$
```

```
[root@server home]# chmod 770 exampledir/
[root@server home]# ls -lrt
total 64
-rwxr--r--. 1 root      root      171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios    nagios    4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietretech trietretech 4096 Oct 13 02:36 trietretech
drwx----- 3 bkrishna  bkrishna  4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamanth  nkamanth  4096 Oct 27 01:03 nkamanth
drwx----- 4 mibrahim  mibrahim  4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan  kyeshwan  4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov  2 05:07 testuser1
drwx----- 3 mdsaber1  mdsaber1  4096 Nov  3 09:34 mdsaber1
-rwxrwxrwx 1 root      root        0 Nov  6 09:39 abc
-rw-r--r-- 1 root      root        0 Nov 10 10:34 example1
drwxr-xr-x 2 root      root      4096 Nov 10 10:36 example2
-rw-r----- 1 root      root        0 Nov 10 10:36 example3
drwxr-x--- 2 root      root      4096 Nov 10 10:37 example4
drwx----- 2 triel     triel     4096 Nov 10 10:57 triel
-rwsr-xr-x 1 root      root        0 Nov 10 10:57 suid
drwxr-xr-x 3 root      root      4096 Nov 10 11:16 data
drwx----- 2 trie2     trie2     4096 Nov 10 11:18 trie2
drwxrwx--- 2 root      example   4096 Nov 10 11:22 exampledir
```

sgid

```
[root@server ~]# cd /home/
[root@server home]# mkdir exampledir
[root@server home]# chown :example exampledir/
[root@server home]# ls -lrt exampledir/
total 0
[root@server home]# ls -lrt
total 64
-rwxr--r--. 1 root      root      171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios    nagios    4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietretech trietretech 4096 Oct 13 02:36 trietretech
drwx----- 3 bkrishna  bkrishna  4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamanth  nkamanth  4096 Oct 27 01:03 nkamanth
drwx----- 4 mibrahim  mibrahim  4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan  kyeshwan  4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov  2 05:07 testuser1
drwx----- 3 mdsaber1  mdsaber1  4096 Nov  3 09:34 mdsaber1
-rwxrwxrwx 1 root      root        0 Nov  6 09:39 abc
-rw-r--r-- 1 root      root        0 Nov 10 10:34 example1
drwxr-xr-x 2 root      root      4096 Nov 10 10:36 example2
-rw-r----- 1 root      root        0 Nov 10 10:36 example3
drwxr-x--- 2 root      root      4096 Nov 10 10:37 example4
drwx----- 2 triel     triel     4096 Nov 10 10:57 triel
1 root      root        0 Nov 10 10:57 suid
3 root      root      4096 Nov 10 11:16 data
2 trie2     trie2     4096 Nov 10 11:18 trie2
2 root      example   4096 Nov 10 11:22 exampledir
home]#
```

Giving full permissions to the group with 770

Refer this Video for interviews

<https://www.youtube.com/watch?v=eupeuONcGOY>


```

-rwxrwxrwx 1 root root 0 Nov 6 09:39 abc
[root@server home]#
[root@server home]# touch testfile
[root@server home]# ls -lrt
total 40
-rwxr--r-- 1 root root 171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios nagios 4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietreotech trietreotech 4096 Oct 13 02:36 trietreotech
drwx----- 3 bkrishna bkrishna 4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamant nkamant 4096 Oct 27 01:03 nkamant
drwx----- 4 mibrahim mibrahim 4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan kyeshwan 4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov 2 05:07 testuser1
drwx----- 3 mdsaber1 mdsaber1 4096 Nov 3 09:34 mdsaber1
-rwxrwxrwx 1 root root 0 Nov 6 09:39 abc
-rw-r--r-- 1 root root 0 Nov 10 10:26 testfile
[root@server home]# chown testuser1 testfile
[root@server home]# ls -lrt
total 40
-rwxr--r-- 1 root root 171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios nagios 4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietreotech trietreotech 4096 Oct 13 02:36 trietreotech
drwx----- 3 bkrishna bkrishna 4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamant nkamant 4096 Oct 27 01:03 nkamant
drwx----- 4 mibrahim mibrahim 4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan kyeshwan 4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov 2 05:07 testuser1
drwx----- 3 mdsaber1 mdsaber1 4096 Nov 3 09:34 mdsaber1
-rwxrwxrwx 1 root root 0 Nov 6 09:39 abc
-rw-r--r-- 1 testuser1 root 0 Nov 10 10:26 testfile
[root@server home]#

```

```

[root@server home]# ls -lrt
total 40
-rwxr--r-- 1 root root 171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios nagios 4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietreotech trietreotech 4096 Oct 13 02:36 trietreotech
drwx----- 3 bkrishna bkrishna 4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamant nkamant 4096 Oct 27 01:03 nkamant
drwx----- 4 mibrahim mibrahim 4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan kyeshwan 4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov 2 05:07 testuser1
drwx----- 3 mdsaber1 mdsaber1 4096 Nov 3 09:34 mdsaber1
-rwxrwxrwx 1 root root 0 Nov 6 09:39 abc
-rw-r--r-- 1 testuser1 root 0 Nov 10 10:26 testfile
[root@server home]# chgrp testuser1 testfile
[root@server home]# ls -lrt
total 40
-rwxr--r-- 1 root root 171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios nagios 4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietreotech trietreotech 4096 Oct 13 02:36 trietreotech
drwx----- 3 bkrishna bkrishna 4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamant nkamant 4096 Oct 27 01:03 nkamant
drwx----- 4 mibrahim mibrahim 4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan kyeshwan 4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov 2 05:07 testuser1
drwx----- 3 mdsaber1 mdsaber1 4096 Nov 3 09:34 mdsaber1
-rwxrwxrwx 1 root root 0 Nov 6 09:39 abc
-rw-r--r-- 1 testuser1 testuser1 0 Nov 10 10:26 testfile
[root@server home]# ch

```

```

[root@server home]# chown root:root testfile
[root@server home]# ls -lrt
total 40
-rwxr--r-- 1 root root 171 Aug 17 06:07 userdir.sh
drwx----- 2 nagios nagios 4096 Oct 12 02:38 nagios
drwxr-x--- 12 trietreotech trietreotech 4096 Oct 13 02:36 trietreotech
drwx----- 3 bkrishna bkrishna 4096 Oct 26 11:58 bkrishna
drwx----- 4 nkamant nkamant 4096 Oct 27 01:03 nkamant
drwx----- 4 mibrahim mibrahim 4096 Oct 29 05:52 mibrahim
drwx----- 4 kyeshwan kyeshwan 4096 Oct 31 00:44 kyeshwan
drwx----- 3 testuser1 testuser1 4096 Oct 31 00:55 dummy
drwx----- 2 testuser1 testuser1 4096 Nov 2 05:07 testuser1
drwx----- 3 mdsaber1 mdsaber1 4096 Nov 3 09:34 mdsaber1
-rwxrwxrwx 1 root root 0 Nov 6 09:39 abc
-rw-r--r-- 1 root root 0 Nov 10 10:26 testfile
[root@server home]#

```

umask

Introduction to Umask

- Definition: Umask is a shell setting that influences default file and directory permissions by subtracting a defined mask from the default permissions.

Default Permissions:

- Directories: 777
- Files: 666
- Understanding Umask Values

Umask 022:

- Files will have permissions 644 (default 666 - 022)
- Directories unaffected (default 777 - 022 = 755)

Umask 027:

- Directories will have permissions 750 (default 777 - 027)
- Practical Demonstration

Locating Umask Settings:

- Command: `sudo grep umask /etc/*` (recursive search)
- Common locations: `/etc/bashrc`, `/etc/login.defs`

Example from /etc/bashrc:

- For user IDs > 199, umask is 002
- Otherwise, umask is 022
- Setting Umask for Specific Users
- Editing User Specific Files:
- Example: `sudo vim /home/Linda/.bashrc`
- Set umask: Add `umask 002` in `.bashrc`

Effect of Umask 002:

- User and group have full permissions
- Others have no permissions
- Verifying Umask Settings

Checking Umask for User:

- Example: Login as Linda and type `umask` to verify
- Creating Files with New Umask:
- Commands: `touch newfile` and `ls -l newfile`
- Verifying file permissions according to the set umask
- Applying Umask in Shared Environments

Editing /etc/skel:

- Modify bashrc in /etc/skel to include desired umask (e.g., 002)
- This sets the default for new users

Caution

- Avoid Editing System-Wide Configuration:
- Risk of locking out users due to errors
- Prefer user-specific .bashrc modifications

Managing Networking

- Understanding IPv4 Basics
- Understanding IPv6 Basics
- Applying Run-time Network Configuration
- Understanding Network Device Naming
- Managing Host Names
- Managing Host Name Resolution
- Using Common Network Tools
- Configuring Persistent Networking

Understanding IPv4 Basics

1. The Concept of the Internet:

1. The internet is visualized as a cloud-like shape.
2. It's the ultimate destination for networked devices.

2. Role of the Router:

1. A router manages connections to the internet.
2. It exists on your local network.

3. Local Network Structure:

1. Local network behind the router.
2. Nodes: Devices with IP addresses.
3. Networks can be simple or complex, with possibly more routers.

4. IP Configuration on Nodes:

1. Each node needs an IP configuration.
2. IP (Internet Protocol) is the default protocol for networking.

5. Understanding IP Addresses:

1. Example of an IP address: 192.168.1.12/24.
2. "/24" indicates the network part of the IP address.
3. The network can be identified as 192.168.1.0/24.

6. Router's IP Address:

1. Typically at 192.168.1.254 or similar.
2. Acts as the default route/gateway.

7. DNS Configuration:

1. DNS stands for Domain Name System.
2. It translates domain names to IP addresses.
3. Essential for easy internet navigation.

Introduction to IPv6:

Point: IPv6 is the successor to IPv4 due to the shortage of IPv4 addresses.

- Example: IPv4 uses 32-bit addresses (e.g., 192.168.1.1), while IPv6 uses 128-bit addresses (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334), allowing for a much larger number of unique addresses.
- Insight: The transition to IPv6 is essential for the continued growth of the internet, as it provides a virtually inexhaustible pool of IP addresses.

IPv6 Addressing Format:

- Point: IPv6 addresses are written in hexadecimal, divided into 8 groups of 4 hex digits.
- Example: An address like 2001:0db8:85a3:0000:0000:8a2e:0370:7334 is a typical IPv6 address.
- Insight: Hexadecimal format allows for a more compact representation of the 128-bit addresses.

Simplifying IPv6 Addresses:

- Point: You can omit leading zeros and use :: to represent consecutive zeros.
- Example: 2001:db8:85a3::8a2e:370:7334 is a simplified form of the previous example.
- Insight: This simplification makes IPv6 addresses easier to read and use.

IPv6 and Network Ports:

- Point: When specifying a port with IPv6, enclose the address in square brackets.
- Example: [2001:db8:85a3::1]:80 indicates port 80 on the IPv6 address.
- Insight: This is necessary because colons are used in both IPv6 addresses and to separate IP addresses from ports.
- Default Subnet Mask in IPv6:

Reserved IPv6 Addresses:

Point: Certain IPv6 addresses have specific purposes.

- Examples:
- `::1` is the loopback address, equivalent to IPv4's 127.0.0.1.
- `::` represents 0.0.0.0
- `::/0` represents the default route.
- `2000::/3` denotes global unicast addresses.
- `fd00::` are unique local addresses.
- `fe80::` are link-local addresses.
- `ff00::` are multicast addresses.
- Insight: These reserved addresses facilitate various network functionalities and protocols.

Applying Run-time Network Configuration

Understanding IP Address Acquisition:

- Servers often use static IP address configuration through tools like NMCLI or NMTUI.
- Workstations typically receive IP addresses from a DHCP server.
- Example: On a Linux server, you might set a static IP using `sudo nmcli con mod [connection name] ipv4.addresses [desired IP address]`.

DHCP Explained:

- DHCP stands for Dynamic Host Configuration Protocol.
- It's used in networks to automatically assign IP addresses to devices.
- Insight: DHCP simplifies network management by avoiding the need for manual IP address configuration.

Runtime Network Configuration:

- This refers to temporary network settings not saved after a reboot.
- The `ip` command is essential for setting and checking runtime configurations.
- Example: To view current IP configuration, use `sudo ip a` (short for `ip addr show`).

Obsolescence of ifconfig:

- `ifconfig` is considered outdated and not accurate for modern Linux networking.
- It has been replaced by `ip` and other tools due to significant changes in the Linux network stack.
- Example: Instead of `ifconfig`, use `ip a` to view network interfaces.

Adding IP Addresses in Runtime:

- Use **ip addr add dev [interface] [IP address]** for adding IP addresses.
- Requires superuser permissions (sudo).
- Example: To add an IP to the ENS33 interface, use `sudo ip addr add dev ens33 192.168.1.2`.

ifconfig's Limitations:

- ifconfig does not properly display certain network configurations due to its outdated design.
- Example: After adding a secondary IP using `ip addr add`, ifconfig may not show this new IP.

Platform-Specific Tools:

- ifconfig remains the default utility in Mac iOS, a BSD Unix distribution, unlike in Linux.
- Insight: Network tools can vary significantly across different operating systems.

Managing Network Interface States:

- Use **ip link set [interface] up** to activate a network interface.
- This can be useful if the link is down or after changing configurations.
- Example: To reactivate the ENS33 interface, use `sudo ip link set ens33 up`.

Viewing and Modifying the Routing Table:

- The **ip route show** command displays the routing table, including the default route.
- Adding a default route is done via `ip route add default via [gateway IP]`.
- Insight: Correct routing configuration is crucial for network communication.

```
[cloud_user@ansible1 ~]$ ip route show
default via 172.31.32.1 dev ens5 proto dhcp src 172.31.38.253 metric 100
172.31.32.0/20 dev ens5 proto kernel scope link src 172.31.38.253 metric 100
```

- Imagine a city with roads (networks) and destinations (computers or devices). Each destination has a unique address, similar to how devices on a network have unique IP addresses.

1. **default via 172.31.32.1 dev ens5 proto dhcp src 172.31.38.253 metric 100:**

1. **default via 172.31.32.1:** This is like saying, "For any destination (website or online service) not in this city (local network), use the main highway exit (gateway) at 172.31.32.1 to get there." In networking terms, this is your default gateway, the primary route your computer uses to reach the internet or any other network outside your local network.
2. **dev ens5:** This specifies the road (network interface) you're using to leave the city. 'ens5' is the name of this road. In computer terms, it's the name of the network interface your computer uses to connect to the network.
3. **proto dhcp:** This indicates how you got your map or directions (IP address). DHCP (Dynamic Host Configuration Protocol) is like a service in the city that automatically gives you a map with directions (an IP address and other network settings) when you arrive.
4. **src 172.31.38.253:** This is your house's address (your device's IP address) in the city.
5. **metric 100:** Think of this as the preference of the route. A lower number means a more preferred route. This number helps in deciding which road to take when there are multiple ways to reach a destination.

2. **172.31.32.0/20 dev ens5 proto kernel scope link src 172.31.38.253 metric 100:**

1. **172.31.32.0/20:** This is like a neighborhood in your city. The '/20' is a way to describe the size of the neighborhood. In networking, it's a range of addresses that are considered local (they don't need to use the main highway to reach each other).
2. **dev ens5:** Again, this is the road (network interface) for this part of the city.
3. **proto kernel:** This indicates that this route (or set of directions) is managed by the city's central planning system (the kernel in your computer's operating system).
4. **scope link:** This means that this route is valid only for direct communication in the neighborhood. It's like saying, "These directions are only good for places within walking distance."
5. **src 172.31.38.253:** Your house's address in this neighborhood.
6. **metric 100:** The preference for this route when choosing how to reach destinations within the neighborhood.

Understanding Network Device Naming

Introduction to Network Device Names in Linux

- Old Linux Naming Convention
- Example: In older Linux versions, the first Ethernet device was named eth0, and a second one would be eth1. A wireless LAN device was named wlan0.
- Insight: This naming was straightforward but lacked information about the device's physical location, which could be a limitation in complex setups.

Modern Linux Naming Conventions

- Physical Location-Based Naming
- Example: In modern Linux, a network device might be named em123, indicating it is an embedded device. Another example is POP1, representing PCI port zero and PCI slot number one.
- Insight: This change was driven by the need for better identification in systems with multiple network cards, such as servers.

Biosdevname and its Role

- Biosdevname Functionality
- Example: Biosdevname might assign a name like eno123, which stands for Ethernet Onboard, indicating a network device integrated into the motherboard.
- Insight: Biosdevname is used for generating names that reflect the physical characteristics and location of the network device.

System D Component: system-udev

- Role in Device Naming
- Example: The system D component, system-udev, generates network device names based on information from the driver.
- Insight: This system is responsible for the assignment of the new naming conventions, bridging the hardware information with the operating system.

Legacy Names and Transition

- Continued Existence of Old Names
- Example: In some cases where the driver doesn't provide enough information, traditional names like eth0 might still appear.
- Insight: Although rare in modern setups, understanding legacy names like eth0 is still relevant for compatibility and legacy systems.

ENS Interface Names in Virtual Environments

- Example in VMware
- Example: In a VMware virtual environment, network interfaces are named using the ENS naming convention, like ens33 or ens192.
- Insight: This showcases the adoption of the new naming convention in virtualized environments, emphasizing the importance of understanding these names in diverse setups.
- The "ens" in network device names like ens33 in Linux stands for "Ethernet Network System." This naming convention is part of the predictable network interface names introduced in recent Linux distributions. It's used to designate Ethernet devices in a way that is consistent and predictable, based on hardware topology and characteristics.

Managing Host Names

Importance of Correct Hostname Setting in Linux:

- Insight: Hostnames are vital for the correct operation of networked Linux systems. They facilitate communication and identification among different hosts.
- Example: If a Linux machine is named incorrectly, it may fail to communicate effectively with other network devices.

Hostname Resolution and Its Significance:

- Insight: Hostname resolution is crucial for network operations, especially for reverse hostname lookups during inter-host communication.
- Example: When a server receives a connection request, it might use reverse hostname lookup to find the hostname associated with the IP address of the incoming connection.

Using `hostnamectl` Command:

- Insight: `hostnamectl` is a modern tool for managing hostnames in Linux, providing detailed information about the system.
- Example: To view the current system hostname and related info, use **`hostnamectl status`**.

Setting Hostname with `hostnamectl`:

- Insight: The command allows setting both Fully Qualified Domain Names (FQDN) and short hostnames.
- Example: To set a new hostname, use **`sudo hostnamectl centos.example.com`**. Afterward, running **`hostnamectl status`** will display the updated hostname.

Editing /etc/hosts for Local Hostname Resolution:

- Insight: The /etc/hosts file is used for local hostname resolution, mapping IP addresses to hostnames.
- Example: To add a hostname, edit the file and add a line like 192.168.29.141 centos.example.com centos. This maps the IP to both the FQDN and the short hostname.

Testing Hostname Resolution Locally:

- Insight: After updating /etc/hosts, local utilities like ping can be used to test hostname resolution.
- Example: To test, execute ping centos or ping centos.example.com to verify that the system correctly resolves the hostname to the IP address.

Difference Between Local and Internet Hostname Resolution:

- Insight: While /etc/hosts is for local resolution, DNS is used for resolving hostnames over the internet.
- Example: The hostname google.com is resolved using DNS, but centos.example.com could be resolved locally via /etc/hosts if specified.

Managing Host Name Resolution

Understanding Host Name Resolution:

- Host name resolution is the process of mapping host names to IP addresses and vice versa.
- Example: Resolving 'example.com' to its corresponding IP address.

Local etc/hosts File:

- The etc/hosts file is a primary solution for host name resolution.
- Example: If 'example.com' is listed in the etc/hosts file with a specific IP, that IP will be used for resolution.
- Insight: This method is useful for overriding external DNS resolutions or for local network configurations.

DNS Name Servers Configuration:

- DNS name servers are specified in the etc/resolv.conf file.
- Example: Specifying a DNS server like 8.8.8.8 (Google DNS) in etc/resolv.conf for general internet domain resolutions.
- Insight: DNS is crucial for resolving public domain names that are not specified in the local etc/hosts file.

Order of Host Name Resolution:

- The order of host name resolution can be managed in the `etc/nsswitch.conf` file.
- Example: The hosts line in `nsswitch.conf` might read `'hosts: files dns'`, indicating that the `etc/hosts` file is checked before DNS servers.
- Insight: Adjusting the order can prioritize local resolutions over external ones, which is useful for specific network configurations.

Practical Demonstration with Ping Command:

- Example: Using `'ping google.com'` initially resolves to Google's IP. After adding `'google.com'` to the `etc/hosts` file with the local machine's IP, `'ping google.com'` resolves to the local machine.
- Insight: This shows that the `etc/hosts` file has higher priority in the resolution order (as per the `nsswitch.conf` settings).

Security Implications and Issues:

- Example: Adding a dummy entry for a service in the `etc/hosts` file can prevent a system from accessing the real service over the internet.
- Insight: This can be a rudimentary form of security to block certain services, though it's not the most elegant solution.
- Caution: Using local host names that also exist on the internet in the `etc/hosts` file can lead to conflicts and connectivity issues, as local resolution takes precedence.

To further illustrate these concepts, let's consider a practical scenario:

- Suppose you have a local development server for 'myproject.com'. You can add an entry in your etc/hosts file to resolve 'myproject.com' to your local server IP, such as '192.168.1.10'. This means that when you access 'myproject.com' from your machine, it will direct you to your local server instead of the actual website on the internet.
- If you change the order in etc/nsswitch.conf to 'hosts: dns files', then DNS resolution will be attempted before checking the etc/hosts file. In this case, accessing 'myproject.com' will lead you to the real website on the internet, not your local server, unless the DNS fails to resolve.

Remember, while these methods are powerful for network management and testing, they should be used with an understanding of their impact on your system's network behavior.

Using Common Network Tools

Using Ping for Host Reachability

- Example: `sudo ping ubuntu`
- Insight: Ping continuously sends packets until stopped manually (Ctrl+C). It's a simple way to check if a host is reachable and measure response times. To limit ping attempts, `-c` option can be used (e.g., `-c 3` for three attempts).

Socket Statistics with ss

- Example: `ss -tunap`
- Insight: ss provides detailed info about open ports and socket connections. Options like `t` (TCP), `u` (UDP), and `nap` (additional protocol info) refine the output, showing active network and local sockets.

DNS Testing with dig

- Example: `dig techitfactory.com`
- Insight: dig is useful for obtaining DNS information, including resolving domain names to IP addresses. It also identifies the DNS server providing the response. The NXDOMAIN result indicates no such domain exists.

Network Analysis with nmap

- Example: **sudo nmap -sn** (network scanning), **sudo nmap <host>** (scanning specific host)
- Insight: nmap is powerful for network analysis, revealing hosts and open ports on a network. It should be used cautiously, as scanning external networks can be seen as hostile.

Deprecated Tools: netstat and nslookup

- Insight: Both **netstat** and **nslookup** are outdated, with ss and dig being their modern replacements, respectively. However, in minimal Linux environments (like BusyBox, Alpine), these older tools might still be necessary.

Practical Application and Caution

- Insight: While these tools are essential for network analysis and troubleshooting, they must be used responsibly, especially nmap, to avoid unintended security implications or being perceived as conducting an attack.

Deprecated Tools in Cloud Environments

- Insight: Knowledge of deprecated tools like netstat and nslookup is still valuable, particularly in cloud environments with minimal Linux distributions where modern tools might not be available.

Storage Management Essentials

- Understanding Linux Storage Solutions
- Creating MBR Partitions
- Creating GPT Partitions
- Creating File Systems
- Mounting File Systems
- Making Mounts Persistent

Understanding Linux Storage Solutions

1. Introduction to Disk Allocation

1. Example: Visualize a computer's disk as a space to store data.
2. Insight: Disk allocation is essential for organizing data efficiently.

2. Understanding Partitions in Disk Allocation

1. Example: Compare a disk without partitions to a disorganized library.
2. Insight: Partitions help in segmenting the disk for better data management.

3. Partitioning Schemes: MBR vs GPT

1. MBR (Master Boot Record)
 1. Example: MBR with a limit of 4 partitions per disk.
 2. Insight: MBR is older and has limitations in terms of partition numbers.
2. GPT (GUID Partition Table)
 1. Example: GPT allowing up to 128 partitions.
 2. Insight: GPT is more modern with fewer limitations than MBR.

4. Dealing with MBR Limitations

1. Example: Creating an extended partition when more than four are needed.
2. Insight: Extended partitions in MBR are a workaround to surpass the 4-partition limit.

5. Extended and Logical Partitions in MBR

1. Example: Turning the fourth partition into an extended one to create additional logical partitions.
2. Insight: Logical partitions within an extended one provide flexibility in MBR.

6. Simplicity of GPT Partitioning

1. Example: Directly creating multiple partitions in GPT without the need for extended/logical partitions.
2. Insight: GPT's design simplifies partitioning compared to MBR.

- **Benefits of GPT over MBR:**

- Example: GPT allows up to 128 partitions, while MBR limits to 4 primary partitions (or 15 using extended/logical partitions).
- Insight: GPT is essential for disks larger than 2TB, a size becoming common in modern computing.

7. Demonstration with Block Devices

1. Command: "lsblk" to list block devices.
2. Example: Identifying different storage devices and partitions like SDA, SDB, and NVME disks.
3. Insight: Understanding the layout and structure of storage on a system.

8. Exploring the Partition File Structure

4. Command: "cat /proc/partitions" to view partition details.
5. Insight: This file provides a different perspective on partition layout.

9. Using fdisk for Partition Management

6. Command: "fdisk -l /dev/sda" to view partition information on a specific disk.
7. Example: Viewing partition details of the first SCSI disk.
8. Insight: fdisk is a powerful tool for managing disk partitions.

Understanding Common Block Device Names:

- `/dev/sda`: First SCSI hard disk device.
- `/dev/sdb`: Second SCSI hard disk device.
- `/dev/vda`: Disk in a KVM virtual machine.
- `nvme0n1`: First NVME hard disk (commonly SSDs).
- `/dev/sr0`: Optical drive.
- Example: Identifying these devices using `lsblk` to manage different storage devices.

Exploring Linux Storage Options

Partitions in Storage Systems:

Explanation: Partitions are traditional methods for allocating storage to specific data types.

Example: For instance, a partition could be used exclusively for system files, while another could be allocated for user data.

Insight: Partitions, while useful, lack flexibility in resizing and managing storage needs.

Logical Volume Manager (LVM):

Explanation: LVM is a more flexible storage solution used in RHEL default installations.

Example: If you need to expand a volume, LVM allows you to do so without needing to repartition the entire disk.

Insight: LVM offers advanced features like resizing volumes, snapshots, and more, enhancing storage management.

Understanding Block Devices:

- Explanation: Linux uses block devices for storage, and these are allocated based on the driver used.
- Example: Common block devices include `/dev/sd` for SCSI and SATA disks, `/dev/vd` in KVM virtual machines, and `/dev/nvme` for NVMe devices.
- Insight: Knowing the type of block device helps in understanding the underlying hardware and storage allocation.

Using lsblk Command:

- Explanation: The lsblk command lists block devices and their information.
- Example: Running lsblk in the terminal will show you devices like /dev/sda (first SCSI/SATA disk) or /dev/nvme0n1 (first NVMe disk).
- Insight: This command is crucial for getting a clear overview of the storage devices and their partitions on your system.

Partition Naming Conventions:

- Explanation: Partition names differ based on the device types.
- Example: On SD and VD devices, partitions are numbered sequentially (e.g., /dev/sda2 for the second partition on the first hard disk). On NVMe devices, partition names are appended (e.g., /dev/nvme0n1p1 for the first partition on the first NVMe disk).
- Insight: Understanding naming conventions helps in identifying specific partitions and disks, especially when managing multiple storage devices.

Importance of lsblk for Partition Management:

- Explanation: lsblk reads the kernel partition table and provides accurate information about partitions.
- Example: To avoid confusion about partition names and numbers, use lsblk to get a clear view of the partition layout on your disks.
- Insight: Regularly using lsblk can help in efficiently managing and troubleshooting storage-related issues.

Creating Partitions with fdisk

```
[jai@control ~]$ lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
sr0	11:0	1	109.9M	0	rom	/run/media/jai/CDROM
sr1	11:1	1	8.9G	0	rom	/run/media/jai/RHEL-9-2-0-BaseOS-x86_64
nvme0n1	259:0	0	100G	0	disk	
└─nvme0n1p1	259:1	0	1G	0	part	/boot
└─nvme0n1p2	259:2	0	99G	0	part	
└─rhel-root	253:0	0	65.2G	0	lvm	/
└─rhel-swap	253:1	0	2G	0	lvm	[SWAP]
└─rhel-home	253:2	0	31.8G	0	lvm	/home

```
[jai@control ~]$
```

```
[root@control ~]# fdisk /dev/nvme0n1
```

```
Welcome to fdisk (util-linux 2.37.4).
```

```
Changes will remain in memory only, until you decide to write them.  
Be careful before using the write command.
```

```
This disk is currently in use - repartitioning is probably a bad idea.  
It's recommended to umount all file systems, and swapoff all swap  
partitions on this disk.
```

```
Command (m for help): █
```

Command (m for help): p

- Command (m for help): p
 - Disk /dev/nvme0n1: 110 GiB, 118111600640 bytes, 230686720 sectors
 - Disk model: VMware Virtual NVMe Disk
 - Units: sectors of 1 * 512 = 512 bytes
 - Sector size (logical/physical): 512 bytes / 512 bytes
 - I/O size (minimum/optimal): 512 bytes / 512 bytes
 - Disklabel type: dos
 - Disk identifier: 0x5485b4ab
-
- | Device | Boot | Start | End | Sectors | Size | Id | Type |
|----------------|------|---------|-----------|-----------|------|----|-----------|
| /dev/nvme0n1p1 | * | 2048 | 2099199 | 2097152 | 1G | 83 | Linux |
| /dev/nvme0n1p2 | | 2099200 | 209715199 | 207616000 | 99G | 8e | Linux LVM |

- Command (m for help): n
- Partition type
 - p primary (2 primary, 0 extended, 2 free)
 - e extended (container for logical partitions)
- Select (default p):

Command (m for help): n

- Partition type
 - p primary (2 primary, 0 extended, 2 free)
 - e extended (container for logical partitions)
 - Select (default p): p
 - Partition number (3,4, default 3):
 - First sector (209715200-230686719, default 209715200):
 - Last sector, +/-sectors or +/-size{K,M,G,T,P} (209715200-230686719, default 230686719): +1G
-
- Created a new partition 3 of type 'Linux' and of size 1 GiB.

Command (m for help): p

- Disk /dev/nvme0n1: 110 GiB, 118111600640 bytes, 230686720 sectors
- Disk model: VMware Virtual NVMe Disk
- Units: sectors of 1 * 512 = 512 bytes
- Sector size (logical/physical): 512 bytes / 512 bytes
- I/O size (minimum/optimal): 512 bytes / 512 bytes
- Disklabel type: dos
- Disk identifier: 0x5485b4ab

- | Device | Boot | Start | End | Sectors | Size | Id | Type |
|----------------|------|-----------|-----------|-----------|------|----|-----------|
| /dev/nvme0n1p1 | * | 2048 | 2099199 | 2097152 | 1G | 83 | Linux |
| /dev/nvme0n1p2 | | 2099200 | 209715199 | 207616000 | 99G | 8e | Linux LVM |
| /dev/nvme0n1p3 | | 209715200 | 211812351 | 2097152 | 1G | 83 | Linux |

Command (m for help): w

- The partition table has been altered.
- Syncing disks.

- [jai@control ~]\$ lsblk
- NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
- sr0 11:0 1 109.9M 0 rom /run/media/jai/CDROM
- sr1 11:1 1 8.9G 0 rom /run/media/jai/RHEL-9-2-0-BaseOS-x86_64
- nvme0n1 259:0 0 110G 0 disk
- └─nvme0n1p1 259:1 0 1G 0 part /boot
- └─nvme0n1p2 259:2 0 99G 0 part
- | └─rhel-root 253:0 0 65.2G 0 lvm /
- | └─rhel-swap 253:1 0 2G 0 lvm [SWAP]
- | └─rhel-home 253:2 0 31.8G 0 lvm /home
- └─nvme0n1p3 259:3 0 1G 0 part

Extended partition

- Command (m for help): n
 - Partition type
 - p primary (3 primary, 0 extended, 1 free)
 - e extended (container for logical partitions)
 - Select (default e): e
-
- Selected partition 4
 - First sector (211812352-230686719, default 211812352):
 - Last sector, +/-sectors or +/-size{K,M,G,T,P} (211812352-230686719, default 230686719): +2G
-
- Created a new partition 4 of type 'Extended' and of size 2 GiB.
-
- Command (m for help): n
 - All primary partitions are in use.
 - Adding logical partition 5
 - First sector (211814400-216006655, default 211814400):
 - Last sector, +/-sectors or +/-size{K,M,G,T,P} (211814400-216006655, default 216006655): +2G
 - Value out of range.
 - Last sector, +/-sectors or +/-size{K,M,G,T,P} (211814400-216006655, default 216006655): +1.5G
-
- Created a new partition 5 of type 'Linux' and of size 1.5 GiB.

- Command (m for help): p
- Disk /dev/nvme0n1: 110 GiB, 118111600640 bytes, 230686720 sectors
- Disk model: VMware Virtual NVMe Disk
- Units: sectors of 1 * 512 = 512 bytes
- Sector size (logical/physical): 512 bytes / 512 bytes
- I/O size (minimum/optimal): 512 bytes / 512 bytes
- Disklabel type: dos
- Disk identifier: 0x5485b4ab

- Device Boot Start End Sectors Size Id Type
- /dev/nvme0n1p1 * 2048 2099199 2097152 1G 83 Linux
- /dev/nvme0n1p2 2099200 209715199 207616000 99G 8e Linux LVM
- /dev/nvme0n1p3 209715200 211812351 2097152 1G 83 Linux
- /dev/nvme0n1p4 211812352 216006655 4194304 2G 5 Extended
- /dev/nvme0n1p5 211814400 214960127 3145728 1.5G 83 Linux

- Command (m for help): w
- The partition table has been altered.
- Syncing disks.

Format FS

- [jai@control ~]\$ sudo mkfs.xfs /dev/nvme0n1p3
- meta-data=/dev/nvme0n1p3 isize=512 agcount=4, agsize=65536 blks
- = sectsz=512 attr=2, projid32bit=1
- = crc=1 finobt=1, sparse=1, rmapbt=0
- = reflink=1 bigtime=1 inobtcount=1
- data = bsize=4096 blocks=262144, imaxpct=25
- = sunit=0 swidth=0 blks
- naming =version 2 bsize=4096 ascii-ci=0, ftype=1
- log =internal log bsize=4096 blocks=2560, version=2
- = sectsz=512 sunit=0 blks, lazy-count=1
- realtime =none extsz=4096 blocks=0, rtextents=0

Mount

- `[jai@control ~]$ sudo mount /dev/nvme0n1p3 /mnt`
- `[jai@control ~]$ lsblk`
- NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
- sr0 11:0 1 109.9M 0 rom /run/media/jai/CDROM
- sr1 11:1 1 8.9G 0 rom /run/media/jai/RHEL-9-2-0-BaseOS-x86_64
- nvme0n1 259:0 0 110G 0 disk
- └─nvme0n1p1 259:1 0 1G 0 part /boot
- └─nvme0n1p2 259:2 0 99G 0 part
- | └─rhel-root 253:0 0 65.2G 0 lvm /
- | └─rhel-swap 253:1 0 2G 0 lvm [SWAP]
- | └─rhel-home 253:2 0 31.8G 0 lvm /home
- └─**nvme0n1p3 259:3 0 1G 0 part /mnt**
- └─nvme0n1p4 259:4 0 1K 0 part
- └─nvme0n1p5 259:5 0 1.5G 0 part
- └─nvme0n1p6 259:6 0 100M 0 part
- `[jai@control ~]$ mount | grep '^/'`
- `/dev/mapper/rhel-root on / type xfs (rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota)`
- `/dev/nvme0n1p1 on /boot type xfs (rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota)`
- `/dev/mapper/rhel-home on /home type xfs (rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota)`
- `/dev/sr0 on /run/media/jai/CDROM type iso9660 (ro,nosuid,nodev,relatime,nojoliet,check=s,map=n,blocksize=2048,uid=1000,gid=1000,dmode=500,fmode=400,uhelper=udisks2)`
- `/dev/sr1 on /run/media/jai/RHEL-9-2-0-BaseOS-x86_64 type iso9660 (ro,nosuid,nodev,relatime,nojoliet,check=s,map=n,blocksize=2048,uid=1000,gid=1000,dmode=500,fmode=400,uhelper=udisks2)`

- [jai@control ~]\$ findmnt

TARGET	SOURCE	FSTYPE	OPTIONS
/	/dev/mapper/rhel-root		
		xfs	rw,relatime,seclabel,attr2,inode64,l
└─/proc	proc	proc	rw,nosuid,nodev,noexec,relatime
└─└─/proc/sys/fs/binfmt_misc	systemd-1	autofs	rw,relatime,fd=29,pgrp=1,timeout=0,m
└─/sys	sysfs	sysfs	rw,nosuid,nodev,noexec,relatime,secl
└─└─/sys/kernel/security	securityfs	securit	rw,nosuid,nodev,noexec,relatime

UnMount

- [jai@control ~]\$ cd /mnt
- [jai@control mnt]\$ ls
- [jai@control mnt]\$ umount /mnt
- umount: /mnt: must be superuser to unmount.
- [jai@control mnt]\$ lsof /mnt
- COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
- bash 2955 jai cwd DIR 259,3 6 128 /mnt
- lsof 3212 jai cwd DIR 259,3 6 128 /mnt
- lsof 3213 jai cwd DIR 259,3 6 128 /mnt
- [jai@control mnt]\$ cd ..
- [jai@control /]\$ sudo umount /mnt

- [root@control ~]# mkdir /tifdata
- [root@control ~]#
- [root@control ~]# vi /etc/fstab
- TypeError:

```
/dev/nvme0n13    /tifdata    xfs    defaults    0 0
```

- [root@control ~]# mount -a
- mount: /tifdata: special device /dev/nvme0n13 does not exist.
- mount: (hint) your fstab has been modified, but systemd still uses
- the old version; use 'systemctl daemon-reload' to reload.
- [root@control ~]# findmnt --verify
- /tifdata
- [W] unreachable source: /dev/nvme0n13: No such file or directory
- [W] cannot detect on-disk filesystem type
- 0 parse errors, 0 errors, 2 warnings

- [root@control ~]# vi /etc/fstab
- [root@control ~]# mount -a
- mount: (hint) your fstab has been modified, but systemd still uses the old version; use 'systemctl daemon-reload' to reload.
- [root@control ~]# systemctl daemon-reload
- [root@control ~]# mount -a
- [root@control ~]# findmnt --verify
- Success, no errors or warnings detected
- [root@control ~]# lsblk
- NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
- sr0 11:0 1 109.9M 0 rom /run/media/jai/CDROM
- sr1 11:1 1 8.9G 0 rom /run/media/jai/RHEL-9-2-0-BaseOS-x86_64
- nvme0n1 259:0 0 110G 0 disk
- └─nvme0n1p1 259:1 0 1G 0 part /boot
- └─nvme0n1p2 259:2 0 99G 0 part
- │ └─rhel-root 253:0 0 65.2G 0 lvm /
- │ └─rhel-swap 253:1 0 2G 0 lvm [SWAP]
- │ └─rhel-home 253:2 0 31.8G 0 lvm /home
- └─nvme0n1p3 259:3 0 1G 0 part /tifdata
- └─nvme0n1p4 259:4 0 1K 0 part
- └─nvme0n1p5 259:5 0 1.5G 0 part
- └─nvme0n1p6 259:6 0 100M 0 part
- [root@control ~]# df -h

Creating and Mounting File Systems

Creating a File System in Linux:

- XFS File System: Default in RHEL 9, known for being fast and scalable. It uses Copy-On-Write (CoW) for data integrity.
- Example: `mkfs.xfs /dev/sda3` to create an XFS file system on partition sda3.
- Insight: XFS's size can be increased but not decreased.

Alternative File Systems:

- Ext4 File System: It was default in RHEL 6 and is compatible with older ext2. It uses a journal for data integrity and allows resizing both ways.
 - Example: `mkfs.ext4 /dev/sda4` to format a partition with ext4.
- VFAT File System: Commonly used for shared devices like USB drives due to its compatibility with multiple operating systems.
 - Example: `mkfs.vfat /dev/sdb1` for a USB drive.

Btrfs File System: A modern, CoW file system not installed by default on RHEL.

- Insight: Btrfs is advanced but less common.

Creating File Systems with mkfs Utility:

- To create a filesystem, use `mkfs.[file system type]`.
- Example: `mkfs.xfs`, `mkfs.ext4`, `mkfs.vfat`.
- Avoiding mkfs Default: Without specifying a type, mkfs creates an ext2 filesystem, which is outdated.
- Insight: Use `mkfs -t [type] /dev/[device]` to specify the file system type.

Mounting and Unmounting File Systems:

- Mounting connects a filesystem to a directory, making it accessible.
- Example: `mount /dev/sda3 /mnt` to mount sda3 at /mnt.
- Use `umount` to unmount, ensuring no files are open on the device (check with `lsdf`).
- Example: `umount /mnt` to unmount the filesystem from /mnt.

Mounting Partitions through /etc/fstab

Introduction to fstab and Its Purpose

- Explanation: /etc/fstab is the main configuration file for persistently mounting partitions.
- Example: /dev/sdb1 /data ext4 defaults 0 0
- Insight: /dev/sdb1 is the device to mount, /data is the mount point, ext4 is the file system type, and defaults are the mount options. The last two columns (0 0) are old and rarely needed.

Systemd and systemd-fstab-generator

- Explanation: systemd generates mount unit files based on /etc/fstab content using systemd-fstab-generator.
- Example: systemctl daemon-reload
- Insight: systemctl daemon-reload updates systemd configurations and can resolve various issues. It's an essential command to remember.

Handling Errors in /etc/fstab

- Explanation: Errors in /etc/fstab can lead to boot issues.
- Example: A typo in /etc/fstab causing an error.
- Insight: Errors like typos can be detected using commands like mount -a or findmnt --verify before rebooting.

Recovery from Boot Errors

- Explanation: In case of boot errors due to `/etc/fstab` issues, it's crucial to troubleshoot.
- Example: Entering a troubleshooting shell after encountering boot errors.
- Insight: Promptly fixing `/etc/fstab` errors during the root shell session is essential to avoid exam failures.

Troubleshooting Process

- Explanation: Check if the filesystem is read/writeable, identify devices, and correct `/etc/fstab` errors.
- Example: Checking the output of `mount` and using `vim` to correct `/etc/fstab`.
- Insight: Understanding the steps to diagnose and fix boot issues is essential for successful troubleshooting.

Exiting Troubleshooting Shell

- Explanation: Exiting the root shell and continuing the boot procedure.
- Example: Using `Control + D` or the `Exit` command.
- Insight: Exiting the root shell allows the system to continue booting normally.

Importance of Troubleshooting Skills

- Explanation: Emphasizing the significance of knowing how to troubleshoot boot issues.
- Example: Failing an exam due to unresolved boot issues.
- Insight: Demonstrating effective troubleshooting skills can be critical in real-world scenarios and exams.

Creating a Swap Partition

What is Swap?

- Basic Concept: Swap in Linux is a type of virtual memory. When the physical RAM (Random Access Memory) is full, the system uses swap space as an overflow area. This swap space is located on the hard drive or SSD.
- Purpose: It's used to temporarily store data that is not actively being used by the RAM, freeing up RAM to load and run new applications.
- Performance Aspect: While swap helps in managing memory more efficiently, it is slower than RAM because it relies on the storage device's speed.

Example:

Let's say you're using a computer with 4GB of RAM, running multiple applications like a web browser, a word processor, and a media player.

- As you open more tabs in your browser, your RAM gets filled up.
- The system then moves some of the less active data, such as background applications or inactive browser tabs, to the swap space.
- This process allows your active applications to continue running smoothly, using the freed-up RAM.

Conclusion:

- Swap is crucial for systems with limited RAM, ensuring that the system doesn't crash or become unresponsive when the RAM is fully utilized.
- However, relying too much on swap can slow down the system, as accessing data from swap is slower than accessing data from RAM.

Introduction

- Topic: Creating a swap partition in Linux.
- Importance: Essential for systems with limited RAM. Swap acts as an overflow area for RAM, stored on the disk.

Understanding Swap

- Definition: Swap is disk space used as virtual memory when RAM is full.
- Usage Consideration: Some applications may not work with swap; always check documentation.

Swap Creation Process

Selecting Partition:

- Use `fdisk` on `/dev/sda`.
- Command: `n` for new partition, choose logical partition (e.g., `sda7`).
- Example: Creating a 2GB partition.

Setting Partition Type:

- Use `t` for type change in `fdisk`.
- Choose `Linux swap` / `Solaris` type or use alias `swap`.
- Example: Setting `sda7` as swap type.

Organizing Partitions (Optional):

- Expert mode in `fdisk` (`x`).
- Use `f` to fix partition order.
- Example: Ensuring `sda5` starts before `sda6`.

```
[root@localhost ~]# fdisk /dev/nvme0n1
```

Welcome to fdisk (util-linux 2.37.4).

Changes will remain in memory only, until you decide to write them.

Be careful before using the write command.

GPT PMBR size mismatch (209715199 != 251658239) will be corrected by write.

This disk is currently in use - repartitioning is probably a bad idea.

It's recommended to umount all file systems, and swapoff all swap partitions on this disk.

```
Command (m for help): n
```

Partition number (4-128, default 4):

First sector (209713152-251658206, default 209713152):

Last sector, +/-sectors or +/-size{K,M,G,T,P} (209713152-251658206, default 251658206): +200M

Created a new partition 4 of type 'Linux filesystem' and of size 200 MiB.

```
Command (m for help): p
```

Disk /dev/nvme0n1: 120 GiB, 128849018880 bytes, 251658240 sectors

Disk model: VMware Virtual NVMe Disk

Units: sectors of 1 * 512 = 512 bytes

Sector size (logical/physical): 512 bytes / 512 bytes

I/O size (minimum/optimal): 512 bytes / 512 bytes

Disklabel type: gpt

Disk identifier: 042FBD63-921E-46E6-9780-B1923C2B1683

Device	Start	End	Sectors	Size	Type
/dev/nvme0n1p1	2048	1230847	1228800	600M	EFI System
/dev/nvme0n1p2	1230848	3327999	2097152	1G	Linux filesystem
/dev/nvme0n1p3	3328000	209713151	206385152	98.4G	Linux LVM
/dev/nvme0n1p4	209713152	210122751	409600	200M	Linux filesystem

Partition type or alias (type L to list all): 19

Changed type of partition 'Linux filesystem' to 'Linux swap'.

Command (m for help): p

Disk /dev/nvme0n1: 120 GiB, 128849018880 bytes, 251658240 sectors

Disk model: VMware Virtual NVMe Disk

Units: sectors of 1 * 512 = 512 bytes

Sector size (logical/physical): 512 bytes / 512 bytes

I/O size (minimum/optimal): 512 bytes / 512 bytes

Disklabel type: gpt

Disk identifier: 042FBD63-921E-46E6-9780-B1923C2B1683

Device	Start	End	Sectors	Size	Type
/dev/nvme0n1p1	2048	1230847	1228800	600M	EFI System
/dev/nvme0n1p2	1230848	3327999	2097152	1G	Linux filesystem
/dev/nvme0n1p3	3328000	209713151	206385152	98.4G	Linux LVM
/dev/nvme0n1p4	209713152	210122751	409600	200M	Linux swap

Command (m for help): w

The partition table has been altered.

Syncing disks.

```
nvme0n1p1 200 MiB 0 200 MiB 0 pure
[root@localhost ~]# free -m

```

	total	used	free	shared	buff/cache	available
Mem:	3615	1002	2297	17	554	2613
Swap:	3999	0	3999			

```

[root@localhost ~]#
[root@localhost ~]# mkswap /dev/nvme0n1p4
Setting up swapspace version 1, size = 200 MiB (209711104 bytes)
no label, UUID=9fbff948-ab42-4f20-9724-2744abf6c287
[root@localhost ~]# free -m

```

	total	used	free	shared	buff/cache	available
Mem:	3615	1003	2296	17	554	2612
Swap:	3999	0	3999			

```

[root@localhost ~]# swapon /dev/nvme0n1p4
[root@localhost ~]# free -m

```

	total	used	free	shared	buff/cache	available
Mem:	3615	1003	2296	17	554	2612
Swap:	4199	0	4199			

```

[root@localhost ~]# vi /etc/fstab
[root@localhost ~]# vi /etc/fstab
[root@localhost ~]# swapoff /dev/nvme0n1p4
[root@localhost ~]# free -m

              total        used        free      shared  buff/cache   available
Mem:           3615         1003         2292          17        558        2612
Swap:          3999           0         3999
[root@localhost ~]# swapon -a
[root@localhost ~]# free -m

              total        used        free      shared  buff/cache   available
Mem:           3615         1003         2292          17        558        2612
Swap:          4199           0         4199
[root@localhost ~]#

```

```

/dev/nvme0n1p4 none swap defaults 0 0

```

Using UUID and Labels

Introduction to UUIDs and Labels for Mounting:

- Point: In data center environments, block device names can change, leading to the need for persistent naming.
- Example: If a server's storage configuration changes, the device name like `/dev/sda1` might no longer point to the correct drive.
- Insight: Using UUIDs or Labels prevents issues caused by these changes, ensuring that the correct device is always referenced.

Understanding UUIDs:

- Point: A UUID (Universal Unique Identifier) is automatically generated for each device with a file system.
- Example: When formatting a new disk with `mkfs`, a UUID is created which uniquely identifies this disk.
- Insight: UUIDs are preferable in environments where disks are frequently added or removed, as they remain consistent regardless of the physical order of the drives.
-

```
[root@localhost ~]# blkid
```

```
/dev/mapper/rhel-swap: UUID="bf3a9f2f-7402-4eb2-9f8b-4c94030a03ac" TYPE="swap"  
/dev/nvme0n1p3: UUID="91h68R-0miJ-PXNa-vyUF-sv8r-gjFW-nll4fQ" TYPE="LVM2_member"  
PARTUUID="6f6cf1db-55c2-4704-813a-754d75f4796a"  
/dev/nvme0n1p1: UUID="3CC7-510D" TYPE="vfat" PARTLABEL="EFI System Partition" PARTUUID="8cbbbe3a-70a9-42c6-b6b4-09e94fc93210"  
/dev/nvme0n1p4: UUID="9fbff948-ab42-4f20-9724-2744abf6c287" TYPE="swap" PARTUUID="671d18a9-ba38-fc4f-83cd-308e5e7814e3"  
/dev/nvme0n1p2: UUID="f1ebc34d-577a-4b1f-b09a-21e08d2ecb07" TYPE="xfs" PARTUUID="a2a3b81d-1b84-461b-a977-0a1b0e88d8d6"  
/dev/sr0: UUID="2023-11-30-13-33-12-00" LABEL="CDROM" TYPE="iso9660"  
/dev/mapper/rhel-home: UUID="c427a028-9a34-481c-be64-27b5f23a36e6" TYPE="xfs"  
/dev/mapper/rhel-root: UUID="d5d18595-2901-4b8d-9c40-b99bf0eb2a4a" TYPE="xfs"  
/dev/sr1: UUID="2023-10-25-09-17-25-00" LABEL="RHEL-9-3-0-BaseOS-x86_64" TYPE="iso9660" PTUUID="ece57f7f" PTTYPE="dos"
```

```
UUID="231f7017-1648-440b-84d8-ccd51a769a83" /ext4 ext4 defaults 0 0
```

Labels

```
[root@localhost ~]# fdisk /dev/nvme0n1

Welcome to fdisk (util-linux 2.37.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

This disk is currently in use - repartitioning is probably a bad idea.
It's recommended to umount all file systems, and swapoff all swap
partitions on this disk.

Command (m for help): n
Partition number (5-128, default 5):
First sector (210122752-251658206, default 210122752):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (210122752-251658206, default 251658206): +500M

Created a new partition 5 of type 'Linux filesystem' and of size 500 MiB.

Command (m for help): w
The partition table has been altered.
Syncing disks.

[root@localhost ~]# lsblk
NAME                MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 112.5M  0 rom
sr1                  11:1    1   9.8G  0 rom
nvme0n1              259:0    0  120G  0 disk
```



```
[root@localhost ~]# mkfs.xfs /dev/nvme0n1p5
meta-data=/dev/nvme0n1p5      isize=512    agcount=4, agsize=32000 blks
                =               sectsz=512    attr=2, projid32bit=1
                =               crc=1        finobt=1, sparse=1, rmapbt=0
                =               reflink=1     bigtime=1 inobtcount=1 nnext64=0
data                =               bsize=4096   blocks=128000, imaxpct=25
                =               sunit=0       swidth=0 blks
naming             =version 2        bsize=4096   ascii-ci=0, ftype=1
log                =internal log     bsize=4096   blocks=16384, version=2
                =               sectsz=512    sunit=0 blks, lazy-count=1
realtime =none            extsz=4096   blocks=0, rtextents=0
[root@localhost ~]# xfs_admin -L jai /dev/nvme0n1p5
writing all SBs
new label = "jai" .
```

```
[root@localhost ~]# mkdir /xfsfs
[root@localhost ~]# vi /etc/fstab
```

```
LABEL=jai                /xfsfs                    xfs                      defaults                0 0
```

```
the old version, use systemctl daemon-reload.
[root@localhost ~]# systemctl daemon-reload.
[root@localhost ~]# mount -a .
[root@localhost ~]# df -h
```


Understanding Advanced Storage Solutions

LVM Logical Volumes

- *Topic:* LVM (Logical Volume Manager) is a versatile storage management solution in RHEL.
- *Key Features:* Allows resizing of storage, creating snapshots, and offers enhanced flexibility.
- *Example:* Create an LVM logical volume and demonstrate resizing it. Show how to create a snapshot for backup purposes.
- *Insight:* Emphasize LVM's importance in scenarios where storage needs change dynamically, like in growing databases or file servers.

Stratis: Next-Generation Volume Managing Filesystem

- *Topic:* Introduction to Stratis, an advanced volume management solution in RHEL.
- *Key Features:* Utilizes thin provisioning by default, operates in user space, and allows API access.
- *Example:* Set up a basic Stratis pool and demonstrate thin provisioning. Highlight the ease of API access for automation or cloud integration.
- *Insight:* Discuss Stratis's relevance in cloud computing and containerized applications due to its user-space implementation and API accessibility.

Exploring LVM Setup

Introduction to Volume Groups (VGs):

- Volume Group is a key concept in LVM.
- It acts as an abstraction layer over physical storage devices.
- Example: Imagine VG as a large container where you can put multiple hard drives (physical volumes) without worrying about managing each one separately.

Physical Volumes (PVs) and Their Role:

- Physical Volumes are actual storage devices like disks or partitions.
- They are the building blocks of a Volume Group.
- Example: If you have three 500GB hard drives, each can be a Physical Volume within a single Volume Group.

Creating Enormous Volume Groups:

- You can combine multiple PVs to create a very large VG.
- Example: Combining 20x 50TB drives to form a 1 Petabyte Volume Group.

Logical Volumes (LVs) - The Practical Units:

- Logical Volumes are created from the space in Volume Groups.
- They are the units you format with a file system and mount.
- Example: Creating a 100GB Logical Volume from a 1TB Volume Group to use as a file storage space.

Mounting Logical Volumes:

- LVs appear as device names like `/dev/vgx/lvx`.
- Example: Mounting a Logical Volume named `lv_home` in a Volume Group named `vg1` would be referenced as `/dev/vg1/lv_home`.

Flexibility and Scalability with LVM:

- Easily expand storage by adding more disks to the VG or extending LVs.
- Example: If `lv_home` is running out of space, you can add another disk to `vg1` and expand `lv_home` without disrupting the file system.

Creating an LVM Logical Volume

Introduction to LVM Logical Volumes

- LVM logical volumes offer flexibility in managing disk space.
- Example: Comparing LVM volumes with traditional partitioning methods highlights their dynamic resizing capability.

Creating a Partition for LVM

- Begin with creating a partition, setting its type to LVM.
- Example: Using `fdisk` or `gdisk` to create a partition on `/dev/sdb`, then setting its type to `8e00` for LVM.

Creating Physical and Volume Groups

- Utilize `pvccreate` to flag the partition as an LVM physical volume.
- Form a volume group using `vgcreate`, optionally setting extent size.
- Example: `pvccreate /dev/sdb1` followed by `vgcreate -s 8M vgdata /dev/sdb1`.

Creating the Logical Volume

- Employ `lvcreate` to establish the logical volume within the volume group.
- Example: `lvcreate -L 1G -n lvdata vgdata` to create a 1GB volume named `lvdata` in `vgdata` group.

Formatting and Mounting the Logical Volume

- Format the logical volume with a filesystem using `mkfs`.
- Persistently mount it in `/etc/fstab`.
- Example: Formatting with `mkfs.vfat /dev/vgdata/lvdata` and mounting it on `/lvdata`.

Understanding Extents in LVM

- Extents are basic units in LVM, defining volume size.
- Use `vgdisplay` to view properties, including extent size.
- Example: Using `vgdisplay` to check the extent size of `vgdata`.

Utilizing Gdisk for Partitioning

- `gdisk` can be used for GPT partitioning, similar to `fdisk`.
- Example: Demonstrating `gdisk` usage on `/dev/sdb` for creating an LVM partition.

Mounting the Logical Volume

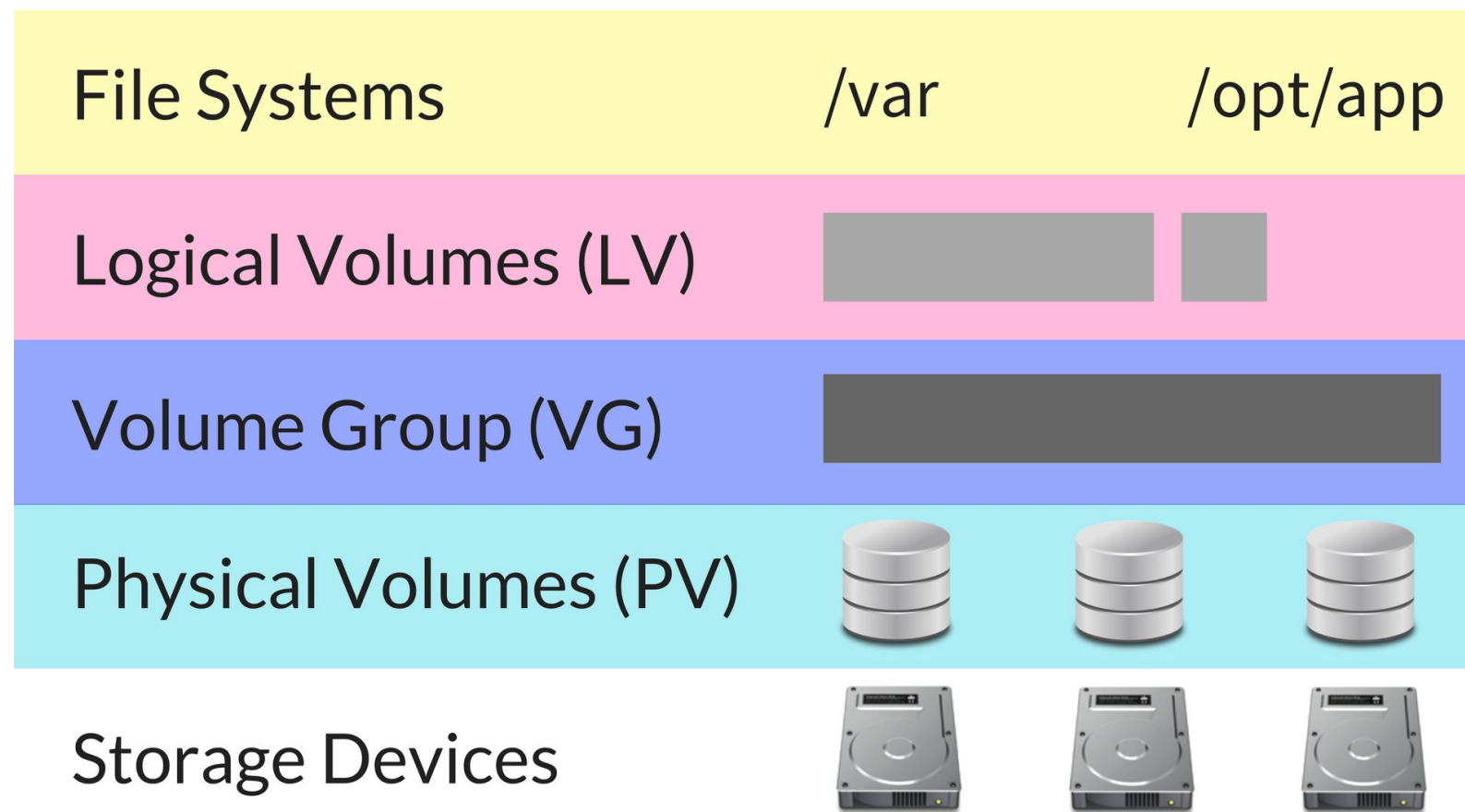
- Final step involves mounting the logical volume to use it.
- Example: Creating a directory `/lvdata` and adding an entry in `/etc/fstab` for `/dev/vgdata/lvdata`.

Verifying the Setup

- Use commands like `mount -a` and `lsblk` to verify the setup.
- Example: Mounting with `mount -a` and verifying the mount with `lsblk`.

Best Practices and Troubleshooting

- Advising on rebooting after changes for verification.
- Example: Rebooting the system to ensure changes don't affect booting.



Creating an LVM Logical Volume

```
[root@localhost ~]# gdisk nvme0n1
GPT fdisk (gdisk) version 1.0.7

Problem opening nvme0n1 for reading! Error is 2.
The specified file does not exist!
[root@localhost ~]# gdisk /dev/nvme0n1
GPT fdisk (gdisk) version 1.0.7

Partition table scan:
  MBR: protective
  BSD: not present
  APM: not present
  GPT: present

Found valid GPT with protective MBR; using GPT.

Command (? for help): ?
b      back up GPT data to a file
c      change a partition's name
d      delete a partition
i      show detailed information on a partition
l      list known partition types
n      add a new partition
o      create a new empty GUID partition table (GPT)
p      print the partition table
q      quit without saving changes
r      recovery and transformation options (experts only)
s      sort partitions
t      change a partition's type code
v      verify disk
w      write table to disk and exit
x      extra functionality (experts only)
?      print this menu
```

```
Command (? for help): n           
Partition number (6-128, default 6):
First sector (34-251658206, default = 211146752) or {+-}size{KMGTP}:
Last sector (211146752-251658206, default = 251658206) or {+-}size{KMGTP}: +500M           
Current type is 8300 (Linux filesystem)
Hex code or GUID (L to show codes, Enter = 8300): L           
Type search string, or <Enter> to show all codes: lvm           
8e00 Linux LVM           
Hex code or GUID (L to show codes, Enter = 8300): 8e00           
Changed type of partition to 'Linux LVM'
```

```
Command (? for help): p           
Disk /dev/nvme0n1: 251658240 sectors, 120.0 GiB
Model: VMware Virtual NVMe Disk
Sector size (logical/physical): 512/512 bytes
Disk identifier (GUID): 042FBD63-921E-46E6-9780-B1923C2B1683
Partition table holds up to 128 entries
Main partition table begins at sector 2 and ends at sector 33
First usable sector is 34, last usable sector is 251658206
Partitions will be aligned on 2048-sector boundaries
Total free space is 39489469 sectors (18.8 GiB)
```

Number	Start (sector)	End (sector)	Size	Code	Name
1	2048	1230847	600.0 MiB	EF00	EFI System Partition
2	1230848	3327999	1024.0 MiB	8300	
3	3328000	209713151	98.4 GiB	8E00	
4	209713152	210122751	200.0 MiB	8200	
5	210122752	211146751	500.0 MiB	8300	
6	211146752	212170751	500.0 MiB	8E00	Linux LVM <u> </u>

```
Command (? for help): w           
Final checks complete. About to write GPT data. THIS WILL OVERWRITE EXISTING
PARTITIONS!!

Do you want to proceed? (Y/N): y           
OK; writing new GUID partition table (GPT) to /dev/nvme0n1.
Warning: The kernel is still using the old partition table.
The new table will be used at the next reboot or after you
run partprobe(8) or kpartx(8)
The operation has completed successfully.
```

```
[root@localhost ~]# pvcreate /dev/nvme0n1p6  
Physical volume "/dev/nvme0n1p6" successfully created.
```

```
[root@localhost ~]# vgcreate vgdata /dev/nvme0n1p6  
Volume group "vgdata" successfully created
```

```
[root@localhost ~]#
```

```
[root@localhost ~]# vgs
```

VG	#PV	#LV	#SN	Attr	VSize	VFree
rhel	1	3	0	wz--n-	98.41g	0
vgdata	1	0	0	wz--n-	496.00m	496.00m

```
[root@localhost ~]# vgdisplay
```



```
[root@localhost ~]# lvcreate -L 200M -n lvdata /dev/nvme0n1p6
Volume group "nvme0n1p6" not found
Cannot process volume group nvme0n1p6
[root@localhost ~]# lvcreate -L 200M -n lvdata vgdata
Logical volume "lvdata" created.
[root@localhost ~]# lvs
LV      VG      Attr      LSize   Pool Origin Data%  Meta%  Move I
home    rhel    -wi-ao---- 31.00g
root    rhel    -wi-ao---- 63.50g
swap    rhel    -wi-ao---- <3.91g
lvdata  vgdata  -wi-a----- 200.00m
[root@localhost ~]# mkfs.vfat /dev/vgdata/lvdata
mkfs.fat 4.2 (2021-01-31)
[root@localhost ~]# mkdir /lvdata
[root@localhost ~]# vi /etc/fstab
[root@localhost ~]# mount -a
mount: (hint) your fstab has been modified, but systemd still uses
the old version; use 'systemctl daemon-reload' to reload.
[root@localhost ~]# systemctl daemon-reload
[root@localhost ~]# mount -a
[root@localhost ~]# df -h
```

/dev/vgdata/lvdata

/lvdata

vfat

defaults

0 0

Understanding Device Mapper and LVM Device Names

```
[root@localhost ~]# mount
```

```
/dev/mapper/vgdata-lvdata on /lvdata type vfat (rw,relatime,fmask=0022,dmask=0002,
ascii,shortname=mixed,errors=remount-ro)
[root@localhost ~]# grep lvdata /etc/fstab
/dev/vgdata/lvdata /lvdata vfat defaults 0 0
```

After creating the volume, Linux gives it a couple of different names:

- A non-persistent name like `/dev/dm-1`. This name can change every time you restart your computer, so it's not reliable for long-term use.
- A persistent name like `/dev/mapper/vgdata-lvdata`. This name is stable and doesn't change, making it reliable.

```
[root@localhost ~]# ls -l /dev/mapper/vgdata-lvdata /dev/vgdata/lvdata
lrwxrwxrwx. 1 root root 7 Nov 30 22:04 /dev/mapper/vgdata-lvdata -> ../dm-3
lrwxrwxrwx. 1 root root 7 Nov 30 22:04 /dev/vgdata/lvdata -> ../dm-3
```

Resizing LVM Logical Volumes

- Create 2 partitions with a size of 1 GiB each and set the **lvm** partition type
- **vgcreate vgfiles /dev/sde1**
- **lvcreate -l 255 -n lvfiles /dev/vgfiles**
- **mkfs.ext4 /dev/vgfiles/lvfiles**
- **df -h**
- **vgs** shows no available extents in the volume group
- **vgextend vgfiles /dev/sde2** adds a new PV to the VG
- **lvextend -r -l +50%FREE /dev/vgfiles/lvfiles** is now possible
- **df -h** shows the newly available space in the volume

Verifying Volume Group Space

- Command: `vgs`
- Purpose: Checks if the volume group has unused disk space.
- Example: Imagine a volume group named 'vgdata'. Run `vgs` to check available space. If space is insufficient, proceed to extend the volume group.
- Insight: Always ensure there's enough space before resizing logical volumes.

2. Extending Volume Group

- Command: `vgextend`
- Usage: Add physical volumes (PVs) to the volume group (VG) if necessary.
- Example: To add a new disk, `/dev/sdb`, to 'vgdata', use `vgextend vgdata /dev/sdb`.
- Insight: Extending VG is crucial when the existing space is not enough for your requirements.

3. Extending Logical Volume

- Commands: `lvextend` or `lvresize`
- Options: `-R -L +size`, e.g., `-R -L +1G`
- Purpose: Increases the size of the logical volume and the filesystem it hosts.
- Example: To increase the size of a logical volume 'lvdocs' in 'vgdata' by 1GB, use `lvextend -R -L +1G /dev/vgdata/lvdocs`.
- Insight: The `-R` option automatically resizes the filesystem, simplifying the process.

4. Manual Filesystem Resizing

- Scenario: If you forget `-R` during `lvextend`.
- Commands: `resize2fs` for Ext filesystems, `xfs_growfs` for XFS.
- Example: For an Ext4 filesystem on 'lvdocs', use `resize2fs /dev/vgdata/lvdocs`.
- Insight: Manual resizing is a fallback method, but it's more error-prone.

5. Reducing Logical Volume Size (Ext4 only)

- Note: Shrinking is only possible for Ext4, not XFS filesystems.
- Commands: Reduce volume size using `lvreduce`.
- Caution: Risky operation; ensure data backup.

Create Two partitions

```
Command (m for help): n   
Partition number (7-128, default 7):  
First sector (212170752-251658206, default 212170752):  
Last sector, +/-sectors or +/-size{K,M,G,T,P} (212170752-251658206, default 251658206): +1G
```

Created a new partition 7 of type 'Linux filesystem' and of size 1 GiB.

```
Command (m for help): t   
Partition number (1-7, default 7):  
Partition type or alias (type L to list all): lvm 
```

Changed type of partition 'Linux filesystem' to 'Linux LVM'.

```
Command (m for help): n   
Partition number (8-128, default 8):  
First sector (214267904-251658206, default 214267904):  
Last sector, +/-sectors or +/-size{K,M,G,T,P} (214267904-251658206, default 251658206): +1G
```

Created a new partition 8 of type 'Linux filesystem' and of size 1 GiB.

```
Command (m for help): t   
Partition number (1-8, default 8):  
Partition type or alias (type L to list all): lvm 
```

Changed type of partition 'Linux filesystem' to 'Linux LVM'.

```
Command (m for help):
```

```
Command (m for help): w 
```

```
The partition table has been altered.  
Syncing disks.
```

Create VG and LV and format, mount

```
[root@localhost ~]# vgcreate vgfiles /dev/nvme0n1p7 ✓  
Physical volume "/dev/nvme0n1p7" successfully created.  
Volume group "vgfiles" successfully created  
[root@localhost ~]# lvcreate -n lvfiles /dev/vgfiles ✗  
No command with matching syntax recognised. Run 'lvcreate --help'  
[root@localhost ~]# lvcreate -l 255 -n lvfiles /dev/vgfiles ✓  
Logical volume "lvfiles" created.  
[root@localhost ~]# mkfs.ext4 /dev/vgfiles/lvfiles ✓  
mke2fs 1.46.5 (30-Dec-2021)  
Creating filesystem with 261120 4k blocks and 65280 inodes  
Filesystem UUID: 759753c5-c996-43db-b8ad-656f8dbac702  
Superblock backups stored on blocks:  
    32768, 98304, 163840, 229376  
  
Allocating group tables: done  
Writing inode tables: done  
Creating journal (4096 blocks): done  
Writing superblocks and filesystem accounting information: done  
  
[root@localhost ~]# mount /dev/vgfiles/lvfiles /mnt ✓  
[root@localhost ~]# df -h
```



```
[root@localhost ~]# vgs
VG          #PV #LV #SN Attr      VSize      VFree
rhel         1   3   0 wz--n-   98.41g       0
vgdata       1   1   0 wz--n-  496.00m  296.00m
vgfiles      1   1   0 wz--n- 1020.00m       0
[root@localhost ~]# vgextend vgfiles /dev/nvme0n1p8
Physical volume "/dev/nvme0n1p8" successfully created.
Volume group "vgfiles" successfully extended
[root@localhost ~]# vgs
VG          #PV #LV #SN Attr      VSize      VFree
rhel         1   3   0 wz--n-   98.41g       0
vgdata       1   1   0 wz--n-  496.00m  296.00m
vgfiles      2   1   0 wz--n-   1.99g 1020.00m
[root@localhost ~]# lvextend -r -l +50%FREE /dev/vgfiles/lvfiles
Size of logical volume vgfiles/lvfiles changed from 1020.00 MiB
).
File system ext4 found on vgfiles/lvfiles mounted at /mnt.
Extending file system ext4 to <1.50 GiB (1606418432 bytes) on vg
resize2fs /dev/vgfiles/lvfiles
resize2fs 1.46.5 (30-Dec-2021)
Filesystem at /dev/vgfiles/lvfiles is mounted on /mnt; on-line res
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/vgfiles/lvfiles is now 392192 (4k) blocks 1

resize2fs done
Extended file system ext4 on vgfiles/lvfiles.
Logical volume vgfiles/lvfiles successfully resized.
[root@localhost ~]# df -h |grep -i lvfiles
/dev/mapper/vgfiles-lvfiles 1.5G   24K  1.4G   1% /mnt
[root@localhost ~]#
```

Reducing Volume Groups

Removing VG for PV

- Use **fdisk** to create 2 partitions with a size of 2GB each and set the type to **lvm**
- **vgcreate vgdemo /dev/sdf1**
- **lvcreate -L 1G -n lvdemo /dev/vgdemo** # Do NOT use all space
- **vgextend vgdemo /dev/sdf2**
- **pvs** shows all extents as available on /dev/sdf2
- **lvextend -L +500M /dev/vgdemo/lvdemo /dev/sdf2** ensure that free extents are used on sdf2 by adding the device name
- **pvs** shows used extents on /dev/sdf2
- **mkfs.ext4 /dev/vgdemo/lvdemo**

Removing PV from VG

- **mount /dev/vgdemo/lvdemo /mnt**
- **df -h**
- **dd if=/dev/zero of=/mnt/bigfile bs=1M count=1100** ensures that data is on both PVs
- **pvmove -v /dev/sdf2 /dev/sdf1** moves all used extents from sdf2 to sdf1 (can take a while)
- **pvs** will show that /dev/sdf2 is now unused
- **vgreduce vgdemo /dev/sdf2** is now allowed

Working with Systemd

1. Understanding Systemd

- Description: Systemd is a fundamental component in most Linux distributions. It acts as the manager of everything, initializing various system components after the Linux kernel is booted.
- Example: Consider a Linux system starting up. After the kernel initializes, systemd is the first service to start. It then proceeds to manage other services and system components.

3. Functions of Systemd

- Description: The primary function of systemd is to manage services. Besides that, it handles mounts, timers, paths, and more.
- Example: If you install a new service on your Linux system, systemd will manage its initialization, stopping, and restarting. It also handles mounting drives and setting up scheduled tasks.

4. Event-Driven Nature of Systemd

- Description: Systemd is designed to be event-driven, meaning it can respond to specific system events, making it effective in managing dependencies.
- Example: If a service that depends on another service is started, systemd can ensure that the dependency is started first, thanks to its event-driven architecture.

5. Systemd Units

- Description: The items managed by systemd are termed 'units'. While there are various types of units, focusing on service units is essential for system management.
- Example: A web server like Apache running on your system is managed as a service unit by systemd. You can use systemd commands to start, stop, enable, or disable this service.

Insights:

- Systemd's comprehensive control over system components simplifies management tasks and ensures a cohesive operation of the Linux system.
- Understanding systemd is crucial for effective Linux system administration, especially in handling services and dependencies.
- The event-driven approach of systemd enhances its efficiency in dependency management, making it a robust tool for system administrators.

Managing Systemd Services

Introduction to systemd and systemctl

- systemd is a system and service manager for Linux operating systems.
- The main command to interact with systemd is `systemctl`.
- Example: To see the types of units managed by systemd, use the command `systemctl -t help`.
- Insight: systemd manages a variety of unit types, with services being the most critical. It can also manage file systems, devices, and network sockets.

Exploring systemd Units

- Use `systemctl list-unit-files` to display all unit files and their statuses.
- Example: This command shows 382 unit files, indicating the extensive role of systemd.
- Insight: This gives a broad overview of what systemd manages, but detailed exploration might be overwhelming due to the sheer number of units.

Viewing Active Units

- The command `systemctl list-units` shows units currently loaded.
- Example: It starts with devices, then mounts (replacing `etc/fstab` functionality), and services.
- Insight: This is useful to understand what systemd is actively managing at any given moment.

Service Management in systemd

- Services have states like loaded, active, and running.
- Example: Checking the status of the cups service with `systemctl status cups` shows it is loaded, active, and running.
- Insight: The different states of a service indicate its readiness and activity levels.

Installing and Managing Services

- Installation example: `sudo dnf install -y httpd` installs the HTTP server.
- Managing services: Use `sudo systemctl` followed by `start`, `status`, `stop`, or `enable` to manage services.
- Example: `sudo systemctl status httpd` shows the HTTP server's status including its loaded configuration, activity state, and log information.
- Insight: This process demonstrates how to install, start, and monitor a service using `systemd`.

Enabling and Disabling Services

- Services can be enabled or disabled for automatic start at boot.
- Example: Use `sudo systemctl enable httpd` to enable the HTTP server to start at boot.
- Insight: Enabling a service links it to a target (like multi-user target), defining the system's startup state.

Stopping Services and Viewing Logs

- Use `sudo systemctl stop httpd` to stop the HTTP server.
- Example: Even after stopping, `systemctl status httpd` shows relevant log information.
- Insight: This feature is useful for troubleshooting as it provides immediate access to service logs.

Summary and Next Steps

- `Systemd` is integral for managing various aspects of a Linux system.
- The `systemctl` command offers a comprehensive toolkit for service management.
- Future lessons will explore advanced `systemd` features and configurations.

Modifying Service Configuration

Default Service Configuration in SystemD:

- Point: Often, the default service configuration is sufficient, but understanding how to modify it is important.
- Example: Assume you're working with the SSH service on a CentOS system. The default configuration might work fine, but you might want to tweak certain parameters for enhanced security or performance.

Viewing Current Unit Configuration with `systemctl cat`:

- Point: Use `systemctl cat [service]` to view the current configuration of a service.
- Example: `systemctl cat sshd.service` shows the current unit configuration for the SSH daemon, including dependencies and service specifics.

Understanding Multiple Configuration Files:

- Point: SystemD can have multiple configuration files for a service.
- Insight: This layered approach allows for default settings and overrides, providing flexibility in configuration management.

Finding Configurable Parameters with `systemctl show`:

- Point: Use `systemctl show [service]` to discover configurable parameters of a service.
- Example: To find parameters for HTTPD, use `systemctl show httpd`.

Editing Service Configuration with `systemctl edit`:

- Point: Modify service configuration using `systemctl edit [service]`.
- Example: To change restart settings for HTTPD, use `sudo systemctl edit httpd`. You can add parameters like `Restart=on-failure` and `RestartSec=42`.

Reloading SystemD Configuration with `systemctl daemon-reload`:

- Point: After modifying a service, tell SystemD to reload its configuration using `systemctl daemon-reload`.
- Insight: This step ensures that SystemD is aware of the new configurations. It's particularly important in older versions of SystemD.

Restarting a Service with `systemctl restart`:

- Point: To apply changes immediately, restart the service using `systemctl restart [service]`.
- Example: After editing HTTPD service, use `sudo systemctl restart httpd` to apply changes.

Understanding the Impact of Configuration Changes:

- Point: Configuration changes can impact service behavior significantly.
- Example: By setting `Restart=on-failure`, you ensure that HTTPD automatically restarts after a failure, enhancing reliability.

Checking Service Status with `systemctl status`:

- Point: Use `systemctl status [service]` to check the status of a service.
- Example: After restarting HTTPD, use `systemctl status httpd` to confirm it's running correctly.

SystemD's Role in Service Protection:

- Point: SystemD offers features like automatic restart to protect services without needing complex setups like high availability clusters.
- Insight: This functionality simplifies system administration and enhances reliability.

Practical Application and Testing:

- Point: It's important to test changes to understand their impact.
- Example: After configuring HTTPD to restart on failure, simulate a failure using `sudo killall httpd` and observe if it restarts as expected.

Managing Software

Installing Software from Source Packages

1. Introduction to Software Installation Methods on Linux

- Linux offers multiple ways to install software.
- Initially, software was installed from source code.
- However, this practice has become less common.

2. Understanding Installing Software from Source

- Open-source software often provides its source code.
- Source code is usually delivered in a compressed tar ball (archive).
- This archive may contain a setup script or just source files with a README.

3. The Concept of Compiling in Software Installation

- Compiling is the process of making software run on a specific platform.
- It's essential for ensuring software works on various Linux platforms (e.g., servers, embedded devices).
- Tools like the C compiler (GCC) and make utility are used for compiling.

4. Challenges of Installing from Source

- No central registration of installed software, making updates and uninstallation difficult.
- Lack of dependency handling.
- This method is more cumbersome and less preferred today.

5. Evolution of Software Installation on Linux

- Installing from source has been largely replaced by package managers.
- Package managers simplify the process and handle dependencies, updates, and uninstalls.

Understanding Software Packages

Understanding Software Packages in Linux:

- Definition: A software package in Linux is essentially a tarball, which includes a script for placing files in appropriate locations and a database to track installed software.
- Example: Consider a package like `vim.tar.gz`, which contains the Vim text editor, its installation script, and database entries for system integration.

Importance of Packages:

- Ease of Management: Packages simplify the process of installing, updating, and uninstalling software.
- Example & Insight: Using a package manager to install `nginx`, you can easily update or remove it without manually tracking files.

Dependency Management:

- Role of Dependencies: Packages often rely on other software (dependencies), usually libraries, to function correctly.
- Dependency Challenges: Without proper dependency management, installing a package can lead to a 'dependency hell'.
- Example: Trying to install `GIMP` might require installing numerous libraries first, each with their own dependencies, complicating the installation process.

The Problem of Dependency Hell:

- Real-world Scenario: Attempting to install a package manually can lead to a cascade of dependency requirements.
- Example: Installing `package A` might require dependencies `B`, `C`, and `D`, and each of these dependencies might have their own set of dependencies.

Package Formats and Their Usage:

- RPM and DEB: RPM is the standard package format in Red Hat-based systems, while DEB is used in Debian-based systems like Ubuntu.
- Example: Installing a package in Red Hat might use `rpm -ivh package.rpm`, whereas in Ubuntu it would be `dpkg -i package.deb`.

Demonstration of a Dependency Problem:

- Practical Example: Attempting to install a package (e.g., `emacs auctex`) using RPM and encountering a failed dependency error.
- Insight: This illustrates the complexity and inconvenience of manual dependency resolution.

Role of Package Managers:

- Solution to Dependency Management: Package managers like YUM, DNF, and APT automate dependency handling and package installation.
- Example: Using `apt-get install emacs` on Ubuntu resolves and installs all necessary dependencies automatically.

Managing Libraries

Understanding Software Libraries and Dependencies:

- Point: Libraries provide common functionality to multiple software packages.
- Example: Consider `glibc` (GNU C Library), a standard C library in Linux systems. It provides essential functions for various applications.
- Insight: Using common libraries like `glibc` avoids the need to rewrite basic functions for every new software package, streamlining development and ensuring consistency.

```
[student@centos ~]$ which passwd
/usr/bin/passwd
[student@centos ~]$ ldd $(which passwd)
        linux-vdso.so.1 (0x00007fff927a8000)
        libuser.so.1 => /lib64/libuser.so.1 (0x00007f9186064000)
        libgobject-2.0.so.0 => /lib64/libgobject-2.0.so.0 (0x00007f9186009000)
        libglib-2.0.so.0 => /lib64/libglib-2.0.so.0 (0x00007f9185ecf000)
        libpopt.so.0 => /lib64/libpopt.so.0 (0x00007f9185ec0000)
        libpam.so.0 => /lib64/libpam.so.0 (0x00007f9185eae000)
        libpam_misc.so.0 => /lib64/libpam_misc.so.0 (0x00007f9185ea8000)
        libaudit.so.1 => /lib64/libaudit.so.1 (0x00007f9185e78000)
        libselinux.so.1 => /lib64/libselinux.so.1 (0x00007f9185e4c000)
        libc.so.6 => /lib64/libc.so.6 (0x00007f9185c42000)
        libgmodule-2.0.so.0 => /lib64/libgmodule-2.0.so.0 (0x00007f9185c3c000)
libcrypt.so.2 => /lib64/libcrypt.so.2 (0x00007f9185c02000)
        libffi.so.8 => /lib64/libffi.so.8 (0x00007f9185bf6000)
        libpcre.so.1 => /lib64/libpcre.so.1 (0x00007f9185b7c000)
        libeconf.so.0 => /lib64/libeconf.so.0 (0x00007f9185b71000)
        libm.so.6 => /lib64/libm.so.6 (0x00007f9185a96000)
        libcap-ng.so.0 => /lib64/libcap-ng.so.0 (0x00007f9185a8d000)
        libpcre2-8.so.0 => /lib64/libpcre2-8.so.0 (0x00007f91859f6000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f918609c000)
[student@centos ~]$
```

I

Understanding Software Managers

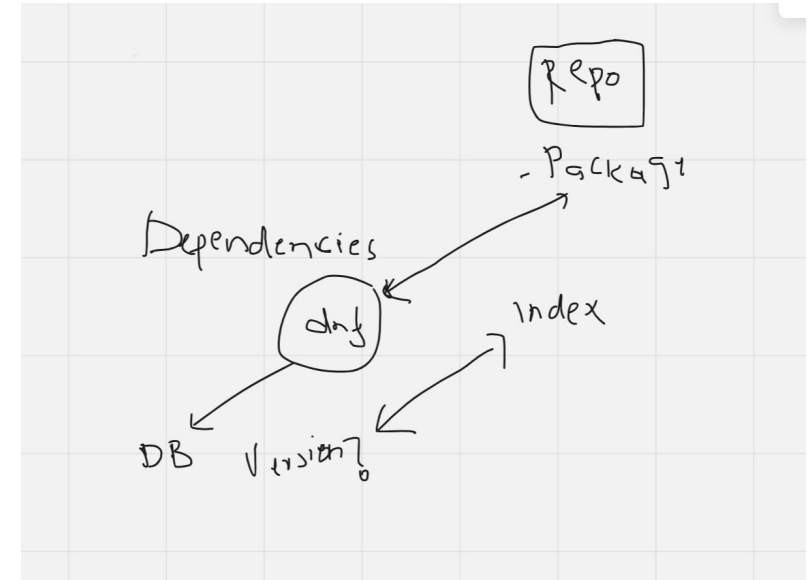
- Definition: Software managers are tools designed to resolve the dependency problem in software installation and management.
- Functionality: They work with repositories to manage dependencies, ensuring all required packages are available and handled efficiently.

Understanding Repositories

- Repositories: Online or onsite resources where software packages are stored.
- Role in Software Management: Software managers rely on repositories to fetch and manage packages and their dependencies.
- Types of Repositories:
 - Standard Repositories: Provided by Linux distributions, offering access to thousands of packages.
 - Vendor-Specific Repositories: Offered by software vendors.
 - Custom Repositories: Users can create their own repositories.

Software Managers in Linux Distributions

- Examples:
 - Red Hat and Fedora: Use YUM and DNF.
 - Ubuntu: Uses APT.
- Function: They analyze dependencies before installing a package and fetch these from the repositories.



Managing Packages with yum/dnf

Introduction to YUM and DNF:

- YUM and DNF are package managers specific to Red Hat and its derivatives.
- DNF replaces YUM in recent Red Hat Enterprise Linux (RHEL) versions.
- Both YUM and DNF offer similar functionalities due to a shared binary via symbolic links.
- Example: Using either `yum` or `dnf` commands yields similar results due to their underlying linkage.

Working with Repositories:

- Repositories in YUM/DNF are configured in text files located at `/etc/yum.repos.d`.
- Example: Editing a file in `/etc/yum.repos.d` to add or modify a repository.

Basic Commands in DNF:

- Installation: `dnf install [package_name]`
- Searching: `dnf search [package_name]`
- Removal: `dnf remove [package_name]`
- Example: `dnf install httpd` to install the Apache HTTP Server.

Package Groups in DNF:

- Package groups allow installation of related packages as a unit, focusing on functionality.
- List groups: `dnf groups list`
- Install a group: `dnf groups install [group_name]`
- Example: `dnf groups install "Development Tools"` to install development-related packages.

YUM Modules:

- YUM modules enable installing different versions of packages, including dependencies.
- Use: `yum module install [module_name]`
- Example: `yum module install container-tools` for container management tools.

Finding the Right Package:

- Sometimes `dnf search` may not locate the desired package.
- Alternative: `dnf provides */[file_name]`
- Example: `dnf provides */bash` to find packages related to the bash executable.

DNF History and Undoing Changes:

- `dnf history` lists recent package management actions.
- Example: `dnf history` to review recent installations, removals, or upgrades.

Using YUM for Package Search and Installation:

- **Searching:** `yum search [package_name]`
- **Installing:** `yum install [package_name]`
- **Use of `-y` to bypass confirmation prompt.**
- **Example:** `sudo yum install -y nmap` to install nmap without confirmation.

YUM Upgrade and History:

- **Upgrading packages:** `yum upgrade`
- **Reviewing history:** `yum history`
- **Example:** `yum upgrade` to update all installed packages.

Installing YUM Groups:

- **Listing groups:** `yum groups list`
- **Installing a group:** `yum groups install "[group_name]"` (use quotes for names with spaces)
- **Example:** `yum groups install "System Tools"` to install all packages in the System Tools group.

Understanding Repositories in YUM:

- **Repositories are defined in** `/etc/yum.repos.d/`.
- **Example:** Viewing the CentOS repository configuration in `/etc/yum.repos.d/centos.repo`.

Managing Packages with apt

Understanding `apt`:

- `apt` is Ubuntu's software manager, similar to `yum` and `dnf` in other distributions.
- It replaces older utilities like `apt-cache` and `apt-get`.
- Example: In older Ubuntu versions, commands were like `apt-get update`, but now it's simply `apt update`.

Repositories and Configuration:

- Apt repositories are listed in `/etc/apt/sources.list`.
- Additional repositories can be added in `/etc/apt/sources.d/`.
- Insight: This structure allows for easy management and addition of new software sources.

Updating Package Lists:

- Use `sudo apt update` to refresh the local package list with the latest versions from repositories.
- Example: Running `sudo apt update` ensures your system knows about the latest available packages.

Searching for Packages:

- Use `apt search` to find packages.
- Example: `sudo apt-cache search nmap` to search for packages related to `nmap`.
- Insight: `apt search` can yield a lot of results; refining search terms is often necessary.

Installing Packages:

- Install packages using `sudo apt install [package_name]`.
- Example: `sudo apt install nmap` installs `nmap` along with its dependencies.
- Insight: If a package includes a service, it's often started and enabled automatically.

Upgrading Packages:

- Use `sudo apt upgrade` to update installed packages.
- For a full distribution upgrade, use `sudo apt dist-upgrade`.
- Example: Regular use of `sudo apt upgrade` keeps your packages up-to-date.

Cleaning Up Unused Packages:

- `apt` informs about unnecessary packages (dependencies no longer required).
- Use `sudo apt autoremove` to remove these.
- Example: After removing a package, running `sudo apt autoremove` cleans up leftover dependencies.

Finding Packages Containing Specific Files:

- Use `apt-cache search [filename]` to find packages containing a specific file.
- Example: `sudo apt-cache search nmap` provides a filtered list of packages related to `nmap`.
- Insight: This is akin to `yum provides` and is useful for tracking down specific files within packages.

Regular Upgrades and Security Policy:

- Regular upgrades are crucial, especially for non-LTS (Long Term Support) versions of Ubuntu.
- Example: Upgrading daily builds like Ubuntu 22.04 ensures access to the latest software.

Using rpm

Introduction to RPM Command Usage

- Point: RPM, traditionally used for package installation, has limitations today due to its inability to handle dependencies.
- Example: In the past, `rpm` was the go-to command for installing packages like `rpm -i package.rpm`, but now, dependency management tools like `dnf` or `yum` are preferred.

RPM for Queries

- Point: RPM remains useful for querying the RPM database and individual packages.
- Example: Use `rpm -qf [file]` to find out which package a file belongs to. For instance, `rpm -qf /usr/bin/ls` identifies the package providing the `ls` command.

Querying Package Contents

- Point: Use RPM to list the contents of a package.
- Example: `rpm -ql [package_name]` lists all files in a package. For example, `rpm -ql net-tools` shows all files in the `net-tools` package.

Querying Specific Package Files

- Point: To query package files instead of contents, add a 'p' to the options.
- Example: `rpm -qpc [package.rpm]` lists configuration files in the package file. `rpm -qpl [package.rpm]` lists all contents of the package file.

Practical Demonstration: Net Tools

- Point: Use RPM to explore and manage specific packages like `Net Tools`.
- Example: `rpm -qf /sbin/ifconfig` finds that `ifconfig` comes from `Net Tools`. Then, `rpm -ql net-tools | grep /bin/` lists binaries in `Net Tools`. Finally, use `dnf remove net-tools` to remove the package after verification.

Exploring Package Files

- Point: RPM allows exploring specific RPM files for their contents.
- Example: First, identify an RPM file using `ls *.rpm`, then explore its contents with `rpm -qpl virtio-win.rpm`.

Downloading and Inspecting Packages

- Point: RPM queries can be combined with package downloaders for detailed inspection.
- Example: Use `yumdownloader vsftpd` to download `vsftpd`, then inspect its scripts with `rpm -qp --scripts vsftpd.rpm` to ensure there are no undesirable actions in the root-executed installation scripts.

Managing SSH

Running the SSH Server

Introduction to SSH Server Configuration:

- SSH provides remote access to a Linux terminal.
- Installation commands vary by Linux distribution: `sudo dnf install openssh-server` for Red Hat, `sudo apt install openssh-server` for Ubuntu.
- Ensure SSH service is started and enabled by systemd: `sudo systemctl status sshd` and `sudo systemctl enable --now sshd`.

SSH Server Security Concerns:

- SSH servers are often targeted by attackers, especially via brute-force attacks on the default SSH port (22).
- Attackers use scripts to try common usernames and passwords.
- Example Insight: Changing default configurations can significantly reduce the risk of such attacks.

Securing the SSH Server:

- Modify `/etc/ssh/sshd_config` to enhance security.
- Change the default SSH port: Port 2022 instead of Port 22.
- Disable root login: `PermitRootLogin no`.
- Restrict user access: `AllowUsers [specified_user]`.
- Disable password authentication: `PasswordAuthentication no`, encouraging key-based authentication.

Firewall Configuration for SSH:

- Firewalls need to be configured to allow SSH, especially if a non-default port is used.
- For Red Hat with `firewalld`: `firewall-cmd --add-port 2022/tcp --permanent` and `firewall-cmd --reload`.
- For Ubuntu with UFW (Uncomplicated Firewall): `sudo ufw allow OpenSSH` and `sudo ufw allow 2022/tcp`.

SELinux Configuration for SSH:

- SELinux on Red Hat does not allow SSH to run on non-default ports by default.
- To allow a non-default port: `semanage port -a -t ssh_port_t -p 2022 tcp`
- Editing `/etc/sysconfig/selinux` to disable SELinux is not recommended, as it compromises overall security.

```
[student@centos ~]$ sudo vim /etc/ssh/sshd_config
[student@centos ~]$ sudo semanage port -a -t ssh_port_t -p tcp 2022
[student@centos ~]$ sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens33
  sources:
  services: cockpit dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[student@centos ~]$ sudo firewall-cmd --add-port 2022/tcp --permanent
success
[student@centos ~]$ firewall-cmd --reload
success
[student@centos ~]$ firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens33
  sources:
  services: cockpit dhcpv6-client ssh
  ports: 2022/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[student@centos ~]$ █
```

Using the SSH Client

SSH and SCP Port Syntax:

- Example: For SSH, use `-p [port number]` (lowercase 'p') to specify a non-default port. For SCP, use `-P [port number]` (uppercase 'P'). This is important if SSH is listening on a non-default port.

Restarting SSH Service After Configuration Changes:

- Example: If you change the SSH port in the configuration (e.g., switching to port 2022), remember to restart the SSH service (`sudo systemctl restart sshd`) to apply the changes.

Verifying and Connecting to Different Hosts:

- Example: After resolving host key issues and ensuring the SSH service is running on the correct port, you can successfully connect to various machines, such as from an Ubuntu machine to a CentOS machine using `ssh -p 2022 user@192.168.29.101`.

Configuring Key-based SSH Login

```
student@student-virtual-machine:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/student/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/student/.ssh/id_rsa
Your public key has been saved in /home/student/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:8ikLmhJU/lghy+QnkqqWwhKZ3iuu9QF7xPv4LyI/04Q student@student-virtual-machine
The key's randomart image is:
+----[RSA 3072]-----+
|
|  + .
| B o .
| + B o
| ooo.O . S
| * E=.o o .
| +o=.=.. o
| =B.*o=.o
| Bo++B=o.
|
+----[SHA256]-----+
student@student-virtual-machine:~$ ssh-copy-id --help
Illegal option --
Usage: /usr/bin/ssh-copy-id [-h|-?|-f|-n|-s] [-i [identity_file]] [-p port] [-F alternative_ssh_config]
[user@]hostname
    -f: force mode -- copy keys without trying to check if they are already installed
    -n: dry run -- no keys are actually copied
    -s: use sftp -- use sftp instead of executing remote-commands. Can be useful if the remote or
    -h|-?: print this help
student@student-virtual-machine:~$ ssh-copy-id -p 2022 192.168.29.101
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/student/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install
student@192.168.29.101's password: █
```

Asks password every time

```
student@student-virtual-machine:~$ ssh -p 2022 192.168.29.101
Enter passphrase for key '/home/student/.ssh/id_rsa':
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Wed Mar  9 14:14:29 2022 from 192.168.29.142
[student@centos ~]$ exit
logout
Connection to 192.168.29.101 closed.
student@student-virtual-machine:~$ ssh -p 2022 192.168.29.101
Enter passphrase for key '/home/student/.ssh/id_rsa':
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Wed Mar  9 14:19:13 2022 from 192.168.29.142
[student@centos ~]$ exit
logout
Connection to 192.168.29.101 closed.
student@student-virtual-machine:~$ █
```


Balancing Security and Convenience:

- Using `ssh-agent` and `ssh-add` offers a balance between security (through the use of passphrases) and convenience (by reducing the need to repeatedly enter the passphrase).
- It's important to remember that if you start a new session, you'll need to repeat the `ssh-agent` and `ssh-add` steps.
- Example: After initial setup, each new session requires reinitializing `ssh-agent` and adding the key with `ssh-add` to avoid constant passphrase prompts.

```
student@student-virtual-machine:~$ ssh-agent /bin/bash
student@student-virtual-machine:~$ ssh-add
Enter passphrase for /home/student/.ssh/id_rsa:
Identity added: /home/student/.ssh/id_rsa (student@student-virtual-machine)
student@student-virtual-machine:~$ ssh -p 2022 192.168.29.101
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Wed Mar  9 14:19:19 2022 from 192.168.29.142
[student@centos ~]$ exit
```

Introduction to Key-based SSH Login:

- Default SSH access relies on usernames and passwords, which are vulnerable to being guessed or stolen.
- Key-based access, which uses a key pair, enhances security since keys are more complex and harder to guess or steal.
- Example: Instead of logging in with a username and password, you use a unique key pair generated specifically for SSH access.

Creating and Using SSH Key Pairs:

- A user must create a key pair using the command `ssh-keygen`.
- The public key of this pair is then copied to the SSH host.
- Example: Run `ssh-keygen` to generate your key pair. The system will prompt you to specify the file to save the key and to enter a passphrase for additional security.

The Role of Passphrases:

- Passphrases add an extra layer of security to SSH keys, functioning as multifactor authentication.
- However, entering a passphrase every time can be inconvenient.
- Example: When generating a key pair, you're asked to enter a passphrase. This passphrase is required every time the key is used unless managed by an SSH agent.

Utilizing SSH Agent for Convenience:

- `ssh-agent` is a program that holds private keys used for public key authentication.
- It eliminates the need to enter the passphrase every time you initiate an SSH session.
- Example: Use `ssh-agent /bin/bash` to start the agent and `ssh-add` to add your private key to the agent. Once added, the passphrase is required only once per session.

Copying the SSH Key to the Remote Host:

- Use `ssh-copy-id` to copy the public key to your SSH server.
- This command simplifies the process of setting up public key authentication on the server.
- Example: Run `ssh-copy-id user@hostname` to copy your public key to the server. You might need to specify the port using `-p` if not using the default SSH port.

Verifying and Using Key-based Authentication:

- To verify if the setup works, attempt to SSH into the server using the `-p` flag to specify the port if needed.
- Upon successful setup, SSH will ask for the passphrase or directly grant access if using `ssh-agent`.
- Example: Use `ssh -p 2022 user@192.168.29.101` to connect to the server. If set up correctly, it should either ask for your passphrase or use the one stored in `ssh-agent`.

Balancing Security and Convenience:

- Using `ssh-agent` and `ssh-add` offers a balance between security (through the use of passphrases) and convenience (by reducing the need to repeatedly enter the passphrase).
- It's important to remember that if you start a new session, you'll need to repeat the `ssh-agent` and `ssh-add` steps.
- Example: After initial setup, each new session requires reinitializing `ssh-agent` and adding the key with `ssh-add` to avoid constant passphrase prompts.

Managing Time

Understanding Linux Time

Managing Linux Time

Understanding the NTP Protocol

Configuring Time Synchronization

Understanding Linux Time

Hardware Time and Its Importance:

- Point: Linux time management starts with hardware time, which is kept in the hardware clock on the motherboard.
- Example: When a computer boots, it only has access to this hardware time.
- Insight: Ensuring accurate hardware time is crucial since it serves as the initial time source for the operating system.

Transition from Hardware Time to System Time:

- Point: Upon booting, the operating system obtains its system time from the hardware time.
- Example: When Linux starts, it reads the current time from the hardware clock.
- Insight: This process highlights the dependency of system time on the accuracy of hardware time.

Manipulating Hardware Time with `hwclock`:

- Point: The `hwclock` utility in Linux is used to manipulate the hardware time.
- Example: To set the hardware clock to a specific time, you could use a command like `"sudo hwclock --set --date='2023-11-26 12:00:00'"`.
- Insight: `hwclock` provides a direct way to adjust the foundational timekeeping device in a system.

Using `timedatectl` for System Time Management:

- Point: `timedatectl` is a preferred utility for managing time in the operating system.
- Example: To display the current system time and settings, you could use `"timedatectl status"`.
- Insight: This utility simplifies time management on the system level, integrating with system services.

Relevance of `hwclock` Despite `timedatectl`:

- Point: `hwclock` remains relevant for resolving discrepancies between hardware and system time.
- Example: If the system clock is wrong after a reboot, use `"sudo hwclock -s"` to synchronize it with the hardware clock.
- Insight: This is essential to maintain consistency in timekeeping, especially across reboots.

Synchronization with NTP (Network Time Protocol):

- Point: The system time often synchronizes with external sources using NTP.
- Example: To enable NTP synchronization, you could use `"timedatectl set-ntp true"`.
- Insight: NTP ensures accurate and consistent timekeeping by referencing external time sources, but it requires the system time to be reasonably accurate initially.

Importance of Time Accuracy for Linux Services:

- Point: Accurate time is crucial for various services on Linux.
- Example: A web server on Linux might require accurate timestamps for logging and SSL certificate validation.
- Insight: Time discrepancies can lead to failures in time-sensitive operations, highlighting the need for correct time settings.

Managing Linux Time

Using the `date` Command:

- Purpose: `date` allows you to check and display the current system time.
- Example: To display the date in day-month-year format, use `date +%d-%m-%Y`.
- Insight: The `date` command is convenient for timestamping in backup commands. For example, you can use it in a command like `sudo tar cvf TMP_backup-$(date +%d-%m-%Y).tar /etc` to create a backup file with the current date in its name.

Manipulating Hardware Time with `hwclock`:

- Purpose: `hwclock` is used to manipulate and synchronize hardware time.
- Example: To synchronize hardware clock with system time, use `sudo hwclock --systohc`.
- Insight: Synchronizing the hardware clock with system time is crucial because the system fetches time from the hardware clock at boot. If they are out of sync, it can cause issues.

Using `timedatectl` for Time Management:

- Purpose: `timedatectl` is a modern utility for managing time-related properties, including time zones and NTP (Network Time Protocol) settings.
- Example: To change the time zone to Africa/Banjul, use `sudo timedatectl set-timezone Africa/Banjul`.
- Insight: `timedatectl` provides a comprehensive way to manage system time, including easy adjustments of time zones and viewing current settings.

Scripting and Automation:

- Example: You can automate the backup process by incorporating the `date` command in a script and scheduling it with cron.
- Insight: This automation ensures regular backups with minimal manual intervention, enhancing system maintenance.

Time Zone Manipulation with `timedatectl`:

- Example: To switch the time zone to America/Los Angeles, use `sudo timedatectl set-timezone America/Los_Angeles`.
- Insight: Managing time zones with `timedatectl` is straightforward and reflects immediately across system time displays.

Understanding the NTP Protocol

Introduction to NTP in Linux:

- Point: NTP (Network Time Protocol) is crucial in Linux for managing system time.
- Example: Imagine a Linux server hosting a time-sensitive application, like a database or a web server. Accurate timekeeping is essential for synchronizing transactions and logs. Using NTP, the server's system time is kept in sync with global time standards.
- Insight: Accurate time management is vital in distributed systems to ensure consistency and reliability of operations.

NTP Services in Linux:

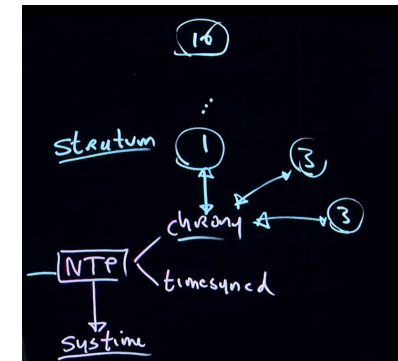
- Point: Linux uses services like Chrony and systemd-timesyncd for NTP.
- Example: On a new Linux installation, you might configure Chrony by editing the `/etc/chrony/chrony.conf` file to specify your preferred NTP servers.
- Insight: Different NTP services offer varying features and performance, but their primary role is consistent time synchronization.

Stratum Levels in NTP:

- Point: NTP uses the concept of stratum to determine the reliability of time servers, ranging from 1 (most reliable) to 16 (unusable).
- Example: A stratum 1 server might be directly connected to an atomic clock, while a stratum 3 server syncs its time from a stratum 2 server.
- Insight: The stratum level is a critical metric in selecting time sources for accuracy and reliability.

Time Synchronization and Selection of Servers:

- Point: Services like Chrony choose to sync with the most reliable time server available, often a stratum 1 server.
- Example: If Chrony is configured with multiple time servers - stratum 1, 3, and 4 - it prioritizes synchronization with the stratum 1 server.
- Insight: This prioritization ensures the highest accuracy for the system's clock.



Configuring Time Synchronization

Enabling/Disabling Time Synchronization:

- **Command:** `timedatectl set-ntp [true/false]`
- **Example:** To enable time synchronization, use `timedatectl set-ntp true`. This should be on by default in most systems.

Checking Time Synchronization Status:

- **Command:** `timedatectl timesync-status`
- **Example:** Use `sudo timedatectl timesync-status` to check the current status of time synchronization on your system.

Using Chrony for Detailed Information (specifically for Red Hat-based systems):

- **Commands:**
 - `chronyc sources`: Displays the time sources being used.
 - `chronyc tracking`: Provides detailed information about the current time sync status.
- **Example:** If `timedatectl timesync-status` gives an unexpected message (e.g., service not activatable), use `sudo chronyc sources` to view the active time sources. For more detail, `sudo chronyc tracking` can be used.

Understanding System Messages and Status:

- When encountering messages like "service not activatable," it indicates a possible conflict or preference for a different time synchronization service.
- **Example:** On a Red Hat system, if `timedatectl timesync-status` shows "service not activatable," it might be because the `chronyd` service is running and has taken over time synchronization duties from `systemd-timesyncd`.

Different Time Synchronization Services in Linux:

- **Two main services:** `chrony` and `systemd-timesyncd`.
- `systemd-timesyncd` is simpler but may not always be active.
- `chrony` is more commonly used in Red Hat systems and offers more detailed information.
- **Example:** On a system using `chrony`, `sudo systemctl status chrony` or `chronyd` will confirm if the `chronyd` service is active.

```
[student@centos ~]$ sudo timedatectl status
    Local time: Wed 2022-03-09 14:55:13 PST
    Universal time: Wed 2022-03-09 22:55:13 UTC
    RTC time: Wed 2022-03-09 22:55:13
    Time zone: America/Los_Angeles (PST, -0800)
System clock synchronized: yes
    NTP service: active
    RTC in local TZ: no
```

```
[student@centos ~]$ sudo chronyc sources
```

MS Name/IP address	Stratum	Poll	Reach	LastRx	Last sample
=====					
^- 185.216.231.116	3	6	327	5	-5274us[-5274us] +/- 87ms
^- 150.230.33.77	2	6	377	10	-9489us[-9489us] +/- 37ms
^- 23.92.64.226	3	6	377	9	-3743us[-3743us] +/- 117ms
^* clock.nyc.he.net	1	6	377	12	-8502us[-56ms] +/- 35ms

```
[student@centos ~]$ sudo systemctl status chrony
```

Process Management

Understanding Linux Processes and Jobs

Managing Interactive Shell Jobs

Monitoring Processes with top

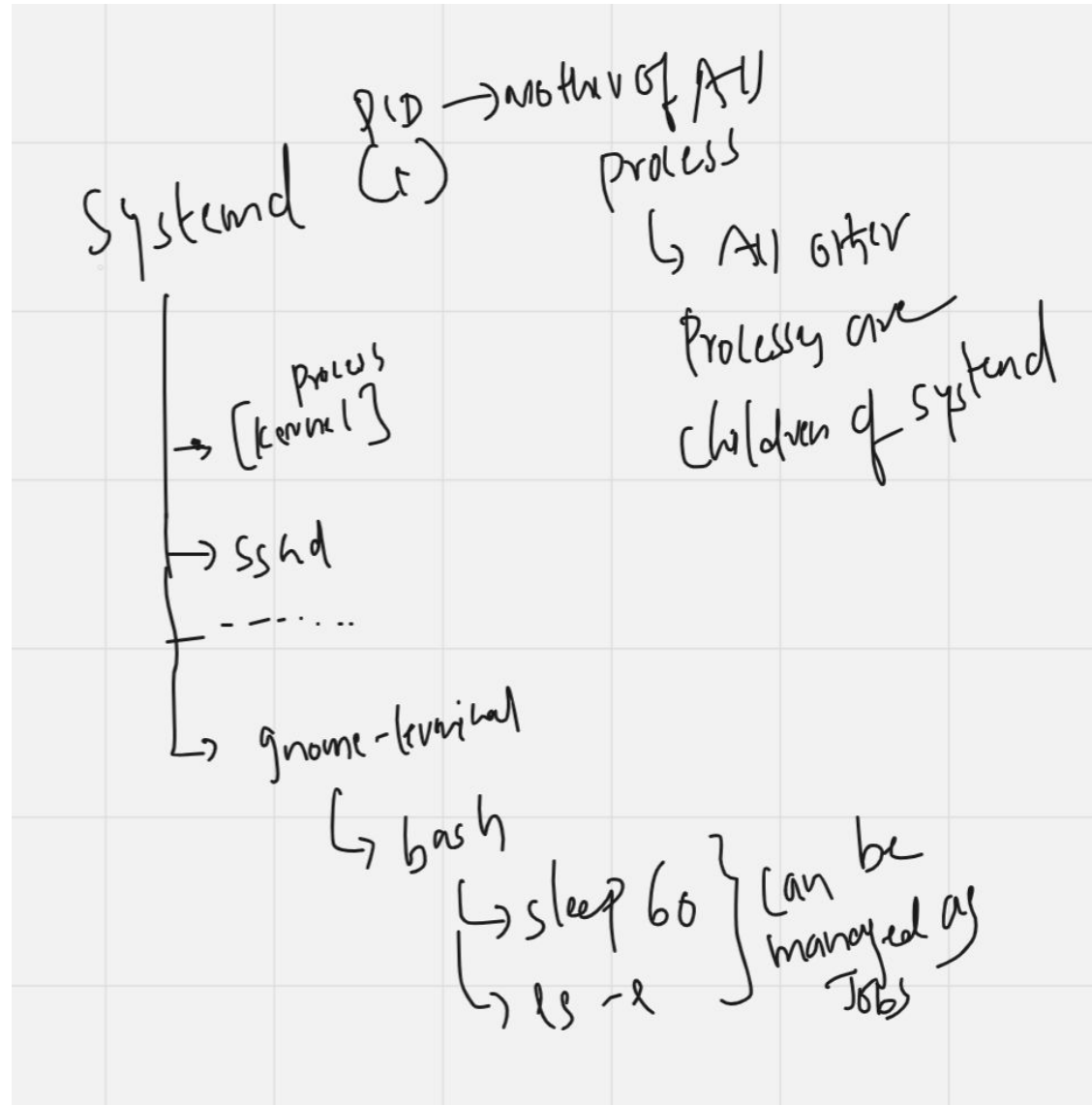
Changing top Display Properties

Monitoring Process Properties with ps

Changing Process Priority

Managing Processes with kill

Understanding Linux Processes and Jobs



Linux Processes and Jobs Overview

- Point: Everything that runs on Linux is a process with a unique process ID (PID).
- Example: When you run a command like `ls`, it creates a new process with a unique PID.

Signal Management in Processes

- Point: Processes can be managed by sending signals, which are special operating system instructions.
- Example: To stop a process, you can send a `Sigterm` (Signal 15) using `kill -15 [PID]`.

Forceful Process Termination

- Point: To immediately stop a process, `Sigkill` (Signal 9) is used.
- Example: If a process is unresponsive, use `kill -9 [PID]` to forcefully terminate it.

Understanding Jobs

- Point: Jobs are user processes started from a shell, which users can manage independently.
- Example: Running `sleep 60` in a terminal starts a job that you can manage (like pausing with `Ctrl+Z`).

Systemd and Process Hierarchy

- Point: All processes start as children of `systemd`, which is PID 1.
- Example: Using `ps tree` shows `systemd` at the top, with all other processes branching off it.

Managing Interactive Shell Jobs

Introduction to Foreground and Background Jobs

- Foreground Job: A command run in the foreground occupies the console, preventing other actions.
- Example: Running `ls -l` will display the list of files and occupy the console until it completes.

Moving a Running Process to the Background

- Use `Control + Z` to stop (pause) a running foreground job.
- Then, use `bg` command to continue it in the background.
- Example: If `ping google.com` is running in the foreground, pressing `Control + Z` and then typing `bg` will move it to the background.

Starting a Process Directly in the Background

- To start a new process in the background, append `&` to the command.
- Example: Using `sleep 60 &` will start the `sleep` command in the background directly.

Listing Background Jobs

- Use the `jobs` command to get an overview of all running background jobs.
- Example: After running multiple commands with `&`, use `jobs` to list them.

Bringing a Job to the Foreground

- Use `fg` command followed by a job number to bring a background job to the foreground.
- Example: If you have multiple jobs, `fg %2` will bring the second job in the `jobs` list to the foreground.

Stopping a Foreground Process

- Use `Control + C` to terminate a foreground process.
- Example: While a process like `top` is running in the foreground, pressing `Control + C` will stop it.

Practical Demonstration with `dd` Command

- Running `dd if=/dev/zero of=/dev/null` will copy from `/dev/zero` to `/dev/null` and occupy the console.
- Use `Control + Z` to pause it, `bg` to move it to the background, and `jobs` to list it.
- Use `fg` to bring it back to the foreground and `Control + C` to stop it.
- Insight: This showcases how you can manage a task that occupies your console, allowing multitasking in a single shell session.

So, the command `dd if=/dev/zero of=/dev/null` is effectively an operation that reads null characters from `/dev/zero` and writes them to `/dev/null`, discarding them immediately. It's a command that does nothing in terms of productive output, but it's useful for demonstrations because it continuously runs without producing any visible result, making it a good candidate for showing how to manage background and foreground processes in a shell.

Monitoring Processes with top

Starting top with Elevated Privileges: To access complete process information, start `top` with elevated privileges.

- Example: Use the command `sudo top` to start `top` with administrative access.

Understanding the top Interface: The `top` command sorts processes by CPU activity, showing the busiest process at the top. It also displays system information like uptime, logged-in users, and load averages.

- Example: When you run `top`, the process consuming the most CPU will be listed first.

Interpreting Load Average: Load average indicates the average number of processes requesting CPU attention over the last 1, 5, and 15 minutes.

- Example: A load average of 1.0 on a single CPU system means the CPU was fully utilized.

Starting Background Processes for Demonstration: Running background processes can help illustrate how `top` displays active processes and changing load averages.

- Example: Use the command `dd if=/dev/zero of=/dev/null &` to start a background process and observe its impact in `top`.

Load Average and CPU Count: The load average should be compared to the number of CPUs to assess system load. Pressing '1' in `top` displays each CPU's activity.

- Example: On a system with two CPUs, a load average above 2.0 might indicate high load and potential performance issues.

Tasks and Process States: `top` shows the number of running and sleeping tasks, as well as any stopped or zombie processes.

- Example: If `top` shows zombie processes, it indicates processes that have completed but still have an entry in the process table.

CPU Line Breakdown: The CPU line in `top` shows the percentage of time spent in user space (`us`), system space (`sy`), idle (`id`), and waiting for I/O (`wa`).

- Example: High `wa` values suggest bottlenecks in disk I/O, requiring investigation.

Memory Usage: `top` displays total memory, free memory, used memory, and memory used for buffer and cache.

- Example: A system with 4 GB of RAM might show 1.2 GB free, 1.3 GB used, and the rest in buffers and cache.

Understanding Swap Usage: Swap space is used for emulating RAM on the disk, useful when running out of physical RAM.

- Example: A system with 2 GB of swap space and all of it free indicates no current usage of swap.

Total vs. Available Memory: The difference between total, free, and available memory is crucial. Available memory includes inactive cache that can be freed up.

- Example: 2.3 GB of available memory might consist of free memory and inactive cache memory.

Process Information in top: Each process is listed with its PID, user, CPU and memory usage.

- Example: A process using 50% CPU in `top` indicates it's consuming half of the CPU resources.

For a complete example of using `top`, consider this scenario: You notice your server is responding slowly. You run `sudo top` to check the processes. You notice the load average is 3.5 on a 2 CPU system, suggesting high load. The `top` interface shows several processes consuming high CPU, mainly in user space (`us`). You also see that the memory usage is high, but there's minimal swap usage. This analysis helps you understand that the CPU is overloaded, and you might need to optimize the running processes or consider upgrading your hardware for better performance.

Changing top Display Properties

Introduction to `top` Command Default Interface:

- Insight: The `top` command provides a real-time view of system processes and resource usage.
- Example: When you first open `top`, you see a default set of columns like process ID, user, priority, etc.

Changing Display Properties using 'F':

- Insight: Customizing the display helps focus on specific metrics important for your needs.
- Example: Press 'F' in `top` to see a list of fields. Use the space bar to select/deselect fields like last used CPU or swap information.

Limitations of Screen Space:

- Insight: While `top` allows adding many columns, screen size limits visibility.
- Example: If you select too many fields, the screen gets cluttered and unreadable. Choose only essential fields to display.

Exiting Field Selection with 'Q':

- Insight: 'Q' finalizes your selection and updates the display.
- Example: After selecting desired fields, press 'Q' to see the updated `top` interface with new columns like CPU and swap usage.

Highlighting Running Processes with 'Z':

- Insight: This feature enhances visibility but may not appeal to everyone.
- Example: Press 'Z' in `top` to highlight running processes in different colors. This is a matter of personal preference.

Saving Configuration with 'w':

- Insight: Saving your configuration allows you to maintain a customized view across sessions.
- Example: After customizing `top`, press 'w' to save these settings to a 'toprc' file in your home directory. This ensures the same settings are loaded in future sessions.

User-Specific Configuration Files:

- Insight: 'toprc' configurations are user-specific and stored in each user's home directory.
- Example: If you run `top` with `sudo`, it uses the root user's 'toprc' configuration. Running it without `sudo` uses the ordinary user's configuration.

Monitoring Process Properties with ps

Introduction to `ps` Utility

- Utility Overview: `ps` is used to find properties about processes.
- Basic Usage: Use `ps` to show running processes for the current user.
- Example: `ps` command displays processes associated with the user's account.

Viewing All Processes with `ps aux`

- Command Explanation: `ps aux` shows all processes, including detailed information.
 - A: Shows all processes.
 - U: Displays the user that started the process.
 - X: Provides additional information.
- BSD Style Command: No hyphen before `aux`.
- Example: `ps aux` to view all system processes.

Parent-Child Relationship in Processes: `ps fax`

- Utility: Shows parent-child relations between processes.
- Importance: Useful for understanding dependencies, especially when terminating processes.
- Example: `ps fax` to view hierarchical process structure.

Filtering and Sorting with `ps`

- Advanced Usage: Combine `ps` with other tools like `awk` and `sort` for customized output.
- Monitoring Specific Processes: Use `ps aux | grep [service]` to check if a service is running.
- Example: `ps aux | grep HTTP` to check if a web service is running.

Using `ps` for Detailed Process Information

- Command Breakdown:
 - User: Username of the process owner.
 - PID: Process ID, crucial for process management.
 - CPU & Memory Usage: Including VSZ and RSS.
 - TTY: Terminal associated with the process.
 - Status: Indicates the state of the process, e.g., 'S' for sleeping.
 - Time: Duration for which the process has been running.
 - Command: The command that initiated the process.
- Example: `ps aux | less` to view detailed process information with headers.

Understanding `ps fax` Output

- Hierarchical View: Displays the tree-like structure of processes.
- Consequences of Terminating Processes: Impact on child processes.
- Example: `ps fax` to see parent and child processes.

Dialects of `ps`: BSD vs. System V

- BSD Style: `ps` command typically follows BSD syntax (without hyphens).
- System V Style: `ps -ef` to use System V options.
- Example: `ps -ef` as an alternative to `ps aux`.

Advanced Filtering with `ps`

- Options for Customized Display: `ps -e -o pid,args --forest` shows PIDs and process forest.
- Process Forest: Illustrates parent-child relations in a tree structure.
- Example: `ps -e -o pid,args --forest` for a detailed process tree.

Sorting by Memory Usage: `ps aux --sort pmem`

- Purpose: To identify processes using the most memory.
- Example: `ps aux --sort pmem` to sort processes based on memory usage.

Summary and Recommendation

- `ps` Versatility: Numerous options available, but not all are commonly used.
- Using `top` for Simplicity: For advanced process properties, `top` is more user-friendly.
- Example: Use `top` for a dynamic overview of process resource usage.

Changing Process Priority

Understanding Process Priority:

- Every user process in Linux starts with the same priority.
- Example: When you start any application, it gets the default priority level.

Using the `nice` Command:

- The `nice` command is used to start processes with an adjusted priority.
- Non-privileged users can only decrease priority, while privileged users can both increase and decrease it.
- Example: Running `nice -n 10 <command>` will start the process with a lower priority. For a non-privileged user, trying `nice -n -10 <command>` will not work due to permission restrictions.

Priority Range:

- Priority values range from -20 (highest priority) to 19 (lowest priority).
- Example: `nice -n -5 <command>` sets the priority higher than the default.

Using `renice` and `top` Commands:

- `renice` adjusts priority for running processes.
- `top` can be used to monitor process priorities and their CPU usage.
- Example: If a process with ID 1234 is running, using `renice 15 1234` will lower its priority.

Demonstration of Priority Adjustment:

- Using `nice` with a negative value requires root privileges.
- Example: As a regular user, `nice -n -10 dd if=/dev/zero of=/dev/null` fails, but works with root access.

Background Process Management:

- Control the execution of processes in the foreground or background.
- Example: Use `Control+Z` and `bg` to move a running process to the background.

Observing Process Behavior:

- Running multiple instances of processes with different priorities to observe their CPU usage.
- Example: Running two `dd` processes with different `nice` values and comparing their CPU usage in `top`.

Impact of Process Priority:

- Processes with higher priority get more CPU time compared to lower priority ones.
- Example: A process with a `nice` value of -10 will have a higher CPU usage compared to a process with a `nice` value of 0.

Adjusting Priorities with `top`:

- Inside `top`, you can change the priority of processes you own.
- Example: Pressing `r` in `top`, then entering the PID and new nice value, such as `renice 15 <PID>`.

Managing Processes with kill

Identify Busy Processes: Use `top` to see which processes are consuming the most resources. Suppose you find a process named 'example' using a lot of CPU.

Manage Using `kill`: If you know the PID of 'example', say 1234, you can gracefully request it to stop using `kill 1234` or `kill -15 1234`.

Force Stop if Necessary: If 'example' doesn't respond, use `kill -9 1234` to forcibly terminate it.

Use `killall` for Multiple Instances: If there are several 'example' processes, use `killall example` to terminate all of them at once.

Require Sudo for Others' Processes: If 'example' isn't your process, you'd need to use `sudo kill 1234` or `sudo killall example`.

Verify and Understand Signals: To understand more about the signals, you can read through `man 7 signal`.

Applying Resource Limits

1. POSIX Resource Limits

- Explanation: POSIX resource limits are the oldest method of limiting resources.
- Usage: Set using `ulimit` command, which is shell-specific.
- Types of Limits:
 - Soft limits: User-modifiable, up to the hard limit.
 - Hard limits: Absolute ceiling, set by administrators.
- Example: To see current limits, run `ulimit -a`. To set a hard limit of maximum file size to 500 KB, use `ulimit -H -f 500`.

NOTE: "POSIX" stands for "Portable Operating System Interface." It is a set of standards specified by the IEEE (Institute of Electrical and Electronics Engineers) for maintaining compatibility between operating systems.

2. PAM (Pluggable Authentication Modules) Resource Limits

- Explanation: PAM limits are more persistent compared to POSIX limits.
- Configuration: Set in `/etc/security/limits.conf` and `/etc/security/limits.d/*.conf`.
- Usage: To restrict the number of logins for a group, like students.
- Example: To set a hard limit of 2 max logins for students, create a file `/etc/security/limits.d/students.conf` with the line `@students hard maxlogins 2`.

3. Systemd Services Limits

- Explanation: Limits specific to systemd services.
- Usage: Add limit entries in the service unit file.
- Example: To limit a service (`whatever.service`) to 60 seconds of CPU time, create `/etc/systemd/system/whatever.service.d/10-cpulimits.conf` with `LimitCPU=60`. Apply changes with `systemctl daemon-reload` and `systemctl restart whatever.service`.

```
[root@centos ~]# systemctl show sshd.servi
```

PAM (Pluggable Authentication Modules) Resource Limits

```
[root@centos ~]# ulimit -a
real-time non-blocking time (microseconds, -R) unlimited
core file size              (blocks, -c) 0
data seg size               (kbytes, -d) unlimited
scheduling priority         (-e) 0
file size                   (blocks, -f) unlimited
pending signals             (-i) 14260
max locked memory           (kbytes, -l) 64
max memory size             (kbytes, -m) unlimited
open files                  (-n) 1024
pipe size                   (512 bytes, -p) 8
POSIX message queues        (bytes, -q) 819200
real-time priority          (-r) 0
stack size                  (kbytes, -s) 8192
cpu time                    (seconds, -t) unlimited
max user processes          (-u) 14260
virtual memory              (kbytes, -v) unlimited
file locks                  (-x) unlimited

[root@centos ~]# man bash
[root@centos ~]# man limits.conf
[root@centos ~]# vim /etc/security/limits.d/students.conf
student hard maxlogins 1

[root@centos ~]# w
 11:04:43 up  2:59,  1 user,  load average: 0.20, 0.06, 0.05
USER      TTY      LOGIN@  IDLE   JCPU   PCPU WHAT
student   tty2      08:10   2:59m  0.02s  0.02s /usr/libexec/gnome-session-binary

[root@centos ~]# ssh student@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is SHA256:RQYOWJBHKXvHfVQy7i7Xb0xyTzjyvX1VI6GC75tm7wg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
student@localhost's password:
Web console: https://localhost:9090/ or https://192.168.29.168:9090/

There were too many logins for 'student'.
Last login: Tue Feb 21 08:10:42 2023
Connection to localhost closed.
```

Scheduling Tasks

Understanding Linux Task Scheduling

Scheduling Tasks with cron

Using systemd Timers

Using at to Schedule Tasks

Understanding Linux Task Scheduling

Cron:

- Overview: Cron is a traditional solution for scheduling recurring tasks.
- How it works: Uses the `crond` daemon. Tasks are edited using `crontab -e`.
- Use Case: Ideal for tasks that need to occur regularly (hourly, daily, etc.).
- Example: To schedule a daily backup at 2 AM, edit the crontab file with `crontab -e` and add the line `0 2 * * *`
`/path/to/backup/script.sh`. This ensures your backup script runs daily at 2 AM.

At Command:

- Overview: Used for scheduling tasks once at a specific time.
- How it works: Utilizes the `atd` daemon. Tasks are scheduled using the `at` command.
- Use Case: Suitable for one-time tasks at a specific moment.
- Example: To run a script at 4 PM today, you would use `echo "/path/to/script.sh" | at 16:00`. This schedules the script to run today at 4 PM.

Systemd Timer:

- Overview: A newer alternative to cron, part of the systemd suite.
- How it works: Involves creating a `.timer` unit and running it with `systemctl`.
- Use Case: Becoming increasingly important, especially in systems where systemd plays a central role.
- Example: To create a systemd timer for a weekly cleanup task, you would create a `cleanup.timer` and a `cleanup.service` file. The `.timer` file defines when the task should run, and the `.service` file defines what the task is.

Scheduling Tasks with cron

Verifying Cron Daemon is Active:

- Command: `sudo systemctl status crond`
- Insight: Ensuring the cron daemon is running is crucial for scheduling cron jobs. Without it, no jobs will be scheduled.
- Example: Before scheduling a job, always check if the cron service is active. If it's inactive, you may need to start it using `sudo systemctl start crond`.

Understanding Cron Timing Syntax:

- Concept: The five positions in crontab represent minute, hour, day of the month, month, and day of the week, respectively.
- Insight: The timing syntax is flexible and allows scheduling from every minute to specific days.
- Example: `"30 8 * * 5"` means every Friday at 8:30 AM.

Understanding Crontab:

- Concept: Crontab is the traditional file used in UNIX for scheduling cron jobs.
- Insight: Modern systems don't rely solely on a single crontab file, but it's still a useful tool for understanding cron syntax.
- Example: Viewing the crontab file can be done with `"crontab -l"`. It shows the job schedule format: minute, hour, day of the month, month, and day of the week.

Scheduling a Cron Job:

- Command: `"crontab -e"`
- Insight: This opens the default editor to schedule a cron job for the current user.
- Example: To schedule a job every hour at the 13th minute, add a line like `"13 * * * * /usr/bin/logger 'Hello'"` in the crontab editor.

Using the Logger Command:

- Command: `"logger"`
- Insight: Logger is used to write messages to the system log, useful for simple debug messages.
- Example: `"logger 'Task Completed'"` logs "Task Completed" in the system log.

Exploring Alternative Cron Directories:

- Command: `"sudo ls /etc/cron.*"`
- Insight: These directories (`cron.daily`, `cron.hourly`, etc.) were used in older Linux distributions for running scripts at regular intervals.
- Example: Placing a script in `/etc/cron.daily/` will schedule it to run once daily.

Checking Job Execution with JournalCTL:

- Command: `"sudo journalctl -xe"`
- Insight: JournalCTL, part of systemd, can be used to verify if a cron job executed successfully.
- Example: After a cron job is supposed to run, use `"sudo journalctl -xe | grep logger"` to check if the logger command executed and logged the message.

Using systemd Timers

Introduction to Systemd Timers

- Point: Systemd timers are used to schedule tasks in Linux.
- Example: Use `sudo systemctl list units -t timer` to view active systemd timers.
- Insight: This command is useful for getting a quick overview of scheduled tasks on your system.

Viewing More Timers with Different Commands

- Point: You can view more timers by modifying the command.
- Example: Use `sudo systemctl list unit files -t timer` to see additional systemd timers, including disabled ones.
- Insight: This expanded view helps understand all potential timers, not just the active ones.

Understanding the Relationship Between Timers and Services

- Point: Each `.timer` file is associated with a `.service` file.
- Example: For `fstrim.timer`, there's a corresponding `fstrim.service`.
- Insight: The timer acts as a trigger for the service, scheduling when the service should run.

Examining a Timer's Configuration

- Point: Use specific commands to view the configuration of a timer.
- Example: `sudo systemctl cat fstrim.timer` shows details like `OnCalendar`, `AccuracySec`, etc.
- Insight: The `OnCalendar=weekly` setting is critical as it defines how often the service is triggered.

Create timer with systemd

Backup Script: `backup-script.sh`

```
#!/bin/bash
```

```
# Define source and destination
```

```
SOURCE_DIR="/home/user/Documents"
```

```
BACKUP_DIR="/home/user/Backups"
```

```
DATE=$(date +"%Y%m%d")
```

```
BACKUP_FILE="$BACKUP_DIR/backup_$(date +%Y%m%d).tar.gz"
```

```
# Create backup directory if it doesn't exist
```

```
mkdir -p "$BACKUP_DIR"
```

```
# Create a compressed backup
```

```
tar -czf "$BACKUP_FILE" "$SOURCE_DIR"
```

Steps to use:

Save this script as `/usr/local/bin/backup-script.sh`.

Make it executable: `chmod +x /usr/local/bin/backup-script.sh`.

Create a service file for it:

File Path: `/etc/systemd/system/backup.service`

```
[Unit]
```

```
Description=Backup service
```

```
[Service]
```

```
ExecStart=/usr/local/bin/backup-script.sh
```

Create a Timer File

This file specifies when the service should be triggered.

Example

To run the backup script every day at midnight:

File Path: `/etc/systemd/system/backup.timer`

```
[Unit]
```

```
Description=Runs backup daily at midnight
```

```
[Timer]
```

```
OnCalendar=daily
```

```
Persistent=true
```

```
[Install]
```

```
WantedBy=timers.target
```


Enable and Start the Timer

Once the files are created, you need to enable and start the timer.

Commands

- Reload systemd to recognize new units: `sudo systemctl daemon-reload`
- Enable the timer to start at boot: `sudo systemctl enable backup.timer`
- Start the timer immediately: `sudo systemctl start backup.timer`

4. Check the Status

You can check the status of your timer to ensure it's active and scheduled correctly.

Command

- Check status: `systemctl status backup.timer`

```
[student@centos ~]$ sudo systemctl list-unit-files -t timer
```

UNIT FILE	STATE	VENDOR PRESET
dnf-makecache.timer	enabled	enabled
fstrim.timer	disabled	disabled
fwupd-refresh.timer	disabled	disabled
logrotate.timer	disabled	disabled
mdadm-last-resort@.timer	static	-
mdcheck_continue.timer	disabled	disabled
mdcheck_start.timer	disabled	disabled
mdmonitor-oneshot.timer	disabled	disabled
mlocate-updatedb.timer	enabled	enabled
podman-auto-update.timer	disabled	disabled
raid-check.timer	disabled	disabled
systemd-tmpfiles-clean.timer	static	-
unbound-anchor.timer	enabled	enabled

13 unit files listed.

```
[student@centos ~]$ systemctl list-unit-files fstrim*
```

UNIT FILE	STATE	VENDOR PRESET
fstrim.service	static	-
fstrim.timer	disabled	disabled

2 unit files listed.

```
[student@centos ~]$ sudo systemctl cat fstrim.timer
```

[Timer]

OnCalendar=weekly

AccuracySec=1h

Persistent=true

RandomizedDelaySec=6000

Using at to Schedule Tasks

Understanding the at Daemon:

- The `at` command relies on the `atd` daemon. You can check if it's active using `sudo systemctl status atd`.
- Example: To verify if the `atd` service is running, use the command `sudo systemctl status atd`.
- Scheduling Commands with `at`:
 - a. To schedule a command, type `at` followed by the time you want the command to run. For instance, `at teatime` schedules a task for 4:00 PM.
 - b. Example: To log a message at teatime, you would type `at teatime`, then `logger TEATIME`, and end input with Control-D (EOF signal).
- Using Control-D to End Input:
 - a. Control-D is used to signify the end of input in various command-line interfaces. It's essential for completing the `at` command input.
 - b. Example: After typing your scheduled command, press Control-D to indicate you're finished.
- Checking Scheduled Tasks with `atq`:
 - a. `atq` lists all scheduled tasks. It helps in verifying if your commands are correctly scheduled.
 - b. Example: After scheduling tasks, use `atq` to see the list of scheduled jobs.
- Removing Scheduled Tasks with `atrm`:
 - a. If you need to cancel a scheduled task, use `atrm` followed by the job number.
 - b. Example: To cancel a scheduled job, use `atrm [job number]`.
- Confirming Execution of Tasks:
 - a. To verify that a task was executed at the scheduled time, you can check for the expected results.
 - b. Example: If you scheduled `touch /tmp/1022` for 10:22, after 10:22, use `ls -l /tmp` to see if the file 1022 was created.

```
[student@centos ~]$ atq
1      Thu Mar 10 16:00:00 2022 a student
[student@centos ~]$ at 10:22
warning: commands will be executed using /bin/sh
at> touch /tmp/1022
at> <EOT>
job 4 at Thu Mar 10 10:22:00 2022
[student@centos ~]$ atq
1      Thu Mar 10 16:00:00 2022 a student
4      Thu Mar 10 10:22:00 2022 a student
[student@centos ~]$
```

Reading Log Files

Understanding Linux Logging

Working with journalctl

Understanding Rsyslogd

Understanding Linux Logging

1. Introduction to Syslog

- Description: Syslog is the foundational logging service in Linux, serving as a legacy system for logging management.
- Implementation: In modern Linux, syslog functionality is primarily handled by `rsyslogd`.
- Example: On a Linux system, you can view syslog files typically located in `/var/log/syslog`. By running the command `cat /var/log/syslog`, you can view the system logs that `rsyslogd` has collected and stored.

2. Role and Evolution of `rsyslogd`

- Description: `rsyslogd` is the daemon responsible for implementing syslog on contemporary Linux distributions.
- Presence: It's commonly found across most Linux distributions.
- Example: To check if `rsyslogd` is running on your system, you can execute `sudo service rsyslog status`. This command will show the status of the `rsyslog` service.

3. Introduction to systemd-journald

- Description: `systemd-journald` is an integrated logging service that is part of the `systemd` suite, which is a newer and more comprehensive solution for system management.
- Integration: Due to `systemd`'s extensive control over various system components, it can efficiently capture logs from these components.
- Example: To view logs managed by `systemd-journald`, you can use the command `journalctl`. For instance, `journalctl -b` displays logs from the current boot.

4. Comparison and Coexistence

- Distributions: While some Linux distributions have shifted to exclusively using `systemd-journald`, others still use `rsyslogd` for its advanced features, often alongside `systemd-journald`.
- Example: In a distribution using both, you might configure `rsyslogd` for certain specific logging tasks (like remote logging) using its configuration file at `/etc/rsyslog.conf`, while relying on `systemd-journald` for general system logs.

5. Practical Insights

- Choosing the Right Tool: The choice between `rsyslogd` and `systemd-journald` often depends on specific requirements like advanced features or system integration.
- Example: For a server that needs detailed log management and forwarding capabilities, `rsyslogd` might be the preferred choice. Conversely, for a system focused on centralized management and simplicity, `systemd-journald` would be more suitable.

Working with journalctl

Introduction to `journalctl`

Overview of `journalctl`:

- `journalctl` is the primary utility for interacting with the systemd journal.
- Typing `journalctl` displays the complete journal.

Using Options with `journalctl`:

- Unit-specific Logs: `journalctl -u <unit-name>` shows logs for a specific unit.
 - *Example:* `journalctl -u sshd` displays logs for the SSH daemon.
- Kernel Messages: `journalctl --dmesg` shows kernel messages.
 - *Example:* `journalctl --dmesg` to view kernel-related logs.
- Advanced Filtering: `journalctl -u crond --since yesterday -p info` shows info-level logs for `crond` since yesterday.
 - *Example:* To filter `crond` logs since the previous day at info level.

Journal Start Time:

- The journal starts from the last system reboot, which may not always be convenient for long-term log analysis.

Making the Systemd Journal Persistent

Default Setting: Most Linux distributions have `Storage=auto` in `/etc/systemd/journald.conf`.

- This setting means if `/var/log/journal` exists, the journal will be saved persistently.

Exploring the Journal:

- Use `sudo journalctl` to view the journal.
- Use uppercase `G` in the journal view to jump to the most recent messages.
- *Example:* Viewing the entire journal and navigating to recent messages.

Tab Completion with `journalctl`:

- `journalctl` supports tab completion but can show an overwhelming number of options (e.g., 112 options).

Specific Use Cases and Tips

Comparing with `systemctl status`:

- `journalctl -u <unit>` shows all logs for a unit, unlike `systemctl status`, which only shows the last few lines.
 - *Example:* `journalctl -u sshd` vs. `sudo systemctl status sshd`.

Real-time Log Monitoring:

- `journalctl -f` for real-time updates. Useful for troubleshooting.
 - *Example:* Monitor real-time failures or changes, like failed login attempts.

Exiting Real-time View:

- Use `Control + C` to exit the real-time (`-f`) mode of `journalctl`.

Enabling Persistent Storage:

- Edit `/etc/systemd/journald.conf` and ensure `Storage=auto` is set.
- Create `/var/log/journal` to enable persistent storage of logs.
 - *Example:* `mkdir /var/log/journal` to ensure logs are saved across reboots.

Understanding Rsyslogd

Understanding Rsyslog:

- Rsyslog is a logging service in Linux that deals with log facilities, priorities, and destinations.
- Example: The rsyslog configuration might specify logging for the 'cron' facility with a priority of 'none', meaning it excludes cron logs from being recorded.
- Insight: Rsyslog's flexibility allows for customized logging, crucial for system management and troubleshooting.

Configuration File (rsyslog.conf):

- The main configuration file for rsyslog is `/etc/rsyslog.conf`.
- Example: A line in the configuration file might read `mail.info /var/log/maillog`, indicating that informational messages from the mail facility should be logged to `/var/log/maillog`.
- Insight: Understanding this file is key to customizing log management, though the syntax and options can initially seem complex.

Priorities in Rsyslog:

- Priority indicates the severity of a log message, ranging from debug (lowest) to emergency (highest).
- Example: `"kern.crit /var/log/kern.log"` means log only critical messages from the kernel to `/var/log/kern.log`.
- Insight: Priorities help in filtering logs, ensuring that only relevant log messages are stored or alerted upon.

Log Destinations:

- Destinations specify where log messages should be written, like a file or a remote server.
- Example: `"local7.* /var/log/boot.log"` logs all messages from the local7 facility to `/var/log/boot.log`.
- Insight: By directing logs to different destinations, administrators can organize logs more efficiently.

Viewing Log Files:

- Logs are typically stored in `/var/log`, and commands like `ls -lrt` and `less` can be used to view them.
- Example: Using `less /var/log/messages` to view the most recent system log entries.
- Insight: Regularly checking log files is a best practice for system health monitoring and troubleshooting.

Additional Rsyslog Functionalities:

- Rsyslog offers advanced functionalities like centralized logging and writing logs to a database.
- Example: Configuring rsyslog to send logs to a central log server for consolidated log management.
- Insight: These advanced features make rsyslog suitable for complex, multi-server environments.

Using the Logger Command:

- The `logger` command is a convenient way to add entries to the system log.
- Example: Executing `logger "System backup completed"` to log a custom message.
- Insight: This command is useful for scripting and automating log entries.

Managing Logrotate

Purpose of Logrotate:

- Point: Logrotate prevents log files from becoming too large.
- Example: Imagine a log file growing endlessly, consuming storage. Logrotate automates the process of archiving and removing old log data to prevent this.

Scheduling Logrotate:

- Point: It can be scheduled via cron jobs or systemd timers.
- Example: To see if logrotate is scheduled as a systemd timer, use the command `systemctl list-units -t timer`. This replaces the older method of using cron jobs.

Configuration Files:

- Point: Main configurations are in `/etc/logrotate.conf` and specific overrides in `/etc/logrotate.d/*`.
- Example: `/etc/logrotate.conf` might have a setting like `weekly, rotate 4, create, dateext`, indicating weekly rotations, keeping four backups, creating new files post-rotation, and appending the rotation date to filenames.

The Kernel

Understanding the Kernel

Managing Kernel Modules

Tuning the Kernel

Understanding the Kernel

Definition of the Kernel

- Explanation: The kernel is the core component of an operating system, essentially its 'heart'.
- Example: Think of the kernel as the conductor of an orchestra, where the orchestra is your computer's hardware. It ensures each section (CPU, memory, devices) works in harmony.

Kernel as an Open Source Project and its Language

- Explanation: The kernel, particularly in Linux, is an open-source project and is primarily written in the C programming language.
- Example: Similar to how Wikipedia allows contributions from anyone, the open-source nature of the kernel means that developers from around the world can contribute to its codebase.

Kernel's Role in Hardware Control

- Explanation: The kernel controls all major functions of the computer's hardware.
- Example: When you move your mouse, it's the kernel that interprets this action and communicates it to the cursor on your screen.

Kernel's Position in System Booting

- Explanation: During the booting process, the kernel is the first part of the operating system that loads.
- Example: Like turning the key in a car's ignition starts the engine first before other components, booting a computer starts with the kernel.

Drivers and Kernel Modules

- Explanation: In Linux, hardware is managed through drivers, implemented as kernel modules.
- Example: A printer requires a specific driver (kernel module) to communicate with your system, just like you need a specific key to start a particular car.

Application Communication with System

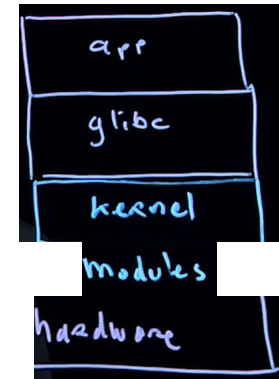
- Explanation: Applications communicate with the system through glibc, which then interacts with the kernel.

Kernel's Mission and Communication with Hardware

- Explanation: The primary mission of the kernel is to facilitate communication between software and hardware.

Kernel Modules as Drivers

- Explanation: Kernel modules are essentially drivers that enable the kernel to interact with different hardware components.



Managing Kernel Modules

Introduction to Kernel Modules

- Definition: Kernel modules are object files that extend the Linux kernel. They are compiled from C programming language files.
- Purpose: Each hardware component requires a kernel module to function.
- Example: Consider a graphics card. It needs a specific kernel module to interact with the Linux kernel, allowing the operating system to utilize the hardware effectively.

Obtaining Kernel Modules

- Source: Kernel modules are typically obtained from `initramfs`, which is created during system installation to match the detected hardware.
- Dynamic Loading: `systemd-udev` acts as a plug-and-play manager, dynamically loading kernel modules when needed.
- Manual Loading: Kernel modules can also be loaded manually.
- Example: When a new USB device is plugged in, `systemd-udev` automatically loads the required kernel module for it. Alternatively, you can manually load a module for a network card using `modprobe`.

Managing Kernel Modules

- Commands Overview:
 - `lsmod`: Lists all currently loaded kernel modules.
 - `modprobe`: Manually loads a kernel module along with its dependencies.
 - `modprobe -r`: Manually unloads a kernel module.
 - `modinfo`: Provides information about kernel modules.
 - `depmod`: Generates a dependency file for kernel modules.
- Example: To see if the `cdrom` module is loaded, use `lsmod`. To load it, use `modprobe cdrom`. If it's already loaded and you want to unload it, use `modprobe -r cdrom`.

Handling Module Dependencies

- Checking Dependencies: Use `modinfo` to list parameters and dependencies of a module.
- Setting Parameters: Modify kernel module parameters in `/etc/modprobe.d/`.
- Example: Suppose `cdrom` is in use because of the `isofs` dependency. First, unload `isofs` using `modprobe -r isofs` and then unload `cdrom` with `modprobe -r cdrom`.

Demonstration: Unloading and Loading Modules

- Scenario: Unloading the `cdrom` module but encountering a dependency issue.

- Steps:

Check loaded modules using `lsmod`.

Attempt to unload `cdrom` with `modprobe -r cdrom`. Encounter an error indicating it's in use.

Use `findmnt` to ensure no mounts are using `cdrom`.

Identify and unload dependencies (e.g., `isofs`).

Retry unloading `cdrom`.

Load `isofs` with `modprobe isofs`, which automatically loads `cdrom` due to dependencies.

- **`lsmod | grep cdrom`**
- if necessary: **`modprobe -r cdrom`**
- if failing: **`modprobe -r isofs; modprobe -r sr_mod`**
- **`lsmod | grep cdrom`**
- **`modprobe isofs`**
- **`lsmod | grep cdrom`**
- **`modprobe -r cdrom`**

Tuning the Kernel

- The /proc pseudo filesystem is used for kernel tuning and optimization
- Use **echo** and command line redirection to modify specific kernel parameters immediately: **echo 60 > /proc/sys/vm/swappiness**
 - Notice these changes are not persistent
- The /proc filesystem also contains most of the runtime information about the kernel

```

[root@localhost ~]# mount | grep proc
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=31,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=21115)
[root@localhost ~]# cd /proc
[root@localhost proc]# ls
1      1725  2103  2265  2355  2414  2530  2823  3327  38    433  5     567  67   764  80    82    86    987      dma      keys      mpt      sysrq-trigger
10     1745  2105  2273  2365  2415  26    2841  3340  39    44   50    568  68   77   805   822   862   acpi     driver   key-users mtrr      sysvipc
106    1752  2108  2279  2370  2418  2618  2862  3379  4     447  533   57   681  772  806   83    863   asound   dynamic_debug kmsg     net       thread-self
1180   19    2150  2285  2379  2423  2677  2908  34    40    448  542   58   684  773  807   830   87    bootconfig execdomains kpagecgroup pagetypeinfo timer_list
1181   1992  2152  2286  2380  2431  2678  2967  3400  41    450  55    59   69   774  808   831   89    buddyinfo fb         kpagecount partitions tty
1186   2     2172  2287  2392  2438  2697  3     3403  42    452  56    60   7    775  81    833   9     bus       filesystems kpageflags schedstat uptime
12     20    22    2288  2393  2446  2699  30    3425  424   453  560   61   70   776  811   835   90    cgroups   fs         loadavg   scsi      version
1222   2057  2210  2293  2394  2461  27    31    3427  425   456  561   62   71   777  812   836   91    cmdline   interrupts locks      self      vmallocinfo
13     2071  2218  2306  2396  2485  2700  3191  3432  426   458  562   63   72   778  813   837   92    consoles  iomem     mdstat    slabinfo  vmstat
14     2075  2234  2326  24    2490  2704  32    3439  427   460  563   64   73   78   815   84    93    cpuinfo    ioports   meminfo   softirqs  zoneinfo
15     2094  2239  2332  2406  2499  2708  3200  35    429   461  564   65   74   782  816   843   94    crypto     irq       misc      stat      swaps
16     21    2240  2335  2407  2506  2748  3237  36    43    462  565   66   75   784  817   85    979   devices   kallsyms  modules   syscalls  sys
17     2101  226   2341  2413  2518  28    33    37    431   463  566   666   76   79   818   851   986   diskstats kcore     mounts    sys

[root@localhost proc]# cd 3327
[root@localhost 3327]# ls
ls: cannot read symbolic link 'exe': No such file or directory
arch_status cgroup      coredump_filter environ gid_map  map_files mounts      numa_maps  pagemap      root      setgroups  stat      task      uid_map
attr         clear_refs  cpu_resctrl_groups exe      io      maps      mountstats oom_adj      patch_state sched      smaps      statm     timens_offsets wchan
autogroup    cmdline     cpuset      fd       limits  mem       net         oom_score    personality schedstat  smaps_rollup status    timers
auxv         comm        cwd         fdinfo   loginuid mountinfo ns          oom_score_adj projid_map  sessionid  stack      syscall   timerslack_ns
[root@localhost 3327]# █

[root@localhost 447]# cd ../816/
[root@localhost 816]# cat cmdline
/usr/sbin/rsyslogd-n[root@localhost 816]# ls
arch_status cgroup      coredump_filter environ gid_map  map_files mounts      numa_maps  pagemap      root      setgroups  stat      task      uid_map
attr         clear_refs  cpu_resctrl_groups exe      io      maps      mountstats oom_adj      patch_state sched      smaps      statm     timens_offsets wchan
autogroup    cmdline     cpuset      fd       limits  mem       net         oom_score    personality schedstat  smaps_rollup status    timers
auxv         comm        cwd         fdinfo   loginuid mountinfo ns          oom_score_adj projid_map  sessionid  stack      syscall   timerslack_ns
[root@localhost 816]# cat status █

[root@localhost proc]# less meminfo
[root@localhost proc]# less cpuinfo
[root@localhost proc]# █

```



```
[root@localhost proc]# cd sys
```

```
[root@localhost sys]# ls
```

```
abi crypto debug dev fs kernel net user vm
```

```
[root@localhost sys]# ls kernel/
```

acct	hung_task_check_interval_secs	ostype
acpi_video_flags	hung_task_panic	overflowgid
auto_msgmni	hung_task_timeout_secs	overflowuid
bootloader_type	hung_task_warnings	panic
bootloader_version	io_delay_type	panic_on_io_nmi
bpf_stats_enabled	kexec_load_disabled	panic_on_oops
cad_pid	keys	panic_on_rcu_stall
cap_last_cap	kptr_restrict	panic_on_unrecovered_nmi
core_pattern	max_lock_depth	panic_on_warn
core_pipe_limit	max_rcu_stall_to_panic	panic_print
core_uses_pid	modprobe	perf_cpu_time_max_percent
ctrl-alt-del	modules_disabled	perf_event_max_contexts_per_stack
dmesg_restrict	msgmax	perf_event_max_sample_rate
domainname	msgmnb	perf_event_max_stack
firmware_config	msgmni	perf_event_mlock_kb
ftrace_dump_on_oops	msg_next_id	perf_event_paranoid
ftrace_enabled	ngroups_max	pid_max

```
[root@localhost sys]# ls vm
```

admin_reserve_kbytes	dirtytime_expire_seconds	memory_failure_early_kill	nr_hugepages	mempolicy	page-cluster
compaction_proactiveness	dirty_writeback_centisecs	memory_failure_recovery	nr_overcommit_hugepages		page_lock_unfairness
compact_memory	drop_caches	min_free_kbytes	numa_stat		panic_on_oom
compact_unevictable_allowed	extfrag_threshold	min_slab_ratio	numa_zonelist_order		percpu_pagelist_high_fraction
dirty_background_bytes	hugetlb_shm_group	min_unmapped_ratio	oom_dump_tasks		stat_interval
dirty_background_ratio	laptop_mode	mmap_min_addr	oom_kill_allocating_task		stat_refresh
		bits	overcommit_kbytes		swappiness
		compat_bits	overcommit_memory		unprivileged_userfaultfd
		yes	overcommit_ratio		user_reserve_kbytes

```
[root@localhost sys]# cat vm/swappiness
```

```
60
```

```
[root@localhost sys]# echo 30 > vm/swappiness
```

```
[root@localhost sys]# cat vm/swappiness
```

```
30
```

/proc/sys/vm/swappiness

The value in this file controls how aggressively the kernel will swap memory pages. Higher values increase aggressiveness, lower values decrease aggressiveness. The default value is 60.

Introduction to Kernel Tuning and the Proc File System

- Kernel tuning involves modifying parameters in the proc pseudo file system.
- Example: To change the swappiness parameter, use `echo 60 > /proc/sys/vm/swappiness`.

Understanding the Proc File System

- The proc file system contains runtime kernel information and is used by commands like `ps`.
- Example: Use `mount | grep proc` to see that proc is mounted on `/proc` with the type `proc`.

Viewing System Information

- `/proc` contains files with system information like memory and CPU details.
- Example: Use `cat /proc/meminfo` for detailed memory information.

Tuning Kernel Parameters through Proc Sys

- `/proc/sys` contains tunable parameters.
- Example: To change the swappiness value, use `echo 30 > /proc/sys/vm/swappiness`.

Using Sysctl for Persistent Kernel Parameter Tuning

- `sysctl` provides an interface for managing parameters in `/proc`.
- Example: To persistently set swappiness, write `vm.swappiness = 70` to `/etc/sysctl.d/swappiness.conf` and reload with `sysctl -p`.

Testing and Applying Tunings

- Test changes by echoing values to the proc file system, then make them persistent using `sysctl`.
- Example: After setting `vm.swappiness = 20` in `/etc/sysctl.d/swappiness.conf`, use `sysctl -p` to apply and `sysctl -a` to verify.

- `sysctl -a`
- `echo vm.swappiness=60 > /etc/sysctl.d/swappiness.conf`
- `sysctl -a | grep swapp`
- `sysctl -p /etc/sysctl.d/swappiness.conf`
- `sysctl -a | grep swapp`

```
[root@localhost ~]# sysctl -a | wc
  1001    3121    36800
[root@localhost ~]# echo vm.swappiness=70 > /etc/sysctl.d/swappiness.conf
[root@localhost ~]# sysctl -a | grep swapp
vm.swappiness = 70
[root@localhost ~]# echo vm.swappiness=20 > /etc/sysctl.d/swappiness.conf
[root@localhost ~]# sysctl -p /etc/sysctl.d/swappiness.conf
vm.swappiness = 20
[root@localhost ~]# sysctl -a | grep swapp
vm.swappiness = 20
[root@localhost ~]#
```

The Boot Procedure

Understanding the Boot Procedure

Managing Systemd Targets

Passing Parameters on the GRUB Boot Prompt

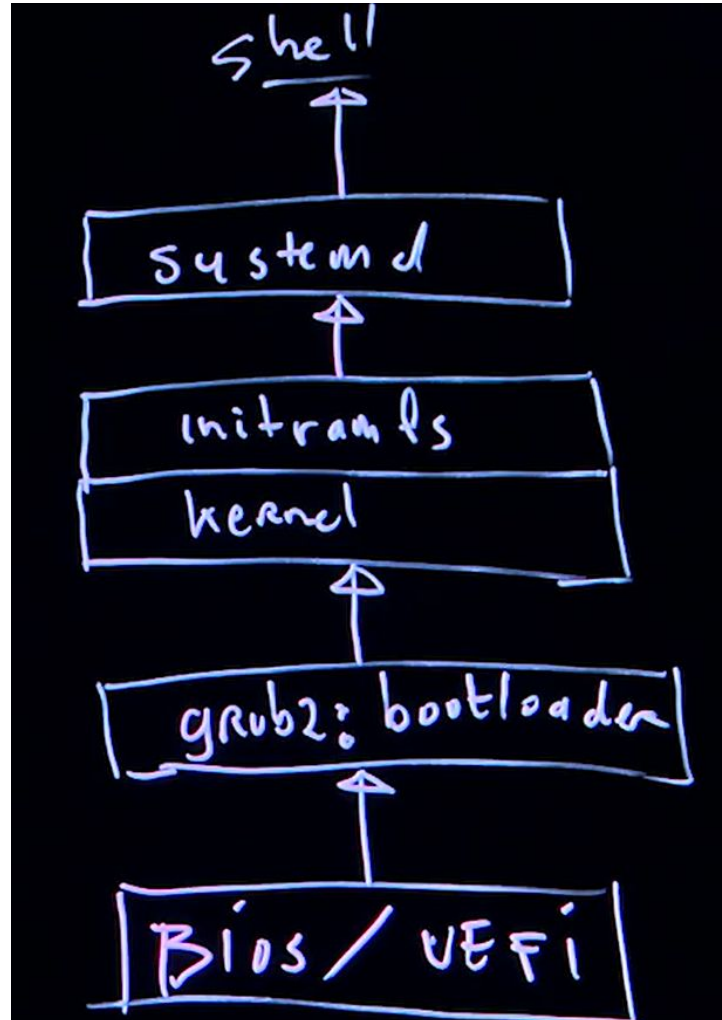
Editing Grub Configuration

Understanding Init and Upstart

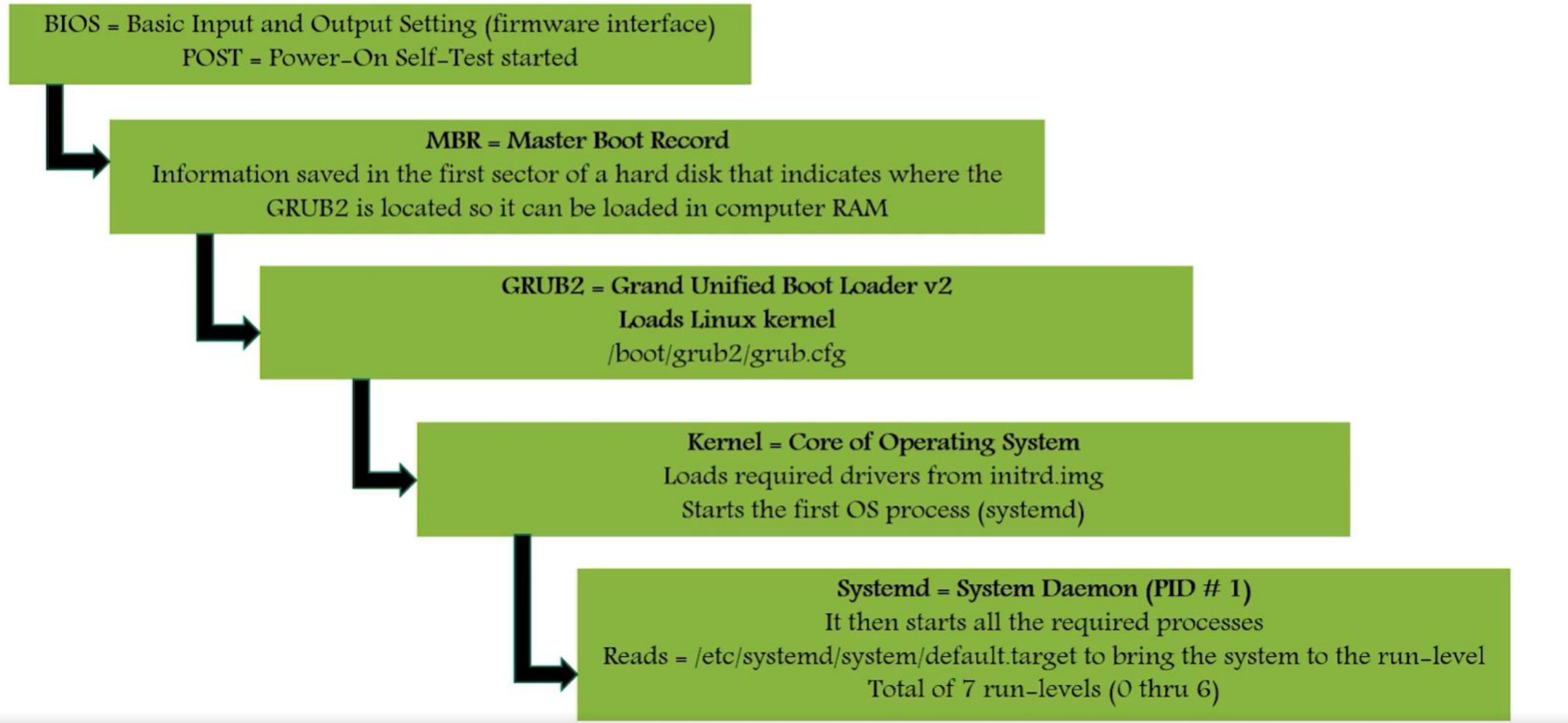
Troubleshooting the Boot Procedure

Using a Rescue Disk

Understanding the Boot Procedure



BIOS = Basic Input and Output Setting (firmware interface)
POST = Power-On Self-Test started



```
graph TD; A["BIOS = Basic Input and Output Setting (firmware interface)  
POST = Power-On Self-Test started"] --> B["MBR = Master Boot Record  
Information saved in the first sector of a hard disk that indicates where the GRUB2 is located so it can be loaded in computer RAM"]; B --> C["GRUB2 = Grand Unified Boot Loader v2  
Loads Linux kernel  
/boot/grub2/grub.cfg"]; C --> D["Kernel = Core of Operating System  
Loads required drivers from initrd.img  
Starts the first OS process (systemd)"]; D --> E["Systemd = System Daemon (PID # 1)  
It then starts all the required processes  
Reads = /etc/systemd/system/default.target to bring the system to the run-level  
Total of 7 run-levels (0 thru 6)"];
```

MBR = Master Boot Record
Information saved in the first sector of a hard disk that indicates where the GRUB2 is located so it can be loaded in computer RAM

GRUB2 = Grand Unified Boot Loader v2
Loads Linux kernel
/boot/grub2/grub.cfg

Kernel = Core of Operating System
Loads required drivers from initrd.img
Starts the first OS process (systemd)

Systemd = System Daemon (PID # 1)
It then starts all the required processes
Reads = /etc/systemd/system/default.target to bring the system to the run-level
Total of 7 run-levels (0 thru 6)

Managing Systemd Targets

Systemd Targets Overview

- A systemd target is a group of units.
- Example: `graphical.target` groups all units necessary for a graphical interface.
- Insight: Targets are like runlevels, defining what services are active.

Isolate Property in Targets

- Targets with the `isolate` property set can define the system's startup state.
- Example: `emergency.target` with `isolate` property for minimal system startup.
- Insight: Isolatable targets are essential for troubleshooting and system recovery.

Key Systemd Targets

- `emergency.target`: For minimal troubleshooting.
- `rescue.target`: For more complete system troubleshooting.
- `multi-user.target`: For a complete non-graphical system.
- `graphical.target`: For a complete graphical environment.
- Example: `sudo systemctl set-default graphical.target` sets the default to a graphical interface.
- Insight: These targets represent different levels of system functionality.

Managing Default Target

- Use `systemctl get-default` and `systemctl set-default` to manage the default target.
- Example: `systemctl get-default` shows the current default target.
- Insight: Changing the default target alters the system's initial state after booting.

Exploring Available Targets

- Use `systemctl list-unit-files -t target` to list all available targets.
-

```
[root@centos ~]# systemctl cat emergency.target
# /usr/lib/systemd/system/emergency.target
# SPDX-License-Identifier: LGPL-2.1-or-later
#
# This file is part of systemd.
#
# systemd is free software; you can redistribute it and/or modify it
# under the terms of the GNU Lesser General Public License as published by
# the Free Software Foundation; either version 2.1 of the License, or
# (at your option) any later version.
```

```
[Unit]
Description=Emergency Mode
Documentation=man:systemd.special(7)
Requires=emergency.service
After=emergency.service
```

```
AllowIsolate=yes
```

```
[root@centos ~]#
```

```
[root@centos ~]# systemctl get-default
```

```
graphical.target
```

```
[root@centos ~]# systemctl set-default multi-user.target
```

```
Removed "/etc/systemd/system/default.target".
```

```
Created symlink /etc/systemd/system/default.target → /usr/lib/systemd/system/multi-user.target.
```

```
[root@centos ~]# reboot
```

```
Password:
```

```
Last login: Tue Feb 21 12:44:36 on tty2
```

```
[student@centos ~]$ sudo systemctl start graphical.target
```

```
[student@centos ~]$ sudo systemctl isolate multi-user.target
```

```
Password:
```

```
Last login: Tue Feb 21 12:57:15 on tty2
```

```
[student@centos ~]$ sudo systemctl set-default graphical.target
```

```
[sudo] password for student:
```

```
Removed "/etc/systemd/system/default.target".
```

```
Created symlink /etc/systemd/system/default.target → /usr/lib/systemd/system/graphical.target.
```

```
[student@centos ~]$ sudo reboot
```

Passing Parameters on the GRUB Boot Prompt

Accessing Boot Menu

- Specific boot options can be passed while booting
- To access the boot menu, press and hold the Shift key, or press the Escape key briefly
- Enter boot options on the line that starts with **linux**

1. Accessing GRUB Boot Menu

- Example with Ubuntu Workstation:
 - Access by pressing the Shift key.
 - Demonstrate on Ubuntu: Pressing the Shift key reveals the GRUB boot menu.

2. Editing Boot Options in Ubuntu

- Example with Ubuntu Workstation:
 - Select the default boot option and press 'E' for edit.
 - Locate the line starting with "linux".
 - Remove "quiet" and "splash" from this line to disable quiet boot.
 - To boot with modified settings, press Control + X.
 - Observe the difference: Detailed boot information is displayed instead of a quiet splash screen.

3. Booting into Emergency Mode in CentOS

- Example with CentOS:
 - Access GRUB menu using the Escape key, especially with recent CentOS/Red Hat policy changes.
 - Press 'E' to edit boot options.
 - Find the line starting with "linux" which may span across three lines.
 - Remove "rhgb" and "quiet".
 - Add "systemd.unit=emergency.target" for troubleshooting.
 - Boot with modified settings using Control + X (not Enter + Control + X).
 - You will enter emergency mode, requiring a root password for maintenance.

4. Troubleshooting in Emergency Mode

- Example with CentOS Emergency Mode:
 - Use `systemctl list-units` to view available units in this mode, usually minimal.
 - For troubleshooting, ensure the file system is in read-write mode.
 - If not, use `mount -o remount,rw/` to remount the root file system.
 - You can then perform troubleshooting actions like creating files.

5. Exiting Emergency Mode

- Example with CentOS Emergency Mode:
 - To reboot from emergency mode, use `systemctl reboot`.
 - Highlight that typing "reboot" actually invokes `systemctl reboot`.

Grub Configuration files

Introduction to Grub Configuration:

- Grub Configuration File Location: The grub configuration file is located at `/boot/grub2/grub.cfg`.
- Importance of Not Editing Directly: Direct modifications to `grub.cfg` should be avoided as they can be overwritten during system updates.

Safe Editing Practices:

- Correct File for Modifications: Modifications should be made to `/etc/default/grub` instead of directly editing `grub.cfg`.
- Example: To change the grub timeout, modify the `GRUB_TIMEOUT` value in `/etc/default/grub`.

Applying Changes:

- Using `grub2-mkconfig`: After modifying `/etc/default/grub`, use `grub2-mkconfig -o /boot/grub2/grub.cfg` to apply changes.
- Example: Run `sudo grub2-mkconfig -o /boot/grub2/grub.cfg` to update grub with the new settings.

init or Runlevels

Introduction to Init and Upstart

- *Point:* Init has been the traditional system for starting services in Linux, while Upstart was an improvement to Init.
- *Example:* In the early days of Linux, when the kernel booted, it would start `/sbin/init`. This Init would then start services defined in `etc/init.d` based on the default runlevel.
- *Insight:* Understanding Init is crucial for dealing with older Linux systems or specific subsystems like WSL2 (Windows Subsystem for Linux Version Two).

Runlevels in Init

- *Point:* Init uses the concept of runlevels to define the state of the system.
- *Examples:*
 - Runlevel 0: Halt the system. Command: `init 0`.
 - Runlevel 1: Rescue mode, similar to Systemd's emergency target. Command: `init 1`.
 - Runlevel 2: Rescue mode with network, akin to Systemd's rescue target.
 - Runlevel 3: Multi-user mode.
 - Runlevel 5: Graphical mode.
 - Runlevel 6: Reboot the system. Command: `init 6`.
- *Insight:* Understanding these runlevels is important for system administration tasks, especially in troubleshooting.

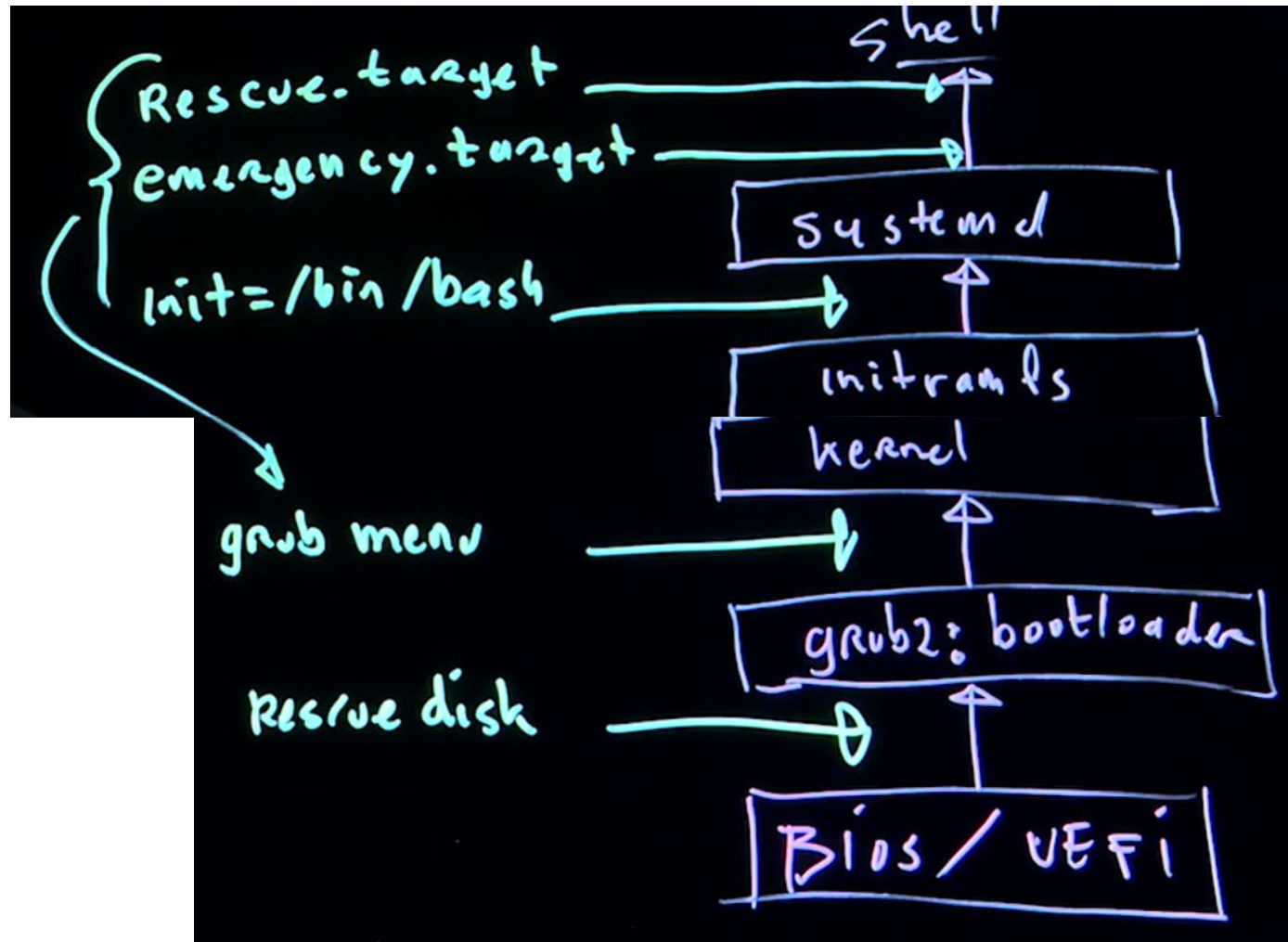
Switching Runlevels

- *Point:* On a running system, the `init` or `telinit` command is used to change runlevels.
- *Example:* To reboot a system, you would use `init 6`.
- *Insight:* Although Systemd has replaced Init in modern distributions, knowing how to switch runlevels is helpful for backward compatibility.

Systemd: The Modern Replacement

- *Point:* Systemd has largely replaced Init and Upstart in modern Linux distributions.
- *Example:* Systemd uses 'targets' instead of runlevels. For instance, `systemctl isolate graphical.target` would be the equivalent of moving to runlevel 5 in Init.
- *Insight:* Emphasizing the importance of learning Systemd, as it is the future of service management in Linux.

Troubleshooting the Boot Procedure



Managing Security

Managing Local Security

Firewall Management

SELinux and Apparmor

Managing Local Security:

File Access Control Lists

Filesystem Attributes

Pluggable Authentication Modules

File Access Control Lists

Introduction to ACLs:

- ACL stands for Access Control List.
- It provides a flexible, additional permission mechanism for file systems, supplementing UNIX file permissions.
- ACLs allow specific permissions for individual users or groups.
- Example: If user Jai is not in the group 'tif', but needs read/write access to a specific file in `/home/user/docs`, ACLs can be used to grant this access without adding Jai to the 'tif' group.

Purpose of ACLs:

- ACLs grant specific permissions to users or groups beyond the basic file permissions.
- Example: A file in `/home/user/docs` created by a user in 'tif' group can be made readable by Jai, who is not in the 'tif' group, using ACLs.

Commands for ACLs:

- `setfacl`: Sets ACL permissions.
- `getfacl`: Displays existing ACL permissions on a file.
- Example: To grant Jai read and write access to a file `/home/user/docs/report.txt`, use `setfacl -m u:jai:rw /home/user/docs/report.txt`.

Adding Permissions:

- For users: `setfacl -m u:[username]:[permission] [file]`
- For groups: `setfacl -m g:[groupname]:[permission] [file]`
- Example: To give read access to the group 'tif' on a file, use `setfacl -m g:tif:r [file]`.

Inheriting ACLs:

- To cascade ACL entries to all files/directories within a directory, use `setfacl -dm [entry] [directory]`.
- Example: To ensure all files in `/home/user/projects` inherit the same ACL, use `setfacl -dm u:jai:rw /home/user/projects`.

Removing ACL Entries:

- **Specific entry:** `setfacl -x [entry] [file]`
- **All entries:** `setfacl -b [file]`
- **Example:** To remove Jai's ACL from `/home/user/docs/report.txt`, use `setfacl -x u:jai /home/user/docs/report.txt`.

ACL Indicators and Limitations:

- A plus sign (+) at the end of permissions in `ls -l` output indicates ACLs are set.
- Write permission (w) via ACL doesn't allow file deletion.
- **Example:** If `ls -l` shows `-rw-rw-r--+` on a file, it indicates ACLs are active.

Practical Application:

- Log in as a user and create a file in `/home/user/newfile`.
- Use `getfacl` to view current permissions.
- Apply `setfacl` to grant Jai specific user permissions.
- Verify with `getfacl`.
- **Example:** Create `newfile.txt` in `/home/user`, grant Jai write access, and verify with `getfacl /home/user/newfile.txt`.

ACLs for Directories:

- Similar to files, ACLs can be set for directories.
- **Example:** To allow Jai to access `/home/user/shared`, set ACLs on the directory, like `setfacl -m u:jai:rwX /home/user/shared`.

Experimentation and Learning:

- Experiment with different files and directories to understand ACLs' impact.
- **Example:** Set different ACLs on various files and directories in `/home/user`, and test access as different users.

```

[root@centos ~]# groupadd account
[root@centos ~]# mkdir -p /data/sales
[root@centos ~]# groupadd sales
[root@centos ~]# chgrp sales /data/sales
[root@centos ~]# chmod 770 /data/sales
[root@centos ~]# ls -ld /data/sales/
drwxrwx---. 2 root sales 6 Feb 21 14:24 /data/sales/
[root@centos ~]# touch /data/sales/file{1..10}
[root@centos ~]# ls -l /data/sales/
total 0
-rw-r--r--. 1 root root 0 Feb 21 14:25 file1
-rw-r--r--. 1 root root 0 Feb 21 14:25 file10
-rw-r--r--. 1 root root 0 Feb 21 14:25 file2
-rw-r--r--. 1 root root 0 Feb 21 14:25 file3
-rw-r--r--. 1 root root 0 Feb 21 14:25 file4
-rw-r--r--. 1 root root 0 Feb 21 14:25 file5
-rw-r--r--. 1 root root 0 Feb 21 14:25 file6
-rw-r--r--. 1 root root 0 Feb 21 14:25 file7
-rw-r--r--. 1 root root 0 Feb 21 14:25 file8
-rw-r--r--. 1 root root 0 Feb 21 14:25 file9
[root@centos ~]# setfacl -R -m g:account:rx /data/sales
[root@centos ~]# ls -l /data/sales/
total 0
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file1
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file10
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file2
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file3
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file4
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file5
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file6
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file7
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file8
-rw-r--r--+ 1 root root 0 Feb 21 14:25 file9
[root@centos ~]# getfacl /data/sales/file1
getfacl: Removing leading '/' from absolute path names
# file: data/sales/file1
# owner: root
# group: root
user::rw-
group::r--
group:account:r--
mask::r--
other::r--

```

```

[root@centos ~]# setfacl -m d:g:account:rx /data/sales
[root@centos ~]# touch /data/sales/newfile
[root@centos ~]# getfacl /data/sales/newfile
getfacl: Removing leading '/' from absolute path names
# file: data/sales/newfile
# owner: root
# group: root
user::rw-
group::rwx
group:account:r-x
mask::rw-
other::---

[root@centos ~]# █

```


Filesystem Attributes

- Posix defines a number of attributes that can be used to add security to files
- Use **chattr** to set them and **lsattr** to get an overview of currently applied attributes
- Of all attributes, the immutable (**i**) attribute is common

```
[root@centos ~]# touch removeme.txt
[root@centos ~]# chattr +i removeme.txt
[root@centos ~]# rm -f removeme.txt
rm: cannot remove 'removeme.txt': Operation not permitted
[root@centos ~]# whoami
root
[root@centos ~]# lsattr removeme.txt
----i----- removeme.txt
[root@centos ~]# chattr -i removeme.txt
[root@centos ~]# rm -f removeme.txt
[root@centos ~]# █
```

Firewall Management

Understanding Linux Firewalling

Opening Ports and Services with Firewalld

Working with Zones

Using Rich Rules

Using UFW

Understanding Linux Firewalling

Kernel and Netfilter

- Point: Linux firewalling starts with the kernel, specifically with a functionality called "netfilter."
- Example & Insight: Consider a Linux system as a building with a security checkpoint. The kernel is the structure of the building, and netfilter acts like the security team, checking the contents of every package (or network packet) that enters or leaves the building.

Packet Flows: Input, Forward, Output

- Point: Netfilter tracks different flows of packets: input, forward (for routing), and output.
- Example & Insight: Imagine a mail room in an office. "Input" is like receiving mail, "forward" is like routing mail to different departments, and "output" is like sending mail out. The firewall tracks and manages these mail flows.

Defining Firewall Rules

- Point: Users need to define rules for what is allowed in each packet flow: input, forward, and output.
- Example & Insight: Like setting up rules for what kind of mail is allowed to be received, sent, or routed in an office. For instance, you might allow all internal emails (input), block certain external emails (output), and route specific emails to certain departments (forward).

Iptables and Nftables

- Point: Earlier, iptables was the main utility for firewall rules, which is now often replaced by nftables.
- Example & Insight: Think of iptables and nftables like older and newer versions of a security protocol. Iptables is like an older, complex manual system, while nftables is a more modern, efficient method.

Complexity of Low-End Utilities

- Point: Utilities like iptables and nftables are complex and hence not widely used directly.
- Example & Insight: Using iptables is akin to programming a VCR – technically possible but complex and outdated for most users.

High-Level Utilities: FirewallD and UFW

- Point: High-level utilities like firewallD (Red Hat) and ufw (Ubuntu) are provided for easier firewall rule writing.
- Example & Insight: These utilities are like user-friendly remote controls for the VCR (iptables/nftables). They provide a simpler interface for the same functionality.

Interaction Between Utilities

- Point: FirewallD or ufw write rules to iptables or nftables, which then define the input, output, and forward rules.
- Example & Insight: When you use firewallD or ufw, it's like telling a smart assistant to record a show. The assistant (firewalld/ufw) understands your simple command and sets up the complex recording (iptables/nftables) for you.

Opening Ports and Services with Firewall

Using `firewall-cmd`:

- The primary command-line interface for managing `firewalld`.
- Offers intuitive options like `get`, `set`, `list`, `add`, and `remove` for different elements.
- Example: `firewall-cmd --list-all` shows the current configuration in the active zone.

Services and Ports:

- Listing Services: `firewall-cmd --get-services` displays all available services.
- Adding Services: `firewall-cmd --add-service http` to enable a service.
- Managing Ports: `firewall-cmd --add-port 123/tcp` adds a specific port.
- Example: `/usr/lib/firewalld/services/http.xml` shows the XML configuration of the HTTP service.

- **`firewall-cmd --list-services`**
- **`firewall-cmd --get-services`**
- **`firewall-cmd --add-service=service`**
- **`firewall-cmd --remove-service=service`**

- **firewall-cmd --list-all**
- **firewall-cmd --get-services**
- **firewall-cmd --add-service http**
- **cat /usr/lib/firewalld/services/http.xml**
- **firewall-cmd --add-port 123/tcp**
- **firewall-cmd --list-all**
- **firewall-cmd --runtime-to-permanent**

```
[root@centos ~]# firewall-cmd --add-port 123/tcp
success
[root@centos ~]# firewall-cmd --add-service http
success
[root@centos ~]# firewall-cmd --runtime-to-permanent
success
```

```
[root@centos ~]# firewall-cmd --add-service http
success
[root@centos ~]# firewall-cmd --add-port 123/tcp
success
[root@centos ~]# firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens33
  sources:
  services: cockpit dhcpv6-client http ssh
  ports: 123/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[root@centos ~]# systemctl restart firewalld
[root@centos ~]# firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens33
  sources:
  services: cockpit dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[root@centos ~]#
```

Working with Zones

- FirewallD and Zones:
 - i. Think of firewalld like a security guard for your computer's network. It decides what internet traffic is allowed in and out.
 - ii. 'Zones' in firewalld are like different security levels or areas that this guard monitors. Each zone has its own rules about what is allowed.
- Network Interfaces and Default Zone:
 - i. A network interface is like a door through which your computer connects to different networks (like the internet or a private network).
 - ii. The 'public' zone is the default, meaning it's the general set of rules applied to any network traffic unless specified otherwise.
- Viewing Available and Active Zones:
 - i. You can check what zones (security levels) are available and see their specific rules.
 - ii. For example, some zones might be very strict ('trusted' zone), allowing almost nothing, while others ('work' zone) might be more lenient.
- Changing the Default Zone and Adding Services:
 - i. You can change the default set of rules (zone) if you want.
 - ii. Adding a service is like saying, "I want to allow this specific type of traffic." You can add it to the general rules or specify a particular zone for it.

- **firewall-cmd --get-zones**
- **firewall-cmd --list-all-zones**
- **firewall-cmd --list-all**
- **firewall-cmd --get-default-zone**
- **firewall-cmd --set-default-zone**
- **firewall-cmd --add-service https --zone dmz**

```
[root@centos ~]# firewall-cmd --get-zones
block dmz drop external home internal nm-shared public trusted work
[root@centos ~]# firewall-cmd --list-all-zones
```

```
[root@centos ~]# firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens33
  sources:
  services: cockpit dhcpv6-client http ssh
  ports: 123/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

```
[root@centos ~]# firewall-cmd --get-default-zone
public
```

```
[root@centos ~]# firewall-cmd --set-default-zone
```

```
usage: see firewall-cmd man page
```

```
firewall-cmd: error: argument --set-default-zone: expected one argument
```

```
[root@centos ~]# firewall-cmd --set-default-zone internal
```

```
success
```

```
[root@centos ~]# firewall-cmd --get-default-zone
```

```
internal
```

```
[root@centos ~]# firewall-cmd --add-service https --zone dmz
```

```
success
```

```
[root@centos ~]# firewall-cmd --set-default-zone public
```

```
success
```

```
[root@centos ~]#
```

Using Rich Rules

Introduction

- FirewallD's introduction faced criticism due to its lack of advanced features.
- This led to the development of direct rules and rich rules for more complex configurations.

Direct Rules

- Direct rules allow administrators to insert hand-coded rules directly.
- These rules are processed before any other rules in the zones.
- However, using direct rules is not generally recommended.

Rich Rules

- Rich rules are a preferred method, using an expressive language for custom rule creation.
- Essential for advanced features like logging, port forwarding, masquerading, rate limiting.
- To create a rich rule: use `firewall-cmd --add-rich-rule='RULE'`.
- Examples and details can be found in the `firewalld.richlanguage` man page.


```
[root@centos ~]# firewall-cmd --permanent --zone=public --add-rich-rule='rule family=ipv4 source address=192.168.29.220 reject'
success
[root@centos ~]# firewall-cmd --permanent --zone=public --add-rich-rule='rule service name=http log limit value=3/m accept'
success
[root@centos ~]# firewall-cmd --permanent --zone=public --add-rich-rule='rule protocol value=igmp accept'
success
[root@centos ~]# less /etc/protocols
[root@centos ~]#
```

```
[root@centos ~]# firewall-cmd --permanent --zone=public --add-rich-rule='rule family=ipv4 source address=10.0.0.0/24 port port=7900-7905 protocol=
tcp accept'
success
```

```
[root@centos ~]# firewall-cmd --permanent --zone=public --add-rich-rule='rule service name="ssh" log prefix="ssh" level="notice" limit value="2/m"
accept'
success
[root@centos ~]# firewall-cmd --list-all
public (active)
```

```
[root@centos ~]# systemctl restart firewalld
[root@centos ~]# firewall-cmd --list-all
```

UFW

- **sudo ufw enable**
- **sudo ufw allow ssh (checks details in /etc/services)**
- **sudo ufw reject out ssh**
- **sudo ufw status**
- **sudo ufw delete reject out ssh**
- **sudo ufw deny proto tcp from 10.0.0.10 to any port 22**
- **sudo ufw reset**
- **sudo ufw app list**
- **sudo ufw app info Samba**
- **sudo ufw logging on**
- **man ufw**

Introduction to UFW:

- UFW is designed for easy use with an intuitive syntax.
- Example: On an Ubuntu server, checking the firewall status is done with `sudo ufw status`.

Activating UFW:

- To activate UFW, use `sudo ufw enable`.
- Example: After activation, using `sudo ufw status` again will show that it is active.

Adding Rules:

- You can allow or reject specific types of traffic.
- Example: Allowing SSH access with `sudo ufw allow ssh`. This adds rules for both IPv4 and IPv6.

Rejecting Specific Traffic:

- To reject outgoing SSH, use `sudo ufw reject out ssh`.
- Example: To check current rules, use `sudo ufw status`. It will show both allowed and rejected rules.

Deleting Rules:

- Rules can be removed if they are no longer needed.
- Example: To delete a rule, use `sudo ufw delete reject out ssh`.

Advanced Rule Specification:

- UFW allows setting advanced rules like denying specific protocols from certain IPs.
- Example: Denying TCP traffic from a specific IP using `sudo ufw deny proto tcp from 10.0.0.10 to any port 22`.

Resetting UFW Rules:

- To reset all rules, use `sudo ufw reset`.
- Example: This is useful as a last resort or when starting fresh with firewall configurations.

Working with Apps and Services:

- UFW integrates with applications and services, using service names defined in `/etc/services`.
- Example: To list available apps, use `sudo ufw app list`. Information on a specific app can be obtained with `sudo ufw app info OpenSSH`.

SELinux and Apparmor

Understanding Mandatory Access Control

Confining Services with AppArmor

Understanding SELinux

Applying Labels to Manage SELinux File Access

Applying Labels to Manage SELinux Port Access

Configuring Booleans

Troubleshooting SELinux Access

Understanding Mandatory Access Control

Introduction to Mandatory Access Control in Linux:

- Linux was not initially developed with a strong focus on security.
- Over time, various security solutions have been introduced to address different aspects of Linux security.
- Example: The introduction of mandatory access control in the 1990s to enhance Linux security.

Main Solutions for Mandatory Access Control: SELinux and AppArmor

- **SELinux:** Primarily found in Red Hat environments.
 - It operates on an "if not explicitly allowed, then denied" principle.
 - SELinux is more complex but offers advanced features like multi-level security.
 - Example: In an organization using Red Hat, SELinux could be used to enforce strict access controls based on security clearances.
- **AppArmor:** Commonly used in Ubuntu environments.
 - Works with profiles to secure specific areas.
 - Easier to learn and implement compared to SELinux.
 - Example: A web server in Ubuntu might have an AppArmor profile to restrict its access to only necessary files and directories.

How SELinux and AppArmor Function:

- **SELinux:**
 - Uses file system labels for security enforcement.
 - Involves matching the correct labels to control access.
 - Example: A file labeled as "confidential" in SELinux can only be accessed by users or processes with the corresponding clearance.
- **AppArmor:**
 - Manages security through profiles for individual applications or services.
 - Example: Creating a profile in AppArmor for a database service to limit its file system access and network permissions.

Learning and Configuration Aspects:

- SELinux has a steeper learning curve but provides more granular security control.
- AppArmor is user-friendly, making it suitable for less complex security needs.
- Example: A system administrator new to Linux security might start with AppArmor for basic security needs and move to SELinux for more advanced requirements.

Default Security in Different Linux Distributions:

- SELinux is the default in Red Hat.
- AppArmor is the default in Ubuntu.

Confining Services with AppArmor

- **sudo systemctl status apparmor**
- **sudo apt search apparmor**
- **sudo apt install apparmor-utils apparmor-profiles apparmor-profiles-extra**
- **aa-status**
- **aa-genprof /usr/bin/vim**
- Scan for events and finish profile
- **vim /etc/motd**
- **vim /etc/apparmor.d/usr.bin.vim.basic**
- **nano /etc/apparmor.d/usr.bin.vim.basic**
- **rm /etc/apparmor.d/usr.bin.vim.basic**

Selinux

Introduction to SELinux

- SELinux provides a policy framework for access control.
- It defines which source object (usually processes) can access which target object (like files and ports).

Policy Rules and Labels

- Policies are based on labels.
- Label Components: User, Role, Type.
- In basic SELinux usage, focus on the 'type' component.

SELinux Labels and Access Control

Access Control Mechanism

- SELinux checks rules for source context type requesting access to target context type.
- If no rule allows it, SELinux denies access.

SELinux Modes and States

SELinux Modes and States

- States: Enabled or Disabled (set during boot).
- Modes: Permissive and Enforcing.
 - Permissive: Logs events, but doesn't enforce policies.
 - Enforcing: Fully operational, enforces all policies.

Managing SELinux Modes

- Use `getenforce` to view current mode.
- Use `setenforce` to change modes between Permissive and Enforcing.
- Changing modes doesn't affect the SELinux state (Enabled/Disabled).

```
[root@centos ~]# getenforce
Enforcing
[root@centos ~]# setenforce permissive
[root@centos ~]# getenforce
Permissive
[root@centos ~]# setenforce disabled
usage: setenforce [ Enforcing | Permissive | 1 | 0 ]
[root@centos ~]# vim /etc/sysconfig/selinux
[root@centos ~]# reboot
```

```
load_video
set gfxpayload=keep
insmod gzio
linux ($root)/vmlinuz-5.14.0-160.el9.x86_64 root=/dev/mapper/cs-root ro crashk\
ernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/cs-swap rd.lvm.lv=cs\
/root rd.lvm.lv=cs/swap selinux=0
initrd ($root)/initramfs-5.14.0-160.el9.x86_64.img $tuned_initrd
```

```
[student@centos ~]$ sudo -i
[sudo] password for student:
[root@centos ~]# getenforce
Disabled
[root@centos ~]# setenforce enforcing
setenforce: SELinux is disabled
[root@centos ~]#
```

Disabling SELinux

- Add `SELinux=0` at the end of the Linux line in GRUB config.
- After reboot, use `getenforce` to confirm SELinux is disabled.
- Note: `setenforce enforcing` won't work if SELinux is disabled; a reboot is necessary to re-enable.

Applying Labels to Manage SELinux File Access

Understanding SELinux Labels:

- Key Concept: SELinux uses context labels for security control.
- Command Insight: Use `-Z` option with commands to display label information.
- Example: `ls -Z /var/www` shows SELinux labels on files.

Importance of Context Type in Labels:

- Key Concept: The context type is crucial in SELinux labels.
- Command Insight: Use `semanage fcontext` to set context labels on files.
- Example: After executing `semanage fcontext`, the policy is updated but not the file itself.

Applying Labels to Files with `restorecon`:

- Key Concept: `restorecon` applies labels from policy to file inodes.
- Example: After using `semanage fcontext`, use `restorecon` to update file labels.

Labeling Network Ports:

- Key Concept: Use `semanage port` for network port labeling.
- Command Insight: No need for `restorecon` after setting labels with `semanage port`.
- Example: After `semanage port`, network ports are immediately updated with new context labels.

```
sed -i -e 's/^DocumentRoot. */DocumentRoot \/web/' /etc/httpd/conf/httpd.conf
```

```
sed -i -e 's\/\var\/www\/\web/' /etc/httpd/conf/httpd
```

```
# Some configuration lines
```

```
DocumentRoot "/web"
```

```
# More configurations
```

```
<Directory "web/html">
```

```
    # Directory specific settings
```

```
</Directory>
```

```
[root@centos ~]# dnf install -y httpd
Last metadata expiration check: 2:36:19 ago on Wed 22 Feb 2023 07:38:16 AM PST.
Package httpd-2.4.53-11.el9.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@centos ~]# systemctl enable --now httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@centos ~]# ps Zaux | grep httpd
system_u:system_r:httpd_t:s0 root 2956 0.0 0.3 20376 11748 ? Ss 10:14 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 2968 0.0 0.2 21700 7532 ? S 10:15 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 2969 0.0 0.2 1538224 11092 ? Sl 10:15 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 2970 0.0 0.2 1538224 11092 ? Sl 10:15 0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 2971 0.0 0.5 1669360 19272 ? Sl 10:15 0:00 /usr/sbin/httpd -DFOREGROUND
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 root 3184 0.0 0.0 221664 2220 pts/0 S+ 10:15 0:00 grep --color=auto httpd
[root@centos ~]# ls -Z /var/www
system_u:object_r:httpd_sys_script_exec_t:s0 cgi-bin system_u:object_r:httpd_sys_content_t:s0 html
[root@centos ~]# mkdir /web
[root@centos ~]# ls -ldZ /web
drwxr-xr-x. 2 root root unconfined_u:object_r:default_t:s0 6 Feb 22 10:17 /web
[root@centos ~]#
```

```
[root@centos ~]# echo hello world > /web/index.html
[root@centos ~]# sed -i -e 's/^DocumentRoot.*/DocumentRoot \/web/' /etc/httpd/conf/httpd.conf
[root@centos ~]# sed -i -e 's/\/var\/www\/web/' /etc/httpd/conf/httpd.conf
[root@centos ~]# systemctl restart httpd
[root@centos ~]# curl localhost
```

```
[root@centos ~]# setenforce 0
[root@centos ~]# curl localhost
hello world
[root@centos ~]# setenforce 1
[root@centos ~]#
```

- **grep AVC /var/log/audit/audit.log**
- **setenforce 1**
- **ls -Z /var/www/**
- **ls -Zd /web**
- **man semanage-fcontext**
- **semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"**
- **restorecon -Rv /web**
- **curl localhost**

Applying Labels to Manage SELinux Port Access

- **sed -i -e 's/^Listen.*/Listen 82/' /etc/httpd/conf/httpd.conf**
- **systemctl restart httpd**
- **systemctl status httpd**
- **grep AVC /var/log/audit/audit.log**
- **semanage port -a -t http_port_t -p tcp 82**
- **systemctl restart httpd**
- **systemctl status httpd**

Configuring Booleans

- A boolean is an on/off switch that allows you to easily apply settings in SELinux
- Booleans are used in addition to context labels
- Use **getsebool -a** to get a list of all booleans
- Use **setsebool -P** to make persistent changes to booleans

Troubleshooting SELinux Access

- SELinux events are logged to the audit log in /var/log/audit/audit.log
- Messages in the audit log are not always very readable
- Apart from that, there is **sealert**, which is part of the setroubleshoot-server package and logs readable messages to your logging system
- Look for the **sealert** command example in your log files and run this command to find out what to do

```
[root@localhost ~]# journalctl | grep sealert —  
Dec 08 09:07:24 localhost.localdomain setroubleshoot[1147]: SEL  
ckup-script.sh. For complete SELinux messages run: sealert -l a  
Dec 09 09:34:54 localhost.localdomain setroubleshoot[1227]: SEL  
ckup-script.sh. For complete SELinux messages run: sealert -l a  
Dec 10 18:20:53 localhost.localdomain setroubleshoot[1488]: SEL  
ckup-script.sh. For complete SELinux messages run: sealert -l a
```

```
[root@localhost ~]# sealert -l 7ae4cd2b-a429-4f68-9458-a63075c05b13 | less
```

BackUps - rsync

Definition and Use-Cases

- rsync is a utility for synchronizing files between directories.
- Primarily used for syncing between remote systems.
- Example: To sync a local directory with a remote one, you might use `rsync -azP /local/directory remote_user@remote_host:/remote/directory`. This command demonstrates the basic syntax of rsync.

Requirements

Remote System Requirements

- The remote system must have sshd (Secure Shell Daemon) running.
- Example: Before using rsync, ensure the remote system is accessible over SSH (e.g., using `ssh remote_user@remote_host` to check connectivity).

Preparing for Demonstration

- Requires two computers.
- Example: Setting up a folder and files using `mkdir files` and `touch files/file{1..100}` to create a series of files in a directory.

Command Breakdown

Understanding Rsync Options

- `-a` (archive mode): Preserves file attributes.
- `-z` (compress): Compresses file data during transfer.
- `-P` (progress): Shows progress during transfer.
- Example: `rsync -azP /source/directory remote_user@remote_host:/destination/directory` demonstrates syncing with progress, compression, and preserving file attributes.

Practical Usage

Syncing Files to a Remote Server

- Syncing created files to a remote directory.
- Example: Using `rsync -azP files/ remote_user@192.168.29.169:/home/student/files/` to sync the local 'files' directory to a remote server.

Additional File Operations

Adding and Removing Files

- Adding more files (`touch files/file{101..110}`).
- Removing a file (`rm files/file50`).
- Example: Demonstrating that added files are synced, but removed files aren't without extra options.

Advanced Syncing Options

Deletion in Rsync

- By default, rsync does not delete files in the destination that are not present in the source.
- Using `--delete` option to sync deletions.
- Example: `rsync -azP --delete /source/directory remote_user@remote_host:/destination/directory` to include deletion of files in the destination that are missing in the source.


```
[student@centos ~]$ mkdir files
[student@centos ~]$ touch files/file{1..100}
[student@centos ~]$ rsync -azP files/ student@192.168.29.169:/home/student/files
```

```
student@student-virtual-machine:~$ ls
Desktop  Documents  Downloads  files  Music  Pictures  Public  snap  Templates  Videos
student@student-virtual-machine:~$ ls files
file1    file14  file2    file25  file30  file36  file41  file47  file52  file58  file63
file10   file15  file20   file26  file31  file37  file42  file48  file53  file59  file64
file100  file16  file21   file27  file32  file38  file43  file49  file54  file6   file65
file11   file17  file22   file28  file33  file39  file44  file5   file55  file60  file66
file12   file18  file23   file29  file34  file4   file45  file50  file56  file61  file67
file13   file19  file24   file3   file35  file40  file46  file51  file57  file62  file68
student@student-virtual-machine:~$
```

Using dd

- **dd** was developed to work with complete devices
- Using **dd** makes it possible to make an ISO of an optical disk, or to clone a complete hard disk
- **dd** should not be used for copying files, or making archives of files

LDAP

Understanding LDAP

Introduction to LDAP

- LDAP stands for Lightweight Directory Access Protocol.
- It's a centralized solution for storing important information.
- Example: Consider LDAP as a digital phone directory where you can store various contact details.

Uses of LDAP

- Primarily used for authentication purposes.
- Capable of storing different types of information beyond authentication data.
- Example: Apart from login credentials, LDAP can store user profiles, email addresses, and organizational roles.

LDAP's Historical Context

- Originated as a phone dictionary.
- Demonstrates its versatility in storing diverse types of data.
- Example: Initially used to store phone numbers and addresses, now expanded to digital information like emails.

Security Concerns with LDAP

- Default LDAP communication occurs over port 389.
- This default setup is not secure as passwords are transmitted unencrypted.
- Example: If a user logs in via LDAP on port 389, their password could be intercepted due to lack of encryption.

Setting up an LDAP Server

Introduction to LDAP Server and IPA

- LDAP server is essential for centralized authentication.
- IPA (Identity, Policy, and Authorization) is a popular choice.
- FreeIPA, a Linux-based alternative to Active Directory, can be installed on various distributions or run as a container.
- Administration is possible through LDAP utilities, IPA command line, or web interface.

Example:

- Install FreeIPA on a CentOS system.
- Use `hostnamectl` to set a unique hostname, such as `freeipa.example.local`, avoiding common domains like `.com`.

Preparation for IPA Installation

- IPA integrates tightly with DNS, requiring a unique hostname.
- Hostname should be unique in the DNS context.
- Use `hostnamectl` for hostname setup.

Example:

- Set hostname: `hostnamectl set-hostname freeipa.example.local`.
- Ensure hostname resolves only on the IPA server.

Installing FreeIPA

- Modify `/etc/hosts` to include the local IP address.
- Install using package manager (e.g., `dnf install -y ipa-server bind dyndb_ldap ipa-server-dns`).

Example:

- Execute: `dnf install -y ipa-server bind dyndb_ldap ipa-server-dns`.

Running IPA Server Installation

- Use `ipa-server-install --setup-dns`.
- This setup installs various components like a standalone certificate authority, NTP client, directory server, and more.
- Accept default settings prompted during installation.

Example:

- Execute: `ipa-server-install --setup-dns`.
- Set passwords for directory manager and IPA admin.
- Configure DNS forwarder if necessary.

Finalizing Installation

- Configure firewall settings.
- Use `firewall-cmd --reload` to activate firewall settings.
- Backup CA certificate stored in `/root/cacert.p12`.

Example:

- Execute firewall loop and reload: `firewall-cmd --reload`.

Creating LDAP Users with IPA Tools

- Use `kinit admin` to access Kerberized environment.
- Add users with `ipa user add`.
- Set passwords for LDAP users using `ipa passwd`.

Example:

- Login: `kinit admin`.
- Add user: `ipa user add ldapuser1`.
- Set password: `ipa passwd ldapuser1`.

Using LDAP Clients

- Utilize LDAP clients to interact with the server.
- Demonstrated in the next part of the demo.

```
[root@centos lfcs]# hostnamectl set-hostname freeipa.example.local
```

```
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.29.50 freeipa.example.local
```

```
[root@centos lfcs]# vim /etc/hosts
```

```
[root@centos lfcs]# dnf install -y ipa-server bind-dyndb-ldap ipa-server-dns
```

```
[root@centos lfcs]# ipa-server-install --setup-dns
```

```
Server host name [freeipa.example.local]:
```

```
Warning: skipping DNS resolution of host freeipa.example.local
The domain name has been determined based on the host name.
```

```
Please confirm the domain name [example.local]:
```

```
The kerberos protocol requires a Realm name to be defined.
This is typically the domain name converted to uppercase.
```

```
Please provide a realm name [EXAMPLE.LOCAL]:
```

```
Certain directory server operations require an administrative user.
This user is referred to as the Directory Manager and has full access
to the Directory for system management tasks and will be added to the
instance of directory server created for IPA.
The password must be at least 8 characters long.
```

```
Directory Manager password:
```

```
Password (confirm):
```

```
The IPA server requires an administrative user, named 'admin'.
This user is a regular system account used for IPA server administration.
```

```
IPA admin password:
```

```
Password (confirm):
```

```
Checking DNS domain example.local., please wait ...
```

```
Do you want to configure DNS forwarders? [yes]: yes
```

```
Following DNS servers are configured in /etc/resolv.conf: 192.168.29.2
```

```
Do you want to configure these servers as DNS forwarders? [yes]:
```

```
All detected DNS servers were added. You can enter additional addresses now:
```

```
Enter an IP address for a DNS forwarder, or press Enter to skip:
```

```
Do you want to search for missing reverse zones? [yes]:
```

```
Checking DNS domain 29.168.192.in-addr.arpa., please wait ...
```

```
Do you want to create reverse zone for IP 192.168.29.50 [yes]:
```

```
Please specify the reverse zone name [29.168.192.in-addr.arpa.]:
```

```
Checking DNS domain 29.168.192.in-addr.arpa., please wait ...
```

```
Using reverse zone(s) 29.168.192.in-addr.arpa.
```

```
Trust is configured but no NetBIOS domain name found, setting it now.
```

```
Enter the NetBIOS name for the IPA domain.
```

```
Only up to 15 uppercase ASCII letters, digits and dashes are allowed.
```

```
Example: EXAMPLE.
```

```
NetBIOS domain name [EXAMPLE]:
```

```
Do you want to configure chrony with NTP server or pool address? [no]:
```

```
The IPA Master Server will be configured with:
```

```
Hostname: freeipa.example.local
```

```
IP address(es): 192.168.29.50
```

```
Domain name: example.local
```

```
Realm name: EXAMPLE.LOCAL
```

```
The CA will be configured with:
```

```
Subject DN: CN=Certificate Authority,O=EXAMPLE.LOCAL
```

```
Subject base: O=EXAMPLE.LOCAL
```

```
Chaining: self-signed
```

```
BIND DNS server will be configured to serve IPA domain with:
```

```
Forwarders: 192.168.29.2
```

```
Forward policy: only
```

```
Reverse zone(s): 29.168.192.in-addr.arpa.
```

```
Continue to configure the system with these values? [no]: yes
```

```
[root@centos lfcs]# for i in https http ldaps ldap kerberos kpasswd dns ntp; do firewall-cmd --permanent --add-service $i; done
success
Warning: ALREADY_ENABLED: http
success
success
success
success
success
success
success
success
success
[root@centos lfcs]# firewall-cmd --reload
success
[root@centos lfcs]#
```

Create LDAP Users

- **kinit admin**
- **klist**
- **ipa user-add ldapuser1; ipa user-add ldapuser2**
- **ipa passwd ldapuser1; ipa passwd ldapuser2**
- Alternatively, **https://ipaserver.example.local** and login as admin with the password that was provided


```
[root@centos lfcs]# kinit admin
Password for admin@EXAMPLE.LOCAL:
[root@centos lfcs]# klist
Ticket cache: KCM:0
Default principal: admin@EXAMPLE.LOCAL
```

```
Valid starting      Expires            Service principal
02/23/2023 12:26:41 02/24/2023 11:30:01 krbtgt/EXAMPLE.LOCAL@EXAMPLE.LOCAL
```

```
[root@centos lfcs]# ipa user-add ldapuser1
```

```
First name: ldapuser1
```

```
Last name: ldapuser1
```

```
-----
Added user "ldapuser1"
-----
```

```
[root@centos lfcs]# ipa user-add ldapuser2
```

```
First name: ldapuser2
```

```
Last name: ldapuser2
```

```
-----
Added user "ldapuser2"
-----
```

```
[root@centos lfcs]# ipa passwd ldapuser1
```

```
New Password:
```

```
Enter New Password again to verify:
```

```
-----
Changed password for "ldapuser1@EXAMPLE.LOCAL"
-----
```

```
[root@centos lfcs]# ipa passwd ldapuser2
```

```
New Password:
```


Using LDAP for Authentication

- All authentication requests are handled by Pluggable Authentication Modules (PAM)
- PAM uses the `pam_sss.so` library to forward authentication requests to the `sssd` service
- The `/etc/sss/sss.conf` file and related drop-in configuration files specify by which service authentication should be further handled
- OpenLDAP has generic tools, like **`ldapsearch`** and **`ldapadd`** that allow for direct communication with the LDAP server
- **`ldapsearch -x -H ldap://192.168.29.170 -b "dc=example,dc=local" -s sub"objectclass=*`**
- FreeIPA comes with the **`ipa`** tool, which provides an easy-to-use interface

- After installing the FreeIPA server, an LDAP client has been configured as well on that server
- This LDAP client allows you to fetch information from the IPA server
- By default, the LDAP client is good for testing; for more secure usage, certificates should be configured
- Check the client configuration in `/etc/sss/sss.conf`

```
[root@centos lfcs]# ldapsearch -x -H ldap://192.168.29.50 -b "dc=example,dc=local" -s sub"objectclass=*"

```

```
[root@centos lfcs]# less /etc/sss/sss.conf

```

```
[root@centos lfcs]# su - ldapuser1

```

```
su: warning: cannot change directory to /home/ldapuser1: No such file or directory

```

```
[ldapuser1@freeipa lfcs]$ exit

```

```
logout

```

```
[root@centos lfcs]# less /etc/pam.d/system

```

```
system-auth    systemd-user

```

```
[root@centos lfcs]# less /etc/pam.d/system-auth █

```

auth	required	pam_env.so
auth	required	pam_faildelay.so delay=2000000
auth	[default=1 ignore=ignore success=ok]	pam_usertype.so isregular
auth	[default=1 ignore=ignore success=ok]	pam_localuser.so
auth	sufficient	pam_unix.so nullok
auth	[default=1 ignore=ignore success=ok]	pam_usertype.so isregular
auth	sufficient	pam_sss.so forward_pass
auth	required	pam_deny.so

↑

```
-----

```

Add-Ons

Downloading Files or Apps (wget)

Curl and Ping Commands

FTP - File Transfer Protocol

SCP - Secure Copy Protocol

System Upgrade and Patch Management

Rollback Patches and Updates

Web Server (Apache - HTTP)

Securing Linux Machine (OS Hardening)

Network File System (NFS)

Samba Installation and Configuration

Downloading Files or Apps (wget)

Understanding the `wget` Command

- `wget` stands for "world wide web get".
- Used for downloading files directly from the internet when they are not available in repositories.
- Example: If a server or repository doesn't have the required file, use `wget [file_URL]` to download it.

Curl and Ping Commands

Using Curl Command

Basics: Curl is used in command-line environments (like Linux) to interact with URLs, especially when a graphical user interface (GUI) is not available.

Functionality:

- Curl accesses URLs to check if an HTTP site is reachable.
- Returns HTML source code or other data, indicating if the page is active.

Example:

- Command: `curl http://www.google.com`
- Use: Checks if Google's homepage is accessible and displays the HTML code.
- Insight: This method shows whether the web page is functioning, even if the server is up but the page itself might be down.

Curl for File Downloading

Command: `curl -O [URL]`

Purpose: Downloads files from a specified URL.

Example:

- Command: `curl -O http://example.com/file.tar.gz`
- Use: Downloads `file.tar.gz` from `example.com`.
- Insight: Alternative to `wget`, useful when `wget` is not available.

Using Ping Command

Function: Checks if a server (e.g., Server B) is reachable from another server (e.g., Server A).

Method: Sends packets to a server's IP address or hostname and waits for a response.

Example:

- Command: `ping www.google.com` or `ping 216.58.217.68`
- Use: Verifies connectivity to Google's server.
- Insight: Helpful to determine if the issue is network connectivity or website-specific.

Advanced Usage of Ping

Continuous Ping: Different options may be required for continuous ping on various systems (e.g., `-t` or `-s` in Windows).

Example:

- Command: `ping -t www.google.com`
- Use: Continuously pings Google's server to monitor connectivity over time.
- Insight: Useful for diagnosing intermittent connectivity issues.

FTP

Introduction to FTP:

- FTP is a standard network protocol for transferring files between a client and a server on a computer network.
- It operates on a client-server model with separate control and data connections.
- Example: Transferring a file from Server A to Server B via FTP involves a set of rules (protocols) for communication.

Understanding FTP Ports and Services:

- FTP typically uses port 21, similar to how SSH uses port 22 and DNS uses port 53.
- Example: When setting up an FTP service, ensure that port 21 is open and configured for FTP traffic.

Setup Requirements:

- Two Linux machines are required for this exercise, preferably running CentOS.

Installing and Configuring FTP on the Server:

- Become root and check for internet connectivity.
- Install the vsftpd package and configure the vsftpd.conf file.
- Example: Modify the vsftpd configuration to disable anonymous login and set the welcome banner.

Firewall Considerations:

- Adjust the firewall settings to allow FTP traffic on port 21.
- Example: Adding a rule in firewalld to accept traffic on port 21 or disabling the firewall in a safe, controlled environment.

FTP User Account Setup:

- Create a user account on the FTP server for file transfers.
- Example: Using an existing account or creating a new one like 'iafzal' for FTP transfers.

Installing FTP Client on the Client Machine:

- Install the FTP client package using yum.

On the Server

Install FTP Server:

```
sudo yum install vsftpd
```

Configure FTP Server:

Edit the configuration file `/etc/vsftpd/vsftpd.conf`. You may need to disable anonymous login and make other necessary adjustments.

```
sudo nano /etc/vsftpd/vsftpd.conf
```

Here's an example change: set `anonymous_enable=NO` to disable anonymous login.

Start and Enable FTP Service:

```
sudo systemctl start vsftpd
```

```
sudo systemctl enable vsftpd
```

Adjust Firewall (if needed):

To allow FTP traffic (port 21):

```
sudo firewall-cmd --permanent --add-port=21/tcp
```

```
sudo firewall-cmd --reload
```

Create FTP User (optional, if needed):

```
sudo adduser ftpuser
```

```
sudo passwd ftpuser
```


On the Client

Install FTP Client:

```
sudo yum install ftp
```

```
sudo yum install ftp
```

Transfer a File to the Server:

To transfer a file, use the ftp command to connect to the server, login, and then use put to transfer files.

```
ftp [server_ip_address]
```

Login with the FTP user credentials.

Then, use the put command to upload a file.

```
put [filename]
```

Use bye to exit the FTP prompt after the transfer.

On the client:

```
ftp 192.168.1.100 # Replace with your server's IP address
```

Then, login with the server's FTP username and password. After successfully logging in:

```
put example.txt
```

```
bye
```

SCP - Secure Copy Protocol

Introduction to SCP

- SCP stands for Secure Copy Protocol.
- It is used for securely transferring files between a local host and a remote host.
- Example: Transferring a file from Server A to Server B using SCP instead of FTP for enhanced security and authentication.

Understanding Protocols

- Protocols are sets of rules for communication between computers.
- SCP uses the SSH (Secure Shell) protocol, specifically SSH port 22.
- Insight: SCP does not have a separate protocol but piggybacks on the security features of SSH.

Requirements for Using SCP

- Two Linux machines are needed: one as the client (source of the file) and the other as the server (destination).
- Example Setup: A Linux machine named MyFirstLinuxVM as the client and another named LinuxCentOS7 as the server.

The SCP Process

- The client wants to copy a file to the server using the SCP command.
- This transfer happens over the existing SSH protocol.
- SSH service must be running on the recipient server to accept incoming traffic.

1. Basic File Transfer from Local to Remote

Scenario: Transfer a file named `document.txt` from your local machine to a remote server.

```
scp document.txt username@remote_server_ip:/path/to/remote/directory
```

2. Transferring a Directory

Scenario: Transfer an entire directory named `my_folder` from your local machine to a remote server.

Command:

```
scp -r my_folder username@remote_server_ip:/path/to/remote/directory
```

3. Secure Copy from Remote to Local

Scenario: Copy a file from a remote server to your local machine.

Command:

```
scp username@remote_server_ip:/path/to/remote/file.txt /local/directory
```

4. Using a Specific Port

Scenario: Transfer a file using SCP on a non-standard SSH port.

Command:

```
scp -P 2222 document.txt username@remote_server_ip:/path/to/remote/directory
```

System Upgrade and Patch Management

Importance of System Upgrades and Patch Management:

- Emphasize the critical nature of regularly updating and patching Linux systems, in line with company policies.
- Example: Mention the risks of security breaches or system failures if updates are not regularly applied.

Understanding Linux Versioning:

- Highlight the difference between major and minor version upgrades in Linux.
- Example: Major version upgrade is like moving from Linux 5 to 6, while minor upgrades are from 7.3 to 7.4. Check current version using `cat /etc/redhat-release` OR `uname -a`.

Major Version Upgrade Process:

- Explain that major upgrades are not possible via the `yum` command and require a more manual approach.
- Example: For upgrading from version 6 to 7, backup the server, build a new server, and transfer files.

Minor Version Upgrade via YUM:

- Detail the process of minor version upgrades using the `yum update` command.
- Example: Run `yum update` to move from 7.3 to 7.4. The `-y` flag can be used to auto-accept prompts during the update.

Difference Between Update and Upgrade Commands:

- Clarify that `yum update` preserves old packages, whereas `yum upgrade` replaces them.
- Example: Upgrading an NTP package using `yum upgrade` will remove the old version and install the new one.

Preservation of Older Packages:

- Discuss why some administrators prefer to keep older packages.
- Example: Older versions of packages might be necessary if certain applications are incompatible with newer versions.

Dependency Management in Linux:

- Explain how Linux package management handles dependencies.
- Example: Installing a package like ABC might also require updating related packages XYZ.

Pre-Upgrade Assessment and Planning

Assess Environment and Dependencies:

- Scenario: A company uses RHEL for its critical applications. Before upgrading, they must assess not just the OS but also the applications and dependencies that might be affected.
- Method: Inventory software and hardware, check compatibility lists, and consult application vendors about support for the new RHEL version.

Backup and Recovery Plan:

- Scenario: A financial institution plans to upgrade its RHEL servers. As these servers host sensitive data, a robust backup plan is essential.
- Method: Implement a full backup strategy, including system settings, applications, and data. Ensure that backups are tested for recoverability.

Testing in a Non-Production Environment:

- Scenario: A healthcare provider plans to upgrade RHEL servers that host patient management systems. They must ensure zero downtime and no data loss.
- Method: Clone the production environment to a test environment. Perform the upgrade and test all functionalities and performance benchmarks.

Upgrade Process

Using Red Hat Upgrade Tool (RHUT):

- Scenario: An IT service company needs to upgrade multiple servers with minimal manual intervention.
- Method: Utilize RHUT, which simplifies the process. RHUT assesses the system, resolves package dependencies, and performs the upgrade.

Manual Upgrade Steps:

- Scenario: A small business with customized RHEL configurations needs a tailored upgrade approach.
- Method: Manually download the new RHEL version, update the repository, and use `yum` to upgrade packages. Carefully merge configuration files where needed.

Post-Upgrade Steps

Verification and Testing:

- Scenario: After upgrading, an e-commerce company needs to ensure that their online platform functions correctly.
- Method: Conduct thorough testing of all system components. Verify that all services start correctly and perform as expected.

Monitoring and Troubleshooting:

- Scenario: Post-upgrade, a media company experiences performance issues.
- Method: Implement monitoring tools to quickly identify and troubleshoot performance bottlenecks or failures.

Documentation and Review:

- Scenario: A university upgrades its research department's RHEL systems and needs to document the process for future reference.
- Method: Document every step, including any issues encountered and how they were resolved. This documentation is valuable for future upgrades.

Real-World Considerations

- Downtime Management: Schedule upgrades during off-peak hours to minimize impact on business operations.
- Communication: Keep stakeholders informed about the upgrade schedule, potential impacts, and post-upgrade changes.
- Compliance and Security: Ensure that the upgraded system complies with relevant security standards and company policies.

Conclusion

Upgrading RHEL in a real-world scenario is a complex process that goes beyond just executing a series of commands. It requires a holistic approach involving assessment, planning, execution, and post-upgrade management, keeping in mind the specific needs and challenges of the organization.

Patch Management

Pre-Patch Assessment and Planning

Inventory and Risk Assessment:

- Scenario: A large enterprise with diverse IT infrastructure must prioritize patches based on risk and importance.
- Method: Perform a thorough inventory of all systems and software. Use tools like Red Hat Satellite or Ansible to assess the risk level and criticality of pending patches.

Policy and Compliance Review:

- Scenario: A financial institution needs to ensure all patches comply with regulatory standards.
- Method: Review patch management policies to ensure compliance with standards like PCI DSS or HIPAA. Include audit trails and documentation as part of the process.

Testing and Validation:

- Scenario: A software company with custom applications must ensure patches don't disrupt operations.
- Method: Test patches in a controlled environment that mirrors production. Verify that patches don't introduce new vulnerabilities or break existing functionalities.

Patch Deployment Process

Automated Patch Management Tools:

- Scenario: An organization with a large number of RHEL servers seeks to streamline patching processes.
- Method: Utilize tools like Red Hat Satellite or Ansible for automated patch deployment. These tools can schedule, manage, and report on the patching process across multiple servers.

Manual Patch Application:

- Scenario: A small business with limited IT resources requires a more hands-on approach.
- Method: Manually apply patches using `yum update` or `rpm` commands. Carefully follow release notes and documentation for each patch.

Post-Patch Steps

Verification and Monitoring:

- Scenario: A healthcare provider needs to ensure system stability post-patch.
- Method: Monitor systems for any irregularities or performance issues post-patch. Use monitoring tools to track system health and logs.

Rollback Strategies:

- Scenario: An update causes issues in a critical system.
- Method: Have a rollback plan in place. Use snapshots or backups taken before patching to revert the system to its pre-patch state if necessary.

Reporting and Documentation:

- Scenario: An organization needs to maintain records for compliance and auditing purposes.
- Method: Document every patch applied, along with details of the process, testing, and any issues encountered. Use patch management tools to generate reports.

Real-World Considerations

- Scheduled Patching: Plan patching during low-traffic hours to minimize impact on business operations.
- Stakeholder Communication: Inform relevant parties about the schedule, potential impacts, and outcomes of the patching process.
- Continuous Improvement: Review and refine patch management processes regularly based on feedback and lessons learned from each cycle.

Conclusion

Real-world patch management in RHEL environments is a multi-faceted process that goes beyond simply applying updates. It involves careful planning, testing, and execution, followed by post-patch monitoring and documentation. Ensuring minimal disruption to services and maintaining compliance are key aspects of effective patch management.

Red Hat Satellite

Centralized Management:

- **Functionality:** Red Hat Satellite provides a central point for managing Red Hat Enterprise Linux environments. It simplifies patch management by offering a single interface to manage multiple systems.
- **Implementation:** Administrators can use Satellite to view available Red Hat patches (errata), assess their applicability to systems in their environment, and determine their criticality.

Automated Patch Deployment:

- **Functionality:** Satellite enables the automation of patch deployment to registered systems.
- **Implementation:** Set up schedules for automatic patch updates. Create policies that automatically apply critical security updates while scheduling less critical updates for a more convenient time.

Reporting and Auditing:

- **Functionality:** Satellite provides detailed reporting on the patch status of all managed systems.
- **Implementation:** Use these reports for compliance auditing and to ensure all systems are up to date. Custom reports can be generated to meet specific organizational needs.

Configuration Management:

- **Functionality:** Satellite also aids in maintaining consistent configurations across the IT environment.
- **Implementation:** Enforce configuration standards, ensuring that systems are not only up to date with patches but also configured according to organizational policies.

Ansible

Infrastructure as Code for Patch Management:

- **Functionality:** Ansible uses simple YAML-based playbooks to automate IT processes, including patch management.
- **Implementation:** Write playbooks that define the desired state of your systems, including which packages should be updated or installed.

Ad-Hoc Patch Deployment:

- **Functionality:** Ansible allows for quick, ad-hoc commands for immediate patch application.
- **Implementation:** Use Ansible ad-hoc commands to apply critical patches immediately across all relevant systems.

Customization and Flexibility:

- **Functionality:** Ansible's modular design allows for a high degree of customization to fit specific workflows and environments.
- **Implementation:** Develop custom modules or playbooks that align with your unique patch management process, including pre-patch checks, post-patch validations, and complex deployment sequences.

Integration with Existing Systems:

- **Functionality:** Ansible easily integrates with other systems and tools.
- **Implementation:** Combine Ansible with other tools like monitoring solutions or custom scripts to create a comprehensive patch management workflow.

Web Server (Apache - HTTP)

Introduction to Web Servers (httpd):

- Web servers serve pages on the internet, like Google, Facebook, etc.
- Example: Setting up your own server to host a website instead of using services like GoDaddy or Wix.com.

Understanding HTTPD Service in Linux:

- The HTTPD package needs to be installed on Linux to run a web server.
- Configuration file: `/etc/http/conf/httpd.conf` - used for settings like user directories, protocols, etc.
- Example: Installing HTTPD with `yum install httpd` and making changes in the `httpd.conf` file.

Setting Up the Home Page:

- The home page is located at `/var/www/html/index.html`.
- Any changes made to this file will be reflected on the website's home page.
- Example: Editing `index.html` to change the content of the website's home page.

Starting, Restarting, and Enabling HTTPD Service:

- Start or restart the service using `systemctl restart httpd`.
- Enable the service to start on boot using `systemctl enable httpd`.
- Example: Running `systemctl restart httpd` to apply changes made to the web server configuration.

Accessing Log Files for Troubleshooting:

- Log files are located at `/var/log/httpd/`.
- Useful for diagnosing server issues.
- Example: Checking the log files to troubleshoot server connection issues.

Building a Basic Web Page:

- Basic HTML structure: DOCTYPE, HTML, Body, and text/content.
- Example: Creating a simple HTML page with headers and text, e.g., "Welcome to My First Page" and "My name is Imran Afzal."

Accessing the Web Page via Browser:

- Use the server's IP address to access the web page if DNS is not set up.
- Example: Entering `192.168.1.12` in a browser to view the newly created web page.

Handling Firewalls:

- Linux machines have firewalls that may block web traffic.
- Disable the firewall or set rules to allow traffic on port 80.
- Example: Running `systemctl stop firewalld` to disable the firewall temporarily.

Starting the HTTPD Service:

- Ensure the HTTPD service is running to serve the web page.
- Example: Using `systemctl start httpd` to start the web server service.

Modifying and Viewing Changes on the Web Page:

- Edit `index.html` to make changes to the web page.
- Refresh the browser to see updates.
- Example: Changing the text in `index.html` to "I DID IT..YAAAY!" and refreshing the browser to see the new message.

Securing Linux Machine

Account Management:

- Avoid standard usernames (like 'admin' or 'Oracle') to reduce predictability. Use variations like 'tadmin' or '123oracle'.
- Example: To list users, use `cat /etc/passwd`. For user 'spiderman', consider renaming to 'spiderman123'.
- Assign user IDs starting from 10,000 for added security.

Password Policies:

- Implement policies for password expiration and complexity.
- Example: Use `chage -l username` to check password settings for a user. Adjust settings in `/etc/login.defs` and `/etc/shadow`.

Removing Unwanted Packages:

- Uninstall unnecessary packages to reduce vulnerabilities.
- Example: Use `rpm -qa` to list installed packages and `rpm -e package_name` to remove them.

Stopping Unused Services:

- Disable services that are not required for your system's operation.
- Example: List services with `systemctl -a` and stop unnecessary ones using `systemctl stop service_name`.

Checking Listening Ports:

- Close or secure open ports that are not in use.
- Example: Use `netstat -tunlp` to list open ports and identify unnecessary ones for closure.

Securing SSH Configuration:

- Modify SSH settings for enhanced security, such as changing default port and disabling root login.
- Example: Edit `/etc/ssh/sshd_config`, change 'Port 22' to a non-standard port, and set 'PermitRootLogin no'.

Enabling and Configuring Firewall:

- Use firewalls like iptables or firewalld to control incoming and outgoing traffic.
- Example: Use `firewall-cmd` or `iptables -list` to manage firewall rules.

SELinux (Security Enhanced Linux):

- Utilize SELinux for managing access rights of users, applications, and processes.
- Example: Check SELinux status with `sestatus`. Configure SELinux modes in `/etc/sysconfig/selinux`.

Changing Listening Service Port Numbers:

- Alter default port numbers for services to obscure them from attackers.
- Example: In SSH configuration, change the default port 22 to a less common port like 2022.

Keeping the System Up-to-Date:

- Regularly update the system, focusing on security patches.
- Example: Use `yum update` to install updates. Subscribe to security alerts from distributions like Red Hat or CentOS.

NFS

Introduction to NFS:

- NFS stands for Network File System, developed by Sun Microsystems.
- It allows sharing of files and directories over a network.
- NFS is a part of Network Attached Storage (NAS) protocols.
- Example: NFS is used in a Unix environment like the Solaris operating system, now owned by Oracle.

How NFS Works:

- NFS operates on a client-server model.
- Files on a server can be accessed across the network as if they were local.
- Example: A file in a server's directory can be accessed from a client machine seamlessly.

Exporting and Mounting in NFS:

- Exporting: The process by which a server provides access to its file system.
- Mounting: The client-side process of accessing the shared file system.
- Example: A server exports a directory, and a client mounts it to appear as a local directory.

Setting Up NFS Server:

Configuring NFS Client:

NFS Server Commands

- 1 `sudo apt update`
- 2 `sudo apt install nfs-kernel-server`
- 4 `sudo mkdir /nfsshare`
- 5 `sudo chown nobody:nogroup /nfsshare`
- 6 `sudo chmod 777 /nfsshare`
- 7 `sudo nano /etc/exports → /nfsshare *(rw,sync,no_subtree_check)`
- 8 `sudo exportfs -a`
- 9 `sudo systemctl restart nfs-kernel-server`
- 10 `sudo ufw allow from 172.31.26.254 to any port nfs`
- 11 `sudo ufw enable`
- 12 `sudo ufw status`
- 15 `ip a`
- 16 `cd /nfsshare/`
- 17 `touch India`

NFS Client Commands

- 1 ip a
 - 2 sudo apt update
 - 3 sudo apt install nfs-common
 - 4 sudo mkdir /nfsshare
 - 5 sudo mount 172.31.28.220:/nfsshare /nfsshare
 - 6 sudo nano /etc/fstab
- 172.31.28.220:/nfsshare /nfsshare nfs defaults 0 0
- 7 cd /nfsshare/
 - 8 ls

Samba Installation

Samba is a free software re-implementation of the SMB/CIFS networking protocol, and it allows for file and print sharing between computers running Windows and those running Unix or Linux. Samba can also act as a Windows domain controller.

```
20 sudo apt update
21 sudo apt install samba
22 sudo cp /etc/samba/smb.conf /etc/samba/smb.conf.backup
23 sudo nano /etc/samba/smb.conf
```

[tifshare]

path = /smbshare

read only = no

browsable = yes

```
24 mkdir /smbshare
26 sudo useradd smbuser
27 sudo smbpasswd -a smbuser
28 sudo systemctl restart smbd
29 sudo systemctl restart nmbd
30 sudo ufw allow samba
31 ip a
32 cd
34 cd /smbshare/
35 touch q w e r
```

Samba client

```
12 apt install smbclient
```

```
13 smbclient //172.31.28.220/tifshare -U smbuser
```

```
smb: \> ls
```

.	D	0 Tue Dec 12 15:35:25 2023
..	D	0 Tue Dec 12 15:29:41 2023
e	N	0 Tue Dec 12 15:35:25 2023
r	N	0 Tue Dec 12 15:35:25 2023
q	N	0 Tue Dec 12 15:35:25 2023
w	N	0 Tue Dec 12 15:35:25 2023

- From a Windows Client: Open File Explorer and enter `\\server_ip\sharename` in the address bar. Replace `server_ip` and `sharename` with the appropriate values.
- From a macOS Client: In Finder, go to Go > Connect to Server, then enter `smb://server_ip/sharename`.

Linux Commands End to End Cheat Sheet

Command: `whoami`

- Description: Displays the current user account.
- Example: Running `whoami` in the terminal will show your username, like `john_doe`.

Command: `ls`

- Description: Lists files and directories in the current directory.
- Example: `ls` will display files and folders in your current directory, and `ls -l` provides detailed information.

Command: `ip a`

- Description: Lists your IP address and network interfaces.
- Example: `ip a` shows all network interfaces and their IP addresses.

Command: `cat`

- Description: Reads the contents of text files.
- Example: `cat myfile.txt` will display the content of `myfile.txt`.

Command: `passwd`

- Description: Used to change the password of the user account.
- Example: Typing `passwd` and following the prompts will allow you to change your password.

Command: `touch`

- Description: Creates empty files, mainly useful to test write ability.
- Example: `touch newfile.txt` creates an empty file named `newfile.txt`.

Command: `pwd`

- Description: Stands for Print Working Directory; shows which directory you are in.
- Example: Running `pwd` might return `/home/john_doe`.

Command: `man`

- Description: Accesses the system documentation for commands.
- Example: `man ls` will show the manual for the `ls` command.

Command: `apropos`

- Description: Searches commands related to a specific task based on description (equivalent to `man -k`).
- Example: `apropos "list files"` might return commands related to listing files.

Command: `pinfo`

- Description: Provides additional information, similar to `man`.
- Example: `pinfo ls` gives detailed information about the `ls` command.

Command: `tldr`

- Description: Shows command line examples, a simplified version of `man`.
- Example: `tldr passwd` provides quick examples of how to use the `passwd`

Essential File Management Tools Commands

Command: `ls`

- Example: `ls /home/user/Documents`
- Description: Lists files and directories within a specified path. This command is used to view the contents of directories.

Command: `cp`

- Example: `cp source.txt destination.txt`
- Description: Copies files from one location to another. This command is used to make duplicates of files.

Command: `mkdir`

- Example: `mkdir new_directory`
- Description: Creates a new directory. This command is used when you need to make a new folder.

Command: `rmdir`

- Example: `rmdir empty_directory`
- Description: Removes a directory, but only if it's empty. This command is used for deleting empty folders.

Command: `mv`

- Example: `mv oldname.txt newname.txt`
- Description: Moves or renames files and directories. This command can change the location or the name of a file.

Command: `rm`

- Example: `rm filename.txt`
- Description: Removes files or directories. This command deletes files or directories.
- Note: When combined with options `-r` (recursive) and `-f` (force), as in `rm -rf directory_name`, it becomes a powerful command that can delete directories and their contents without asking for confirmation.

Advanced File Management Tools Commands

ln: Used for creating hard and symbolic links.

- Example: `ln -s source_file symbolic_link` creates a symbolic link to `source_file`.
- Description: Links a file or directory to another location.

find: A utility to find files based on various properties.

- Example: `find /home -name "*.txt"` searches for text files in the `/home` directory.
- Description: Searches for files and directories in a given path.

locate: Used for quickly searching the database of system's contents.

- Example: `locate filename` finds the location of `filename`.
- Description: Fast file searching, requires running `updatedb` for an updated database.

which: Finds the location of executables in the PATH.

- Example: `which python` shows the path of the Python executable.
- Description: Useful for finding executable files in system's PATH.

tar: Archives multiple files into a single file.

- Example: `tar -cvf archive.tar /folder` creates an archive of a folder.
- Description: Combines multiple files into a tarball.

gzip: Compresses files.

- Example: `gzip file.txt` compresses `file.txt`.
- Description: Reduces the size of files using compression.

bzip2: A file compression utility.

- Example: `bzip2 file.txt` compresses `file.txt`.
- Description: Offers better compression ratios than `gzip` at the cost of speed.

xz: High compression utility.

- Example: `xz file.txt` compresses `file.txt`.
- Description: Provides the best compression ratio.

zip: Used for packaging and compressing (archiving) files.

- Example: `zip archive.zip file1 file2` creates an archive with `file1` and `file2`.
- Description: Commonly used for creating `.zip` files.

file: Determines the type of a file.

- Example: `file image.jpg` displays the type of `image.jpg`.
- Description: Useful for identifying file formats.

mount: Attaches a filesystem to a specified directory.

- Example: `mount /dev/sdb1 /mnt` mounts the device `sdb1` to `/mnt`.
- Description: Used to access filesystems on different storage devices.

df: Displays disk space usage of all mounted filesystems.

- Example: `df -h` shows disk space in human-readable format.
- Description: Useful for monitoring disk space usage.

findmnt: Lists all mounted filesystems.

- Example: `findmnt -t ext4` lists all `ext4` type mounts.
- Description: Displays a tree-like structure of mounts.

lsblk: Lists information about block devices.

- Example: `lsblk` lists all block devices.
- Description: Useful for identifying storage devices and their partitions.

Working with Text Files Commands

nano

- Description: A user-friendly text editor.
- Example: `nano filename.txt` - Opens `filename.txt` in nano editor.

vimtutor

- Description: A tutorial program for learning VIM, an advanced text editor.
- Example: `vimtutor` - Starts the VIM tutorial.

vi

- Description: The original text editor, with basic features.
- Example: `vi filename.txt` - Opens `filename.txt` in VI editor.

vim

- Description: An improved version of VI with more features.
- Example: `vim filename.txt` - Opens `filename.txt` in VIM editor.

more

- Description: A pager to view the contents of text files, one screen at a time.
- Example: `more filename.txt` - Views the content of `filename.txt`.

less

- Description: Similar to more, but provides more control over viewing text files.
- Example: `less filename.txt` - Opens `filename.txt` for viewing with navigation options.

head

- Description: Displays the first few lines of a text file.
- Example: `head filename.txt`- Shows the top lines of `filename.txt`.

tail

- Description: Displays the last few lines of a text file.
- Example: `tail filename.txt`- Shows the last lines of `filename.txt`.

cat

- Description: Concatenates and displays the entire content of text files.
- Example: `cat filename.txt`- Displays the full content of `filename.txt`.

cat -A

- Description: Similar to `cat`, but also displays special characters.
- Example: `cat -A filename.txt`- Shows the content of `filename.txt`, including special characters.

tac

- Description: Prints the contents of a text file in reverse order.
- Example: `tac filename.txt`- Displays `filename.txt` content backwards.

grep

- Description: Searches for text in files or command output.
- Example: `grep 'pattern' filename.txt`- Searches for 'pattern' in `filename.txt`.

Advanced Text File Processing Commands

grep:

- Command: "grep"
- Example: "grep 'pattern' file.txt"
- Description: Filters text in files by searching for specific patterns.

awk:

- Command: "awk"
- Example: "awk '/pattern/ {print}' file.txt"
- Description: Filters and processes text, with additional features for text manipulation.

sed:

- Command: "sed"
- Example: "sed 's/old/new/' file.txt"
- Description: A stream editor used for modifying text files directly from the command line.

tr:

- Command: "tr"
- Example: "tr '[:lower:]' '[:upper:]' < file.txt"
- Description: Translates or deletes characters from input text.

cut:

- Command: "cut"
- Example: "cut -d':' -f1 file.txt"
- Description: Extracts sections from each line of files, such as columns from a table.

sort:

- Command: "sort"
- Example: "sort file.txt"
- Description: Sorts text files in alphabetical or numerical order, as specified.

Connecting to a Server Commands

su Command

- Example: `su username`
- Description: This command is used to switch user identity in a Unix-like operating system. When you type `su` followed by a username, it prompts you for the password of that user and, if entered correctly, logs you in as that user.

sudo Command

- Example: `sudo apt update`
- Description: The `sudo` command allows a permitted user to execute a command as the superuser or another user, as specified in the `sudoers` file. This example updates the list of available packages and their versions, but it requires administrative privileges, hence the use of `sudo`.

ssh Command

- Example: `ssh username@hostname`
- Description: `ssh` stands for Secure Shell. It's used to connect and log into a remote machine securely. The example shows how to connect to a remote host where `username` is your username on the remote machine, and `hostname` is the address of the remote machine.

scp Command

- Example: `scp file.txt username@hostname:/path/to/destination`
- Description: The `scp` command is used for secure copy from one machine to another. In the example, `file.txt` is being copied to a remote host. The `username@hostname` specifies the user and host of the remote machine, and `/path/to/destination` is where the file will be placed on the remote machine.

Working with the Bash Shell commands

tee

- Example: `ls | tee file.txt`
- Description: Used in pipes to direct the output of a command both to a file and the next command in the pipe.

history

- Example: `history`
- Description: Displays a list of previously executed commands.

history -d

- Example: `history -d 1023`
- Description: Deletes a specific command from the history, where `1023` is the history entry number you want to remove.

alias

- Example: `alias ll='ls -l'`
- Description: Creates a shortcut for a longer command. In this example, typing `ll` will execute `ls -l`.

export

- Example: `export VAR=value`
- Description: Sets an environment variable `VAR` to a value `value` and makes it available to subshells.

User and Group Management and Permissions Commands

useradd: Adds users.

- Example: `useradd johndoe`
- Description: Creates a new user account named 'johndoe'.

usermod: Modifies user properties.

- Example: `usermod -L johndoe`
- Description: Locks the user account 'johndoe'.

userdel: Deletes users.

- Example: `userdel johndoe`
- Description: Removes the user account 'johndoe'.

groupadd: Adds groups.

- Example: `groupadd developers`
- Description: Creates a new group named 'developers'.

groupmod: Modifies group properties.

- Example: `groupmod -n newname oldname`
- Description: Renames a group from 'oldname' to 'newname'.

groupdel: Deletes groups.

- Example: `groupdel developers`
- Description: Removes the group named 'developers'.

id: Shows information about user accounts.

- Example: `id johndoe`
- Description: Displays user and group IDs for 'johndoe'.

getent: Gets information from administrative databases.

- Example: `getent passwd johndoe`
- Description: Retrieves user 'johndoe' details from the passwd database.

passwd: Changes passwords.

- Example: `passwd johndoe`
- Description: Sets or changes the password for 'johndoe'.

chage: Modifies password properties.

- Example: `chage -E 2023-12-31 johndoe`
- Description: Sets the expiry date of 'johndoe's password.

w: Shows logged-in users and details.

- Example: `w`
- Description: Lists users currently logged in with session details.

who: Shows currently logged-in users.

- Example: `who`
- Description: Lists users currently logged in with minimal details.

vipw: Edits the passwd file.

- Example: `vipw`
- Description: Opens the password file for editing.

vigr: Edits the /etc/group file.

- Example: `vigr`
- Description: Opens the group file for editing.

loginctl: Manages user sessions.

- Example: `loginctl list-sessions`
- Description: Lists current user sessions.

Permissions Management commands

chown

- Example: `chown user:group filename`
- Description: Changes the ownership of a file or directory. The command alters both user and group ownership. It's a versatile command that eliminates the need for `chgrp` in most cases.

chgrp

- Example: `chgrp group filename`
- Description: Changes the group ownership of a file or directory. While this command is dedicated to changing group ownership, `chown` can also perform this function.

chmod

- Example: `chmod 755 filename`
- Description: Changes the permission mode of a file or directory. It's an essential command for managing file and directory permissions in terms of read, write, and execute rights.

diff

- Example: `diff file1 file2`
- Description: Compares two files and displays the differences between them. This command is particularly useful in scenarios like checking changes in configuration files or script versions.

find (in the context of set user ID)

- Example: `find / -perm /4000`
- Description: Searches for files with set user ID permissions. This is important for security and system administration, to locate files that might have special privileges.

umask

- Example: `umask 022`
- Description: Sets the default permissions for newly created files and directories. It's a foundational command for establishing baseline security and access controls.

Storage Management Essentials commands

lsblk

- Description: Lists all block devices.
- Example: `lsblk`
- This command displays information about all block devices connected to the system, such as hard drives and USB drives.

fdisk

- Description: Manages partitions on an MBR disk.
- Example: `fdisk /dev/sda`
- Used for creating and managing disk partitions on an MBR (Master Boot Record) formatted disk.

gdisk

- Description: Manages partitions on a GPT disk.
- Example: `gdisk /dev/sda`
- Similar to `fdisk` but for GPT (GUID Partition Table) formatted disks, which are more modern.

mkfs

- Description: Creates a filesystem.
- Example: `mkfs.ext4 /dev/sda1`
- A generic utility to format a partition with a specific file system, such as ext4, xfs, or btrfs.

mount

- Description: Attaches a filesystem to the file tree.
- Example: `mount /dev/sda1 /mnt`
- This command is used to mount a filesystem found on a device to a specific directory in the file tree.

findmnt

- Description: Lists mounted filesystems.
- Example: `findmnt`
- Provides a detailed overview of all the filesystems currently mounted in the system.

umount

- Description: Detaches the filesystem from the file tree.
- Example: `umount /mnt`
- Used to unmount a previously mounted filesystem.

lsof

- Description: Lists open files and the processes using them.
- Example: `lsof /dev/sda1`
- Useful for diagnosing why a device might be busy, particularly if trying to unmount it and receiving a "device is busy" message.

Managing Networking commands

Command: `ip a`

- Example: `ip a`
- Description: Prints current IP address information, showing the IP addresses assigned to all network interfaces.

Command: `ip route show`

- Example: `ip route show`
- Description: Displays the routing table, including the default route, which is essential for understanding network routes.

Command: `ping`

- Example: `ping google.com`
- Description: Tests connectivity to a network host (e.g., a website like google.com) by sending ICMP packets.

Command: `ss`

- Example: `ss -tuln`
- Description: Shows socket statistics. Used to check open ports and listening services on the system.

Command: `dig`

- Example: `dig example.com`
- Description: Queries DNS information. It's useful for troubleshooting DNS issues or verifying DNS records.

Command: `nmap`

- Example: `nmap -v 192.168.1.1`
- Description: An advanced security analysis tool that scans networks and systems for open ports, services, and other security vulnerabilities.

Command: `hostname`

- Example: `hostname`
- Description: Displays the system's host name.

Command: `hostnamectl`

- Example: `hostnamectl set-hostname myhostname`
- Description: Used for host name management, including viewing and changing the system's hostname.

Working with Systemd commands

systemctl start

- Example: `systemctl start httpd`
- Description: Starts a specified unit file.

systemctl status

- Example: `systemctl status httpd`
- Description: Displays the current status of a unit file.

systemctl restart

- Example: `systemctl restart httpd`
- Description: Restarts a unit file, often used after making configuration changes.

systemctl stop

- Example: `systemctl stop httpd`
- Description: Stops a running unit file.

systemctl enable

- Example: `systemctl enable httpd`
- Description: Enables a unit file to start automatically on system boot.

systemctl disable

- Example: `systemctl disable httpd`
- Description: Disables a unit file from starting automatically on boot.

systemctl list-units

- Example: `systemctl list-units`
- Description: Lists all currently loaded unit files.

systemctl set-default

- Example: `systemctl set-default graphical.target`
- Description: Sets the default target that the system boots into.

systemctl get-default

- Example: `systemctl get-default`
- Description: Displays the current default target.

systemctl cat

- Example: `systemctl cat httpd`
- Description: Shows the configuration of a unit file.

systemctl show

- Example: `systemctl show httpd`
- Description: Displays all the options set for a unit file.

systemctl edit

- Example: `systemctl edit httpd`
- Description: Opens an editor to modify a unit file.

systemctl daemon-reload

- Example: `systemctl daemon-reload`
- Description: Instructs the systemctl main process to reload its configuration.

systemctl isolate

- Example: `systemctl isolate graphical.target`
- Description: Switches the system to a different target.

systemctl list-dependencies

- Example: `systemctl list-dependencies graphical.target`
- Description: Lists all dependencies of a specific target.

Managing Software Commands

Command: `rpm`

- Description: Originally used for installing packages, `rpm` is now mainly used for querying information about installed packages.
- Example: `rpm -q package_name` - This command queries details about a specific package.

Command: `yum`

- Description: `yum` is a software package manager used in Red Hat-based environments for installing, updating, and managing packages.
- Example: `yum install package_name` - This command installs a specific package.

Command: `dnf`

- Description: Similar to `yum`, `dnf` is a modern package manager for Red Hat-based systems, offering improved performance and package management capabilities.
- Example: `dnf update` - This command updates all installed packages to their latest versions.

Command: `apt`

- Description: `apt` is a command-line tool used in Ubuntu and other Debian-based systems for handling packages, including installation, updating, and removal.
- Example: `apt install package_name` - This command installs a specific package.

Command: `apt-cache`

- Description: `apt-cache` is used for querying package information in Debian-based systems. It's especially useful for searching for packages containing specific files.
- Example: `apt-cache search keyword` - This command searches for packages related to a specific keyword.

Managing SSH Commands

Change SSH Port on Ubuntu:

- Command: `sudo vim /etc/ssh/sshd_config`
- Description: Open the SSH configuration file using `vim` to change the SSH port.

Update SSH Port to 2022:

- Command: Change the line `Port 22` to `Port 2022` in the file.
- Description: In the SSH configuration file, update the default port from 22 to 2022.

Restart SSH Service:

- Command: `sudo systemctl restart sshd`
- Description: Restart the SSH daemon to apply the new configuration.

Allow Port 2022 in Firewall:

- **Command:** `sudo ufw allow 2022/tcp`
- Description: Update the Ubuntu firewall to allow TCP connections on port 2022.

Check SSH Service Status:

- **Command:** `sudo systemctl status sshd`
- Description: Verify that the SSH service is active and listening on port 2022.

Create SSH Session from Windows:

- Description: Use MobaXterm on Windows to create a new SSH session.
- Steps:
 - Open MobaXterm and select to create a new SSH session.
 - Enter the remote host IP address (e.g., `192.168.29.142`).
 - Specify the username (`student` in this case).
 - Set the port to 2022.
 - Click OK to initiate the connection, then enter the password when prompted.

Managing Time commands

Command: `date`

- Example: `date`
- Description: Displays the current date and time of the system. It's a basic and long-standing command in many operating systems.

Command: `hwclock`

- Example: `hwclock --show`
- Description: Used for managing and viewing the hardware clock. It can also be used to synchronize the hardware clock with the system clock.

Command: `timedatectl`

- Example: `timedatectl status`
- Description: A modern utility for managing various aspects of time synchronization and settings. It can display and set time, date, and timezone, among other features.

Command: `chronyc`

- Example: `chronyc tracking`
- Description: Interfaces with the `chronyd` time-synchronizing service, providing detailed information about the current synchronization status and more control over the `chronyd` service.

Process Management Commands

Command: `top`

- Example: `top`
- Description: Monitors all processes and the general health of the system from a performance perspective.

Command: `ps`

- Example: `ps aux`
- Description: Shows all processes and their properties; useful for monitoring system processes.

Command: `jobs`

- Example: `jobs`
- Description: Lists the jobs (processes) started by the user in the shell, allowing for management of interactive processes.

Command: `fg`

- Example: `fg %1`
- Description: Brings a job to the foreground, where `%1` is the job number.

Command: `bg`

- Example: `bg %1`
- Description: Sends a job to run in the background.

Command: `nice`

- Example: `nice -n 10 command`
- Description: Starts a process with a defined niceness (priority), where `command` is the command to be run with a priority.

Command: `renice`

- Example: `renice -n 10 -p 1234`
- Description: Changes the priority of an already running process, where `1234` is the Process ID (PID).

Command: `kill`

- Example: `kill 1234`
- Description: Sends a signal to a process (usually to terminate it), where `1234` is the Process ID (PID).

Command: `killall`

- Example: `killall processname`
- Description: Terminates all processes with the given name.

Scheduling Tasks Commands

at Command

- Description: Allows you to schedule a job to run at a specific time.
- Example: `at 10:30` schedules a job to run at 10:30.

atq Command

- Description: Displays a list of all jobs currently scheduled.
- Example: `atq` shows all scheduled jobs with their job numbers.

atrm Command

- Description: Used to remove scheduled jobs.
- Example: `atrm 5` removes the job with the job number 5.

Editing Cron Configuration with `crontab -e`

- Description: Opens the cron configuration for editing, allowing you to schedule jobs to run periodically.
- Example: `crontab -e` opens the cron configuration file in a text editor.

Working with Systemctl Timers Using `systemctl`

- Description: Used to manage and interact with systemctl timers, a more systemwide approach for scheduling tasks.
- Example: `systemctl list-timers` shows all active timers.

Reading Log Files Commands

Command: `journalctl`

- Description: This command is used to check the contents of the systemd journal, which is a centralized logging system for Linux.
- Example: `journalctl -u nginx.service`
 - This example shows how to view logs for the Nginx service using `journalctl`.

Command: `tail`

- Description: The `tail` command is used to view the contents of log files, typically the last few lines.
- Example: `tail -n 100 /var/log/syslog`
 - This example displays the last 100 lines of the syslog file.

Command: `less`

- Description: Similar to `tail`, `less` is used for viewing log files but allows for more controlled navigation through the file.
- Example: `less /var/log/apache2/access.log`
 - This opens the Apache access log file in a scrollable interface for in-depth examination.

Command: `logger`

- Description: The `logger` command is a convenient tool for writing messages to the system log.
- Example: `logger "System backup completed"`
 - This example sends a custom message, "System backup completed," to the system log.